University of Lagos

An Explainable Prolog Expert System for Fault Diagnosis and Corrective Maintenance of Data Centre Cooling and Power Infrastructure

Ibrahim Asogba

Student number: 259047013

A capstone project report submitted in partial fulfilment of the requirements of the degree of MSc Systems Engineering

Supervisor: Dr. J. A. Ogbemhe

October 29, 2026

Declaration

Declaration of originality

I declare that this report and the software that accompanies it are submitted for Expert Systems Application to Engineering Problems (SSG816) in the MSc Systems Engineering programme at the University of Lagos.

Declaration of the use of generative AI

Tool. Claude, a generative AI model developed by Anthropic (model Claude Opus 5.5), used through Claude Code, an AI coding assistant running in Visual Studio Code on my own computer, between 7 October 2026 and 05 November 2026.

What the AI tool did. The tool was used extensively, at every stage of the project, under my direction:

- **Planning.** It explained the requirements of the project framework and proposed candidate engineering systems. I chose the cooling and power infrastructure of a data centre, and my lecturer approved the topic on 20 October 2026.
- **Design and engineering model.** It drafted the system definition, the fault taxonomy, the observation variables, the knowledge-acquisition log and the equation set (the design documents in `docs/`).
- **Software.** It wrote most of the Prolog knowledge base and inference engines, the Python physics simulator, and the test, evaluation and reproduction scripts. It ran them on my computer as directed.
- **Evaluation.** It designed and ran the experiments of Chapter 4. That includes the significance tests, the model-mismatch, explanation-fidelity, parameter-sensitivity and multiple-seed experiments, and the data-driven baseline, several of which it proposed in a critical review of the draft that it also wrote.
- **Report.** It drafted the text of every chapter and of the abstract, the TikZ figures, and the scripts that generate the tables from the code.
- **Literature.** It proposed the sources and drafted the bibliography entries. It then checked every entry against online records (Crossref and publisher, library and repository records) and corrected the errors it found. I verified each entry, and what the report cites it for, against the original source.

My own contribution. I chose the engineering system and its scope, after asking for an alternative better suited to my software-engineering background, and obtained my lecturer's approval. I directed each stage of the work, decided which of the tool's suggestions and review items to act on, and provided the project and title-page details. I set up the software environment (SWI-Prolog, Python and the LaTeX tools), with the tool installing some components at my request, and ran the system and its reproduction checks on my computer. I reviewed the declaration and corrected the claims in it. I verified the references against the original sources and worked through the report and code so that I can explain and defend every part of it.

Verification. The results were verified with an automated reproduction script, `tests/reproduce.py`, which regenerates the datasets from their seeds and checks every reported count; all 35 checks passed when it was last run, on 05 November 2026. The bibliography entries were checked against online records. I checked the engineering model and its thresholds against their sources, and each reference against the original source.

Records. The full transcripts of my sessions with the AI tool have been kept and can be provided on request.

Responsibility. I have read and checked the whole report and the software, I take responsibility for their content, and I can explain and defend every part of them.

Signed: Ibrahim Asogba Date: 05 November 2026
Ibrahim Asogba, student number 259047013

Abstract

Faults in the cooling and power infrastructure of a data centre can throttle or shut down the servers it supports, yet monitoring data rarely say directly what has failed: different faults share symptoms, sensors can be wrong, and readings are noisy or missing. This project asks whether engineering knowledge can be represented formally and reasoned over in Prolog to produce explainable, defensible diagnoses and appropriate corrective-maintenance decisions.

An expert system was developed for one row of a data centre, covering eight equipment faults and two instrumentation faults across the cooling unit, racks, servers, power distribution, uninterruptible power supply (UPS) and battery. Its knowledge base holds a 60-equation engineering model, 47 diagnostic rules with sourced thresholds, causal signatures and 20 analytical redundancy relations with fault models. Deductive, abductive and consistency-based reasoning are combined, and every diagnosis is explained and linked to a corrective plan. Because no measured fault data were available, the system was tested on synthetic cases from a physics simulator, with separate calibration and held-out datasets.

On 255 held-out cases, exact-match accuracy was 91.0% to 94.5% across four reasoning configurations, every normal, single-fault and instrumentation case was diagnosed correctly, and every diagnosis was explained, at about a millisecond per case. Over five held-out seeds (1275 cases), deductive plus abductive reasoning was the most accurate configuration at 93.6%, ahead of the complete system (92.9%), deductive plus consistency-based reasoning (91.9%) and deduction alone (90.8%); its lead over each was significant after correction for multiple comparisons. The complete system fell behind deductive plus abductive reasoning only in cases where consistency checking added a false sensor alarm, mostly under heavy noise. Abduction with noise-adaptive evidence was the most robust configuration: at four times the nominal sensor noise it kept 73% exact match against 42% for deduction alone. Consistency checking separated sensor faults from equipment faults, but was fragile under heavy noise and to mis-set relation thresholds. When the simulator's physics was changed by 10 to 25%, the abductive configurations lost about four points while deduction lost ten. Larger changes made every configuration fail, because fixed thresholds encode the nominal normal envelope, until three thresholds were re-baselined from normal operating data, which restored about 83%. Standard classifiers trained on the raw readings reached at most 89% even with ten times the labelled data, and about 50% under mild model mismatch, without explaining anything. The results support a qualified yes to the research question, qualified because validation on measured data remains to be done and thresholds must be calibrated to each site.

Contents

Declaration **1**

List of Abbreviations **8**

1 Introduction **9**
1.1 Background . . . 9
1.2 Problem Statement . . . 9
1.3 Aim . . . 10
1.4 Objectives . . . 10
1.5 Research Questions . . . 11
1.6 Significance . . . 11
1.7 Scope and Limitations . . . 11
1.8 Structure of the Report . . . 12

2 Theoretical Foundation and Knowledge Representation **13**
2.1 Literature Review . . . 13
2.1.1 Data centre cooling and power . . . 13
2.1.2 Failure modes and maintenance . . . 14
2.1.3 Fault detection and diagnosis . . . 14
2.1.4 Knowledge representation . . . 15
2.1.5 Expert systems . . . 15
2.1.6 Prolog and logic programming . . . 15
2.1.7 Deductive reasoning . . . 15
2.1.8 Abductive reasoning . . . 16
2.1.9 Consistency-based diagnosis . . . 16
2.1.10 Explainable artificial intelligence . . . 16
2.1.11 Engineering mathematical models . . . 17
2.1.12 Research gaps . . . 17
2.2 Engineering Theory of the Data Centre Row . . . 18
2.2.1 Purpose and system boundary . . . 18
2.2.2 Energy and material flows . . . 19
2.2.3 Operating conditions and control . . . 19
2.2.4 Governing principles . . . 20
2.2.5 Normal and abnormal operation . . . 20
2.3 Fault Mechanisms . . . 21
2.3.1 Fault taxonomy . . . 21
2.3.2 Multiple and interacting faults . . . 24
2.4 Foundations of Knowledge Representation and Reasoning . . . 24

2.4.1 Logic programming and Horn clauses . . . 24
2.4.2 Negation as failure and the closed-world assumption . . . 24
2.4.3 Expert systems and explanation . . . 25
2.5 Knowledge Representation Scheme . . . 25
2.5.1 Observation vector . . . 25
2.5.2 Layers of the knowledge base . . . 26
2.5.3 Graded evidence and the open-world treatment of missing data . . . 26
2.6 Diagnosis Theory: Deductive, Abductive and Consistency-Based . . . 27
2.7 Mathematical Formulation . . . 28
2.7.1 Sets and indices . . . 28
2.7.2 IT power . . . 28
2.7.3 Server and rack thermal model . . . 29
2.7.4 Rack inlet indicators . . . 29
2.7.5 CRAH air side and fan . . . 30
2.7.6 Chilled-water side and coil . . . 30
2.7.7 Energy balances and analytical redundancy . . . 31
2.7.8 Electrical distribution and battery . . . 32
2.7.9 Evidence, noise estimation and ranking . . . 32
2.7.10 Where each equation is used . . . 33
2.7.11 Closed-world normality and evaluation metrics . . . 33
2.8 Knowledge Acquisition . . . 35

3 Design and Development of the Prolog Expert System 38
3.1 System Architecture . . . 38
3.2 Knowledge-Base Design . . . 40
3.2.1 Facts . . . 40
3.2.2 Thresholds and provenance . . . 40
3.2.3 Engineering calculations . . . 40
3.3 Predicates and Rules . . . 41
3.4 Inference Mechanisms . . . 44
3.4.1 Graded, noise-adaptive evidence . . . 45
3.4.2 The four configurations . . . 45
3.4.3 Design parameters set by judgement . . . 46
3.5 Deductive Reasoning . . . 46
3.6 Abductive Reasoning . . . 47
3.7 Consistency-Based Reasoning . . . 48
3.8 Explanation Facility . . . 48
3.9 Corrective-Maintenance Planner . . . 51
Generated tables . . . 54

4 Dataset, Experimental Methodology and Evaluation 57
4.1 Dataset and Data Dictionary . . . 57
4.1.1 Data origin . . . 57
4.1.2 The simulator . . . 57
4.1.3 Data dictionary . . . 58
4.1.4 Datasets . . . 60
4.2 Test Cases . . . 60
4.3 Experimental Design . . . 60
4.4 Validation . . . 61

4.5 Metrics . . . 62
4.6 Results . . . 63
4.6.1 Overall performance . . . 63
4.6.2 Variability across seeds . . . 65
4.6.3 Results by fault class . . . 66
4.6.4 Where the system fails . . . 67
4.6.5 Explanation fidelity . . . 68
4.7 Sensitivity and Ablation Analysis . . . 69
4.7.1 Sensor noise . . . 69
4.7.2 Missing observations . . . 70
4.7.3 Multiple simultaneous faults . . . 71
4.7.4 Contradictory evidence: sensor bias . . . 71
4.7.5 Threshold changes . . . 73
4.7.6 Reasoning parameters . . . 74
4.7.7 Summary . . . 75
4.8 Robustness to Model Mismatch . . . 75
4.9 Comparison with a Data-Driven Baseline . . . 78

5 Expert-System Software Development, Documentation and Reproducibility 80
5.1 Development Process . . . 80
5.2 Implementation Environment . . . 80
5.3 Software Structure . . . 82
5.4 Debugging . . . 83
5.4.1 Knowledge errors found by the simulator . . . 83
5.4.2 Robustness problems found by the sensitivity analysis . . . 83
5.4.3 Implementation problems . . . 84
5.5 Verification and Validation . . . 84
5.6 Execution Instructions . . . 84
5.7 Reproducibility . . . 85

6 Assessment Framework and Research Deliverables 87
6.1 Assessment Criteria . . . 87
6.2 Minimum Technical Requirements . . . 88
6.3 Research Contributions . . . 88
6.4 Required Tables and Figures . . . 90
6.5 Software Deliverables . . . 91
6.6 Limitations . . . 92
6.7 Future Improvements . . . 93

7 Conclusion and Final Project Specification 95
7.1 Summary of the Developed System . . . 95
7.2 Diagnostic Performance . . . 95
7.3 Answers to the Research Questions . . . 96
7.4 Contribution to Knowledge Representation and Reasoning . . . 97
7.5 Engineering Contribution . . . 97
7.6 Limitations and Future Work . . . 97
7.7 Final Project Specification . . . 97

List of Figures

2.1 Plan view of the data centre row: cooling unit, contained hot aisle, power path and sensors. . . . 19
2.2 Fault propagation: how each fault in the taxonomy leads to observable effects and consequences. . . . 23
2.3 Knowledge-acquisition workflow, from sources to Prolog representation, with feedback from calibration tests. . . . 36

3.1 Architecture of the Prolog expert system. . . . 39
3.2 Diagnostic reasoning workflow of the complete system: abduction proposes, consistency disposes. . . . 44
3.3 Explanation-generation workflow of the explanation facility. . . . 49
3.4 Corrective-maintenance planning workflow. . . . 52

4.1 Experimental and evaluation workflow, with the calibration and held-out splits kept apart. . . . 61
4.2 Exact-match accuracy against sensor noise, by reasoning mode. Each point is 60 synthetic cases. . . . 70
4.3 Exact-match accuracy against the share of readings removed, by reasoning mode. The three model-based configurations give almost identical results, so their lines overlap. . . . 70
4.4 Exact-match accuracy against the number of simultaneous faults, by reasoning mode. . . . 71
4.5 Mean reasoning time per case against the number of simultaneous faults. Times are wall-clock and include measurement noise. . . . 72
4.6 Exact-match accuracy against the size of an injected temperature-sensor bias. 72

List of Tables

2.1 Traceability from literature to knowledge, rule, predicate and test. 17
2.2 Governing principles and their diagnostic use. 20
2.3 Normal operating envelope of the row. 21
2.4 Fault taxonomy. 21
2.5 Interacting faults used in the multiple-fault tests. 24
2.6 Observation variables. 25
2.7 Layers of the knowledge base. 27
2.8 Where each engineering equation is used, generated from the equation references in the code. Reasoning: computed or applied by the knowledge base or an inference engine. Simulator: used to generate the synthetic data. Threshold derivation: used offline to set a threshold that the rules apply. Descriptive: part of the model, but computed by no code. The file-by-file list is in `docs/14_equation_usage.md`. 33
2.9 Nomenclature. 34
2.10 Representative knowledge-acquisition entries. 36

3.1 Source modules. 39
3.2 Contents of the knowledge base. 40
3.4 Diagnostic rules, generated from `kb/rules.pl`. In each rule `A` is the case being diagnosed and `\+` is negation as failure. 41
3.3 Main predicates. 45
3.5 Reasoning parameters set by judgement, with their rationale. Their effect is measured in Table 4.19. 46
3.6 Maintenance-action mapping, generated from `kb/maintenance.pl`: the corrective plan issued for each diagnosis, in order. 52
3.7 Thresholds, generated from `kb/thresholds.pl`. 54
3.8 Provenance of rules and thresholds, generated from `kb/sources.pl`. 54

4.1 Physical parameter changed for each injected fault. 58
4.2 Main simulator assumptions. 59
4.3 Datasets. 60
4.4 Test categories and the framework tests they cover. 60
4.5 Comparison of reasoning modes, held-out split (255 cases). 95% CI: Wilson confidence interval of the exact-match rate. Explain.: ExplainabilityRate. Plan val.: PlanValidity. 63
4.6 Comparison of reasoning modes, calibration split (255 cases). 95% CI: Wilson confidence interval of the exact-match rate. Explain.: ExplainabilityRate. Plan val.: PlanValidity. 64

4.7 Paired comparison of reasoning modes, held-out split: exact two-sided McNemar test on the cases where the two modes disagree. Only A: cases the first mode gets right and the second wrong. Holm: p adjusted for the six comparisons by Holm's method. Bold: below 0.05. 64
4.8 Exact matches by test category and reasoning mode, held-out split. 65
4.9 Exact match on five held-out splits of 255 cases with different seeds. Seed 777 is the held-out split reported in Table 4.5. SD: sample standard deviation over the five seeds. Pooled: all 1275 cases, with the Wilson 95% confidence interval. 66
4.10 Paired comparison of reasoning modes pooled over the five held-out seeds (1275 cases): exact two-sided McNemar test on the cases where the two modes disagree. Holm: p adjusted for the six comparisons by Holm's method. Bold: below 0.05. 66
4.11 Per-class results of the complete system, held-out split. Counts are of fault labels; class accuracy is at case level (Equation (2.7)). Because each class is absent from most cases, class accuracy is dominated by true negatives and is high even for a weak class; precision, recall and F1 are the informative measures. 67
4.12 Confusion matrix of the complete system, held-out split, single-label cases (normal, single-fault and instrumentation categories). Rows are the true class and columns the predicted class. *Mult.* means more than one label was predicted and *Unex.* an unexplained abnormality. 67
4.13 Explanation fidelity on the held-out split. Necessity: share of reported fault labels that disappear when the observations their explanation cites are removed. The next three columns say what replaced them. Control: share that survive when the same number of uncited observations are removed at random. 68
4.14 Sensitivity to sensor noise: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage. . . . 69
4.15 Sensitivity to missing observations: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage. . 71
4.16 Sensitivity to the number of simultaneous faults: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage. 72
4.17 Sensitivity to the size of a sensor bias, a source of contradictory evidence: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage. 73

4.18 Threshold sensitivity on the held-out split: change in exact-match accuracy of the complete system, in percentage points, when one threshold is perturbed. Temperature thresholds move by −2, −1, +1 and +2 K; other thresholds are scaled by 0.8, 0.9, 1.1 and 1.2. The last column is the worst change for deduction alone. A dot means no change. 73
4.19 Sensitivity to the reasoning parameters set by judgement. Each parameter is changed alone. The first two rows give exactly matched cases at the defaults; the other rows give the change in exactly matched cases, for deductive + abductive / deductive + consistency-based / complete system. For the perturbed weights, the range over five random draws is given in brackets. Defaults: bonus 0.1, cut-off $k = 1$, search bound 4, weights from `kb/causal.pl`, noise of unredundant features tied to σ_T. Raw counts are in `docs/15_parameter_sensitivity.md`. 75
4.20 Summary of the sensitivity analysis. 76
4.21 Simulator physics in the two model-mismatch profiles. 77
4.22 Model mismatch: exact match of each reasoning mode when the held-out cases are regenerated with changed simulator physics and diagnosed with the unchanged knowledge base. The last pair of columns re-baselines three envelope thresholds from normal operating data only. p: exact McNemar test against the nominal split, paired case by case. Confidence intervals are in `docs/12_model_mismatch.md`. 77
4.23 Model mismatch by test category: exact matches of the complete system. . 77
4.24 Exact match of the expert system and of data-driven classifiers trained on the raw readings, on the same test cases (255 held-out, 1275 over five seeds, 255 per mismatch profile). The classifiers are a baseline for comparison only. Bold: significantly different from deductive + abductive reasoning on the same cases (exact McNemar, $p < 0.05$). Details in `docs/17_baseline_comparison.md`. 79

5.1 Development lifecycle and its products. 81
5.2 Implementation environment. 81
5.3 Expected exact-match counts, checked by `tests/reproduce.py`. 86

6.1 Evidence against the assessment criteria. 87
6.2 Minimum technical requirements. 89
6.3 Index of the required tables and figures. 90
6.4 Deliverables. 91

7.1 Final project specification. 98

List of Abbreviations

Mathematical symbols are listed in the nomenclature, Table 2.9.

Abbreviation	Meaning
AB	Abnormal, the predicate of consistency-based diagnosis
ASHRAE	American Society of Heating, Refrigerating and Air-Conditioning Engineers
CI	Confidence interval
COMPS	The set of components in consistency-based diagnosis
CPU	Central processing unit
CRAH	Computer-room air handler
CSV	Comma-separated values
EMR	Exact-match rate
IT	Information technology: the servers and their equipment
IU	Phase current imbalance
JSON	JavaScript Object Notation
KB	Knowledge base
KRR	Knowledge representation and reasoning
MAD	Median absolute deviation
NTU	Number of transfer units
OBS	The observations in consistency-based diagnosis
PDU	Power distribution unit
PUE, pPUE	Power usage effectiveness; partial power usage effectiveness
RCI	Rack cooling index
RHI	Return heat index
SD	System description, in consistency-based diagnosis; standard deviation, in the results
SHI	Supply heat index
SLD	Selective linear definite clause resolution
UPS	Uninterruptible power supply
VRLA	Valve-regulated lead-acid (battery)

Chapter 1

Introduction

1.1 Background

Data centres house the computing on which much of modern life depends, and their servers can only run while two supporting systems work. The cooling system must remove almost all of the electrical power the servers draw, which reappears as heat, so that every server inlet stays within the thermal envelope recommended by the American Society of Heating, Refrigerating and Air-Conditioning Engineers (ASHRAE) (ASHRAE Technical Committee 9.9, 2021). The power system must deliver conditioned electricity continuously, with stored energy to ride through interruptions of the utility supply (Barroso et al., 2018). When either fails, servers throttle, shut down or lose power, and the services they host fail with them.

These systems are densely instrumented. A single row of racks reports hundreds of temperatures, currents, powers and status flags. The readings alone, however, do not say what is wrong. A hot rack inlet may mean a failed cooling fan, a loss of chilled water, hot air leaking back through a missing blanking panel or simply a faulty sensor, and each calls for a different repair. Turning readings into a diagnosis, and a diagnosis into the right maintenance action, requires engineering knowledge.

Fault detection and diagnosis for engineering systems has a long history, spanning quantitative model-based methods, qualitative and knowledge-based methods and data-driven methods (Isermann, 2006; Venkatasubramanian, Rengaswamy, Yin, & Kavuri, 2003). Knowledge-based methods have one property that matters particularly for maintenance decisions: they can explain their conclusions in terms an engineer can check (Jackson, 1999). Logic programming offers a natural way to build them, because Prolog's Horn clauses have both a declarative reading as knowledge and a procedural reading as a program (Bratko, 2012; Kowalski, 1979). Artificial intelligence research has also developed formal theories of diagnosis that go beyond simple rules: abduction, which seeks the faults that best explain what is observed (Peng & Reggia, 1990; Poole, 1989), and consistency-based diagnosis, which seeks the components whose failure is consistent with a model of correct behaviour (de Kleer & Williams, 1987; Reiter, 1987).

1.2 Problem Statement

Diagnosing faults in data centre cooling and power infrastructure from monitoring data raises five difficulties:

- different faults produce overlapping symptoms, such as row-wide overheating from either a fan failure or a chilled-water loss;
- instrumentation is itself fallible, so a biased sensor can imitate a fault or hide one;
- readings are noisy and sometimes missing or stale;
- several faults can occur at once and interact;
- a diagnosis is useful only if a maintenance engineer can trust it and act on it, which requires an explanation and a concrete plan.

A purely data-driven classifier can learn symptom patterns, but it cannot easily explain its conclusions or recognise a sensor that is lying, and it needs fault data that are rarely available. A simple rule base is transparent, but brittle under noise and silent about faults it has no rule for. The problem addressed by this project is to build a diagnostic system that keeps the transparency of explicit knowledge while handling these difficulties.

1.3 Aim

The aim of this project is to develop and evaluate an explainable expert system, implemented in Prolog, that diagnoses faults in the cooling and power infrastructure of a data centre row and recommends corrective maintenance, by representing engineering knowledge formally and reasoning over it deductively, abductively and by consistency.

1.4 Objectives

1. To define the system, its normal operating envelope and an engineering-justified taxonomy of physical and instrumentation faults.
2. To formulate the engineering model of the system mathematically, in at least 40 equations.
3. To acquire the diagnostic knowledge from standards, literature and stated engineering assumptions, and to represent it in Prolog with provenance for every rule and threshold.
4. To implement deductive, abductive and consistency-based reasoning, together with an explanation facility and a corrective-maintenance planner.
5. To build a documented, reproducible synthetic dataset that covers normal operation, single and multiple faults, and imperfect evidence.
6. To evaluate the system quantitatively, compare the reasoning modes, and analyse its sensitivity to noise, missing data, multiple faults, sensor bias and threshold choice.
7. To document the development of the software so that its results can be reproduced.

1.5 Research Questions

The central question, set by the project framework, is:

> *Can engineering knowledge be formally represented and computationally reasoned over in Prolog to produce an explainable, defensible diagnosis and an appropriate corrective-maintenance decision?*

It is broken down into four sub-questions:

RQ1 How can the engineering knowledge of data centre cooling and power infrastructure be represented formally, with traceable provenance?

RQ2 How do deductive, abductive and consistency-based reasoning compare in accuracy, robustness to imperfect evidence and computational cost?

RQ3 Can the system distinguish instrumentation faults from equipment faults that produce similar symptoms?

RQ4 Can every diagnosis be explained, and linked to a valid corrective-maintenance plan?

Chapter 7 answers each question.

1.6 Significance

The project is significant in three ways. For practice, it shows how a maintenance engineer could be given not only a diagnosis but the evidence, rules and alternatives behind it, together with an ordered plan of work. For knowledge representation and reasoning, it provides a worked, quantitatively evaluated comparison of three classical theories of diagnosis on one engineering problem, including where each helps and where it costs accuracy. For methodology, it shows how a physics-based simulator with separate calibration and held-out data can expose errors in engineering knowledge, and how documentation can be generated from the code so that the report cannot drift from the system it describes.

1.7 Scope and Limitations

The system covers one row of six racks with one chilled-water computer-room air handler (CRAH), one uninterruptible power supply (UPS) with its valve-regulated lead-acid (VRLA) battery string, and the power distribution units (PDUs) that feed the racks. The chiller plant, the utility supply, the standby generator and humidity control are outside its scope. The engineering model is steady state. The modelled row also has no cooling redundancy. Real rows are normally served by several cooling units in an N+1 or 2N arrangement (Barroso et al., 2018), in which the healthy units take up part of a failed unit's load. A cooling fault would then produce milder and more local symptoms than in this model, so the cooling faults studied here are easier to detect than they would be in most facilities.

No measured fault data were available, so every test case is synthetic, generated by a physics simulator of the row. Because the knowledge base and the simulator rest on the same engineering model, the evaluation demonstrates internal consistency and robustness, not accuracy on a real facility. This and the other limitations are discussed in Section 6.6.

1.8 Structure of the Report

Chapter 2 sets out the engineering theory, the fault mechanisms, the foundations of knowledge representation and reasoning, the mathematical model and the knowledge acquisition. Chapter 3 describes the design and implementation of the Prolog expert system. Chapter 4 presents the dataset, the experimental method and the results. Chapter 5 documents the software development and how to reproduce the results. Chapter 6 assesses the project against the framework, and Chapter 7 concludes.

Chapter 2

Theoretical Foundation and Knowledge Representation

This chapter sets out the engineering knowledge that the expert system reasons over and the formal machinery it reasons with. It begins with a review of the literature that the project draws on (Section 2.1). It then describes the physical system and how it fails (Sections 2.2 and 2.3). It then introduces the foundations of knowledge representation and reasoning (Section 2.4), the representation scheme used in this project (Section 2.5) and the three theories of diagnosis that the system implements (Section 2.6). Section 2.7 gives the engineering and logical model in full, and Section 2.8 explains where the knowledge came from.

2.1 Literature Review

The review is organised around the twelve themes of the project framework, from the engineering system to the gaps that motivate this project. For each theme it asks what the literature establishes and what this project takes from it. Table 2.1 at the end traces the most important sources through to the rules, predicates and tests that use them.

2.1.1 Data centre cooling and power

Data centres are significant consumers of electricity. Estimates for the United States (Shehabi et al., 2016) and the world (Masanet et al., 2020) put their share at around 2% and 1% respectively, and Masanet et al. (2020) show that efficiency gains have largely offset rapid growth in demand. Cooling is a major part of that load, reaching up to 40% of facility energy where cooling is inefficient (Capozzoli & Primiceri, 2015). Reviews of cooling technology describe the air-cooled architectures studied here, with computer-room air handlers supplying a cold aisle and hot-aisle or cold-aisle containment separating supply from exhaust (Capozzoli & Primiceri, 2015; Ebrahimi et al., 2014). Schmidt et al. (2005) trace how rising rack power density strains thermal management and present measurements and models of high-density racks. Patankar (2010) shows how computational models predict the full airflow and temperature distribution of a data centre, validated against measurements in operating facilities. The supply heat index of Sharma et al. (2002) quantifies one consequence of that airflow: the hot exhaust that recirculates into the cold aisle. The thermal envelope that defines normal operation comes from ASHRAE

Technical Committee 9.9 (2021), and the quantitative indices of rack cooling performance from Sharma et al. (2002) and Herrlin (2005). Barroso et al. (2018) describe the power path from utility to server, including UPS and distribution losses, and Fan et al. (2007) provide the linear relationship between server utilisation and power that this project uses to estimate heat loads.

Taken into this project: the system boundary, the operating envelope, the hot-aisle containment layout, the server power model (Equation E1) and the recirculation mechanism behind fault F_4.

2.1.2 Failure modes and maintenance

At data centre scale, hardware failure is routine rather than exceptional. Vishwanath and Nagappan (2010) analysed server failures and repairs across large data centres, and Schroeder and Gibson (2007) showed that field replacement rates of disks greatly exceed what datasheet reliability figures suggest. Failures of the supporting infrastructure are less studied in the open literature, so the fault taxonomy of this project draws on engineering sources for each subsystem: fans and filters (ASHRAE, 2020), branch circuits (National Fire Protection Association, 2023), and the maintenance and testing of VRLA batteries (IEEE, 2005). On the maintenance side, Jardine et al. (2006) describe condition-based maintenance as data acquisition, data processing and maintenance decision, with diagnosis and prognosis as its core. Moubray (1997) sets out reliability-centred maintenance, which ties each maintenance task to a failure mode and its consequences.

Taken into this project: the failure mechanisms of Table 2.4, and the principle that each diagnosis must lead to a specific, verifiable maintenance action.

2.1.3 Fault detection and diagnosis

The classical review by Venkatasubramanian, Rengaswamy, and Kavuri (2003), Venkatasubramanian, Rengaswamy, Kavuri, and Yin (2003), and Venkatasubramanian, Rengaswamy, Yin, and Kavuri (2003) divides diagnostic methods into quantitative model-based, qualitative model-based and process-history-based families, and compares them on criteria that include explanation, isolability and robustness. Isermann (2005, 2006) develop model-based fault detection through residuals: differences between measured and model-predicted behaviour that stay near zero when the system is healthy. Gao et al. (2015) survey model-based and signal-based diagnosis, and Chandola et al. (2009) survey anomaly detection more widely. For buildings, Katipamula and Brambley (2005a, 2005b) review fault detection, diagnosis and prognosis for heating, ventilation and air-conditioning, and Kim and Katipamula (2018) update that review and compare commercial tools. The closest precedent for the deductive layer of this project is APAR, a set of expert rules for air-handling units derived from mass and energy balances (House et al., 2001). It applies only the rules relevant to the current operating mode and is simple enough to embed in building controls (Schein et al., 2006). Its fault detection was found to depend on fault severity and operating conditions (House et al., 2001), which the sensitivity analysis of this project also shows.

Work specific to data centres is recent and mostly data-driven. Asgari et al. (2021) combine a grey-box temperature model with convolutional and recurrent neural networks to diagnose simultaneous cooling faults, with up to 100% accuracy in low-noise conditions. Wang et al. (2024) combine a multi-label random forest with Bayesian inference to

detect, diagnose and correct sensor faults in computer-room air handlers, with about 97% diagnostic accuracy.

Taken into this project: residuals from energy balances as the basis of the redundancy relations (Section 2.7.7), rules derived from mass and energy balances, and the evaluation of single and multiple faults with standard classification metrics.

2.1.4 Knowledge representation

Davis et al. (1993) identify five roles that a knowledge representation plays: a surrogate for the world, a set of ontological commitments, a fragmentary theory of intelligent reasoning, a medium for efficient computation and a medium of human expression. Brachman and Levesque (2004) and the handbook of van Harmelen et al. (2008) develop the formal foundations, which general textbooks place within artificial intelligence as a whole (Russell & Norvig, 2021), including the trade-off between expressiveness and tractable reasoning. For an engineering expert system, the medium-of-expression role is as important as the computational one, because an engineer must be able to read, check and correct the knowledge.

Taken into this project: representing knowledge in a restricted, readable form (Horn clauses over named thresholds with sources) so that each rule can be read as an engineering statement and traced to its origin.

2.1.5 Expert systems

The rule-based expert system was established by MYCIN, whose developers also showed the value of explaining a conclusion by the rules that produced it (Buchanan & Shortliffe, 1984). Textbooks set out the architecture of separate knowledge base and inference engine (Giarratano & Riley, 2005; Jackson, 1999). Liao (2005) review 166 articles on expert-system applications from 1995 to 2004 and classify the methods into eleven families, from rule-based systems to case-based reasoning and ontologies. They observe that the methods were tending towards expertise orientation, with applications developed around specific problems.

Taken into this project: the separation of knowledge from inference, and rule identifiers that support explanation.

2.1.6 Prolog and logic programming

Logic programming treats a set of Horn clauses as both a theory and a program (Kowalski, 1979). Prolog grew out of work on natural-language processing and theorem proving (Colmerauer & Roussel, 1993), and its programming techniques, including meta-interpreters, are set out by Sterling and Shapiro (1994) and Bratko (2012). Lloyd (1987) gives the formal semantics. SWI-Prolog, used here, is described by Wielemaker et al. (2012).

Taken into this project: the proof-recording meta-interpreter of Section 3.5, a standard Prolog technique that makes rule-based reasoning explainable without leaving the logical framework.

2.1.7 Deductive reasoning

Resolution gave automated deduction a single, complete inference rule (Robinson, 1965). For definite clauses, the least-model and fixpoint semantics coincide with what SLD

resolution derives (Apt & van Emden, 1982; Lloyd, 1987). Negation as failure, needed for conclusions such as "no fault is present", is sound with respect to the completion of the program (Clark, 1978).

Taken into this project: deductive diagnosis as $KB \cup E \models F$, and the controlled use of negation as failure for guards and for the closed-world conclusion of normal operation.

2.1.8 Abductive reasoning

Abduction infers the hypotheses that would best explain an observation (Josephson & Josephson, 1994). Poole (1989) formalise explanation as a set of hypotheses that, with a background theory, entails the observations while staying consistent with them. Kakas et al. (1992) survey abductive logic programming and its relation to default reasoning and negation as failure. Peng and Reggia (1990) develop parsimonious covering theory, in which the preferred explanations are the smallest sets of disorders that cover all observed manifestations.

Taken into this project: parsimonious covering over causal signatures, minimality as the preference criterion, and masking, by which one hypothesis can account for the absence of another's expected symptom.

2.1.9 Consistency-based diagnosis

Reiter (1987) defined a diagnosis as a set of abnormal components that makes the system description consistent with the observations, and showed that minimal diagnoses are the minimal hitting sets of the conflict sets. Greiner et al. (1989) corrected an error in Reiter's algorithm for computing them. de Kleer and Williams (1987) developed the general diagnostic engine for multiple faults, and de Kleer et al. (1992) characterised diagnoses more fully through kernel diagnoses. Because pure consistency-based diagnosis cannot exonerate a component, Struss and Dressler (1989) added fault models that predict how a component behaves when it fails. Console and Torasso (1991) place abductive and consistency-based diagnosis on a single spectrum, and Hamscher et al. (1992) collect the key papers of the field. Cordier et al. (2004) show how conflicts in this tradition correspond to the analytical redundancy relations of control engineering.

Taken into this project: redundancy relations with support sets, minimal hitting sets, fault models for exoneration, sensors treated as components, and the combination of abduction and consistency in one system.

2.1.10 Explainable artificial intelligence

Explanation was central to early expert systems. Swartout (1983) argued that a good explanation must justify why a conclusion is reasonable, not merely trace how it was computed. Recent work on explainable artificial intelligence is surveyed by Adadi and Berrada (2018). Miller (2019) draws on the social sciences to show that people expect explanations to be contrastive, saying why this conclusion and not another. Rudin (2019) argues that for high-stakes decisions inherently interpretable models are preferable to explaining black boxes after the fact, and Doshi-Velez and Kim (2017) call for rigorous evaluation of interpretability, including evaluation with users.

Taken into this project: explanations that report competing hypotheses and the reason each was rejected, which makes them contrastive in Miller's sense; a symbolic model

that is interpretable by construction; and the recognition that explanation quality should eventually be evaluated with users (Section 6.6).

2.1.11 Engineering mathematical models

The model of Section 2.7 rests on standard engineering sources: sensible heat transfer and the effectiveness–NTU method for the coil (Bergman et al., 2011), the fan affinity laws (ASHRAE, 2020), and three-phase power and the continuous-load limit (National Fire Protection Association, 2023). The thresholds on energy-balance residuals follow first-order propagation of measurement uncertainty (Joint Committee for Guides in Metrology, 2008). The noise estimate uses the median absolute deviation, a robust scale estimator that one outlying sensor cannot distort (Rousseeuw & Croux, 1993).

2.1.12 Research gaps

The review points to three gaps that this project addresses:

1. **Explainable diagnosis of data centre infrastructure.** Diagnosis work specific to data centres found in this review is data-driven (Asgari et al., 2021; Wang et al., 2024). It reports high accuracy, but it needs labelled fault data for training and does not explain its conclusions in engineering terms. The closest knowledge-based work addresses building air-handling units (House et al., 2001; Schein et al., 2006), not the combined cooling and power path of a data centre.

2. **Combining reasoning modes on one engineering problem.** The theories of abductive and consistency-based diagnosis are well developed (Console & Torasso, 1991; Peng & Reggia, 1990; Reiter, 1987), but this review found few studies that implement all three reasoning modes for one engineering system and compare them quantitatively under noise, missing data and multiple faults.

3. **Separating sensor faults from equipment faults with explanations.** Sensor faults in data centre cooling have been addressed with machine learning (Wang et al., 2024). The use of redundancy relations, fault models and hypothesis tests to separate a biased sensor from a real fault, while explaining why, has received less attention for this domain.

These gaps define the contribution claimed in Section 6.3. The review did not search exhaustively, so the gaps are stated relative to the literature examined here.

Table 2.1: Traceability from literature to knowledge, rule, predicate and test.

Source	Knowledge	Rule or predicate	Test
ASHRAE Technical Committee 9.9 (2021)	Recommended inlet envelope, 18 to 27 °C	`threshold/4`; rule r01	Normal and all thermal fault cases
Fan et al. (2007)	Server power linear in utilisation	Equation E1; simulator	All cases
Sharma et al. (2002)	Hot exhaust recirculates into the cold aisle	Equations E14, E18, E19; rules r22 to r24	F_4 cases

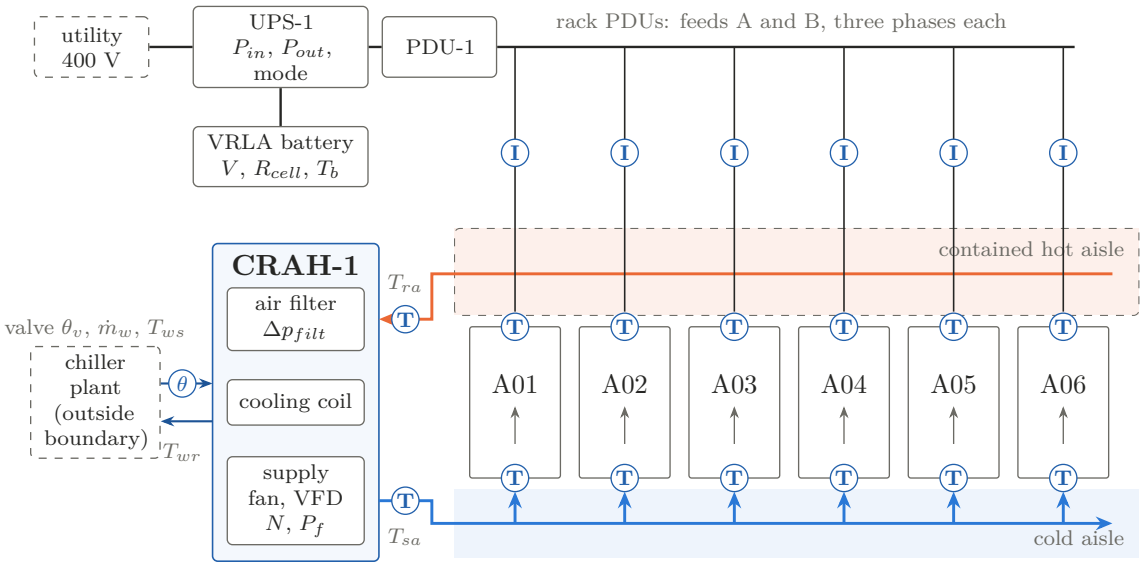
Source	Knowledge	Rule or predicate	Test
ASHRAE (2020)	Fan affinity laws	`expected_fan_power/3`; rules r07, r08, r48, r49	F_1, F_3 cases
National Fire Protection Association (2023)	80% continuous-load limit	`branch_loading/3`; rule r17	F_6 cases
IEEE (2005)	Resistance rise shows capacity fade	`cell_resistance_ratio/3`; rule r18	F_7 cases
House et al. (2001)	Rules from mass and energy balances	Coil balance relations; rule r43	Sensor-bias cases
Isermann (2006)	Residual-based detection	`coil_balance_residual/3`	Sensor-bias and fan cases
Joint Committee for Guides in Metrology (2008)	Uncertainty propagation	Equations E39, E40; residual thresholds	Threshold sensitivity
Rousseeuw and Croux (1993)	Robust scale estimation	`sigma_temp/2`	High-noise sweep
Peng and Reggia (1990)	Parsimonious covering	`abduce/2`	All categories
Reiter (1987)	Minimal hitting sets	`cbd_diagnoses/2`	Multiple-fault cases
Struss and Dressler (1989)	Fault models exonerate components	`fm/2`	Sensor-bias cases
Miller (2019)	Contrastive explanation	`why/1`, competing hypotheses	Explanation traces

2.2 Engineering Theory of the Data Centre Row

2.2.1 Purpose and system boundary

The system studied is the cooling and power infrastructure that serves one row of a data centre. It has two functions. It must remove the heat that the servers dissipate, so that every server inlet stays within the thermal envelope recommended by ASHRAE (ASHRAE Technical Committee 9.9, 2021). It must also deliver conditioned electrical power continuously, with enough stored energy to ride through a utility interruption until standby generation takes over.

To keep the project tractable, the boundary encloses one row of six racks, A01 to A06, in a hot-aisle containment layout (Figure 2.1). Each rack holds twenty servers. The row is served by one chilled-water computer-room air handler (CRAH-1), one double-conversion uninterruptible power supply (UPS-1) with a valve-regulated lead-acid (VRLA) battery string, one row power distribution unit (PDU-1) and a rack power distribution unit with two independent feeds, A and B, in every rack. The chiller plant, the utility supply and the standby generator lie outside the boundary and are treated as inputs. Humidity control is excluded, so the thermal model is sensible-heat only.



T temperature (inlet middle and top at the front of each rack, outlet mean and top at the rear); I phase currents; blue arrows supply air, orange arrows return air. Each server also reports CPU temperature, clock and fan speed.

Figure 2.1: Plan view of the data centre row: cooling unit, contained hot aisle, power path and sensors.

2.2.2 Energy and material flows

Four flows connect the components:

- **Electrical.** Utility power passes through UPS-1, PDU-1 and the rack PDUs to the servers. Almost all of the electrical energy that the servers draw ends up as heat.
- **Air.** CRAH-1 discharges supply air into the cold aisle. Server fans draw it through the servers into the contained hot aisle, from which it returns to CRAH-1 through the filter and cooling coil.
- **Water.** Chilled water from the plant passes through a modulating control valve and the coil, absorbs the heat and returns to the plant.
- **Heat.** Server heat is carried by the air, transferred to the chilled water at the coil and leaves the boundary. UPS and PDU losses add to the room load.

In steady state, the heat removed from the air must equal the IT power plus the fan heat. The same heat must also equal what the water absorbs. These two balances can be computed from independent sensors, which gives the analytical redundancy that later separates sensor faults from equipment faults (Isermann, 2006).

2.2.3 Operating conditions and control

The row operates between low, nominal and high load, corresponding to mean processor utilisations of about 10%, 50% and 90% and row IT loads of 54, 78 and 102 kW (Section 2.7). Four control loops shape its behaviour:

- the CRAH valve modulates water flow to hold a supply-air setpoint of 20 °C;

- the CRAH fan speed follows the airflow demand of the servers;
- each server's management controller sets its own fan speed from its internal temperatures;
- the UPS runs in double conversion, on battery or on static bypass.

2.2.4 Governing principles

Table 2.2 summarises the physical principles that the diagnostic knowledge relies on. Sensible heat transfer and the first law of thermodynamics underpin the energy balances. The fan affinity laws predict fan power from fan speed (ASHRAE, 2020). The effectiveness–NTU method describes the cooling coil (Bergman et al., 2011). On the electrical side, three-phase power and the continuous-load limit of branch circuits (National Fire Protection Association, 2023) govern overloads, while rising internal resistance indicates fading battery capacity (IEEE, 2005). On the computing side, server power grows roughly linearly with utilisation (Fan et al., 2007), and processors throttle their clock when they approach their junction temperature limit.

Table 2.2: Governing principles and their diagnostic use.

Domain	Principle	Diagnostic use
Heat transfer	Sensible heat: $Q = \dot{m} c_p \Delta T$	Air-side and water-side heat flows
Thermodynamics	First law over the row	Cross-checks between independent sensors
Fluid mechanics	Fan affinity laws	Expected fan power at a given speed
Fluid mechanics	Filter pressure drop grows with loading	Filter clogging
Heat exchange	Effectiveness–NTU coil model	Coil capacity and valve behaviour
Electrical	Three-phase power; 80% continuous-load limit	Branch overload, rack power balance
Electrochemistry	Internal resistance rises as capacity fades	Battery degradation
Computing	Server power linear in utilisation; thermal throttling	Expected heat; consequences of faults

2.2.5 Normal and abnormal operation

Table 2.3 defines the normal operating envelope. Abnormal operation is any sustained departure from this envelope, or any break in the energy balances that exceeds the combined sensor uncertainty (Section 2.7.7). Values marked as assumptions are design choices for the modelled row. They are justified where they are used, and their sensitivity is quantified in Chapter 4.

Table 2.3: Normal operating envelope of the row.

Variable	Normal range	Source
Rack inlet temperature	18 to 27 °C (allowable 15 to 32 °C, class A1)	ASHRAE Technical Committee 9.9 (2021)
Inlet stratification, top minus middle	below 3 K	Assumption
Supply-air temperature	20 °C ± 2 K	Design setpoint
Chilled-water temperature rise	about 5 to 7 K	Design value
Fan power over affinity-law prediction	0.95 to 1.05	Assumption
Branch current	at most 80% of breaker rating	National Fire Protection Association (2023)
UPS efficiency, double conversion	about 0.95	ENERGY STAR minimum for a 50 to 100 kW unit (U.S. Environmental Protection Agency, n.d.)
VRLA float voltage	2.25 to 2.30 V per cell at 25 °C	13.5 to 13.8 V per 12 V battery (CSB Energy Technology, 2026)
Cell internal resistance	within 15% of commissioning baseline	Assumption, after IEEE (2005)
Telemetry sample age	below 60 s	Assumption

2.3 Fault Mechanisms

2.3.1 Fault taxonomy

The fault universe contains eight physical faults, two instrumentation faults and normal operation:

$$\mathcal{F} = \{F_1, \ldots, F_8, S_1, S_2, F_N\}. \tag{2.1}$$

Each fault was admitted only if it is a recognised failure mode with a distinct observable signature inside the system boundary. No category was created merely to increase the number of classes. Physical faults (F) are kept separate from instrumentation faults (S), because a sensor fault calls for a different corrective action than an equipment fault. Table 2.4 gives the physical meaning, causes, symptoms and consequences of each.

Table 2.4: Fault taxonomy.

Fault	Physical meaning	Typical causes	Observable symptoms	Consequences
F_N normal	All variables within the envelope; balances close	–	Inlets 18 to 27 °C; supply at setpoint; fan power matches speed	–

Fault	Physical meaning	Typical causes	Observable symptoms	Consequences
F_1 CRAH fan failure	Fan delivers little or no air at its commanded speed	Motor burnout, broken belt or coupling, drive trip	Fan power far below prediction; row-wide hot inlets; supply air colder than setpoint	Row overheating, throttling, shutdowns
F_2 chilled-water loss	Coil receives too little water flow	Seized valve, plant pump trip, closed isolation valve, air lock	Water flow far below the valve prediction; supply air hot; fan healthy	Row overheating
F_3 filter clogging	Filter resistance rises, cutting airflow at a given speed	Missed replacement, construction dust	High filter loading ratio; fan speed rising	Higher fan energy; leads towards F_1
F_4 hot-air recirculation	Hot exhaust re-enters the server inlets	Missing blanking panels, open containment doors	Top inlet and exhaust stratified on one rack; CRAH healthy	Local overheating of upper servers
F_5 server fan failure	One server loses internal airflow	Fan wear, failed fan controller	Server fan speed near zero; CPU near 95 °C; clock reduced	Throttling or shutdown of one server
F_6 branch overload	A phase circuit exceeds its continuous rating	Added servers, peaks, phase imbalance	Phase current above 80% of the breaker rating	Breaker trip dropping all servers on the circuit
F_7 battery degradation	Capacity has faded, shortening ride-through	Age, heat, sulphation, dry-out	Cell resistance well above baseline; low float voltage	Load drop during a utility event
F_8 UPS on bypass	UPS has transferred the load to static bypass	Inverter or rectifier fault, overload	Mode flag shows bypass; efficiency about 1.0	No battery protection
S_1 sensor bias	A temperature sensor reads offset from the truth	Drift, poor placement, loose mounting	One sensor disagrees with neighbours and with the energy balance	False alarms or a masked fault
S_2 telemetry dropout	Readings are stale or missing	Agent crash, network loss	Sample age above the limit	Incomplete evidence

Several pairs of faults share symptoms, and the observation vector was chosen to separate them. F_1 and F_2 both overheat the whole row. They differ in fan power and in chilled-water flow measured against valve position. F_3 and F_1 are points on the same airflow-loss path, gradual and abrupt, but call for different actions. F_4 is local to a rack while the cooling unit is healthy, and F_5 is local to one server. S_1 can either imitate F_4, when a top inlet sensor reads high, or hide it, when the sensor reads low. Figure 2.2 traces each fault from its mechanism to its observable effect and its consequence.

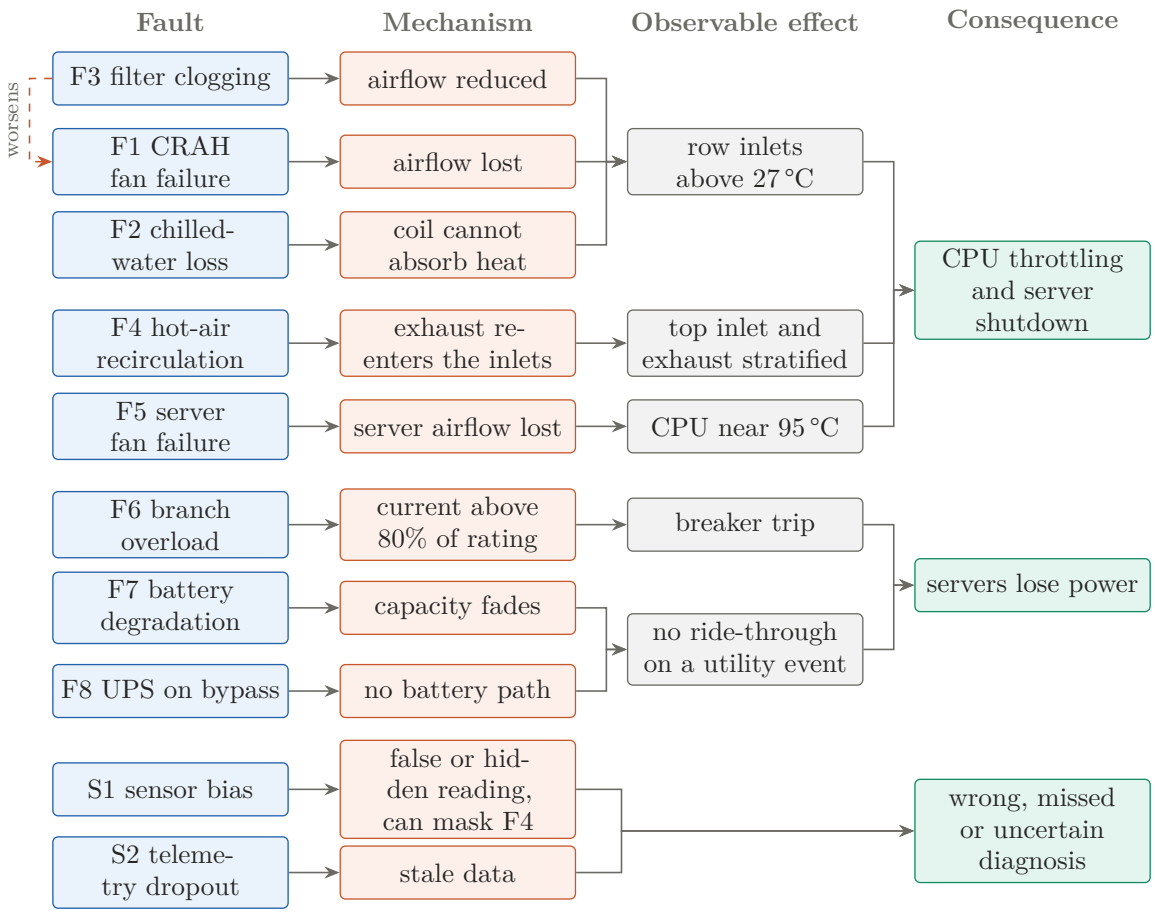


Figure 2.2: Fault propagation: how each fault in the taxonomy leads to observable effects and consequences.

2.3.2 Multiple and interacting faults

Faults can occur together and interact. Five combinations are used in the multiple-fault tests (Table 2.5). They were chosen because each tests a distinct interaction: aggravation, masking, compound risk and co-location.

Table 2.5: Interacting faults used in the multiple-fault tests.

Combination	Interaction
F_3 with F_4	Reduced airflow aggravates recirculation at a leaky rack.
F_4 with S_1	A low-reading top inlet sensor hides a real recirculation hot spot.
F_7 with F_8	The load sits on bypass while the battery is weak, a compound risk to continuity.
F_1 with F_6	A cooling and an electrical fault occur together and must both be reported.
F_4 with F_5	A failed server fan inside a recirculating rack.

2.4 Foundations of Knowledge Representation and Reasoning

Knowledge representation and reasoning studies how knowledge can be written in a formal language so that a program can draw conclusions from it that are justified by the knowledge alone (Brachman & Levesque, 2004). Two properties matter most for engineering diagnosis. The representation must be *declarative*, so that each piece of knowledge has a meaning independent of how it is used. The reasoning must be *sound* with respect to that meaning, so that every conclusion can be justified by tracing it back to the knowledge and the observations that support it.

2.4.1 Logic programming and Horn clauses

Logic programming represents knowledge as definite clauses, also called Horn clauses, of the form $H \leftarrow B_1 \wedge \cdots \wedge B_n$: the head H holds whenever every body literal B_i holds (Kowalski, 1979; Lloyd, 1987). A set of such clauses has a least Herbrand model, and SLD resolution derives exactly the ground atoms that are true in it, so the procedural reading of a Prolog program coincides with its logical reading for this fragment (Lloyd, 1987). Prolog adds arithmetic, built-in predicates and a fixed left-to-right, depth-first search strategy (Bratko, 2012).

2.4.2 Negation as failure and the closed-world assumption

Diagnosis needs negative conclusions, such as "the supply air is *not* hot" or "no fault rule fires". Prolog provides negation as failure: $\neg G$ succeeds when every attempt to prove G fails. Under the closed-world assumption, whatever cannot be derived is taken to be false, and Clark (1978) showed that negation as failure is sound with respect to the completion of the program. This project uses negation as failure in two controlled places: to conclude normal operation when no fault rule fires and every inlet is within the envelope, and to block a rule when contrary evidence can be proved. Section 2.5 explains why missing

evidence is never treated as negative evidence in the model-based modules, which would otherwise make the closed-world assumption dangerous.

2.4.3 Expert systems and explanation

An expert system separates a knowledge base, written by or with domain experts, from an inference engine that applies it (Jackson, 1999). Its defining requirement is transparency: it must be able to say why it reached a conclusion and which knowledge it used. Rule-based systems meet this naturally, because every conclusion is the head of a rule whose body was proved. This project keeps that property for all three reasoning modes by recording the proof of every deductive conclusion, the evidence behind every abductive hypothesis and the conflict sets behind every consistency-based diagnosis (Chapter 3).

2.5 Knowledge Representation Scheme

2.5.1 Observation vector

The observation vector is $\mathbf{x} = [x_1, x_2, \ldots, x_{26}]^\mathsf{T}$. Every variable that appears in an equation, a table, a figure or a Prolog predicate is defined in Table 2.6. Each observation is stored as a fact `obs(Case, Variable, Entity, Value)`. Entities are named systematically: racks `rack_a01` to `rack_a06`, servers `rack_a01_s01` onwards, rack PDU phases `rack_a01_a_l1` onwards, and `crah_1`, `ups_1` and `battery_1`.

Table 2.6: Observation variables.

Symbol	Variable	Unit	Prolog name	Diagnostic relevance
x_1	Rack inlet temperature, middle	°C	`t_inlet_mid`	Envelope; neighbour consistency
x_2	Rack inlet temperature, top	°C	`t_inlet_top`	Stratification (F_4, S_1)
x_3	Rack outlet temperature, mean	°C	`t_outlet`	Outlet stratification
x_4	CRAH supply-air temperature	°C	`t_supply_air`	F_1, F_2, S_1
x_5	CRAH return-air temperature	°C	`t_return_air`	Air-side balance
x_6	CRAH fan speed	%	`fan_speed`	Affinity-law prediction
x_7	CRAH fan electrical power	kW	`fan_power`	F_1
x_8	Filter differential pressure	Pa	`dp_filter`	F_3
x_9	Chilled-water supply temperature	°C	`t_chw_supply`	Water-side balance
x_{10}	Chilled-water return temperature	°C	`t_chw_return`	Water-side balance
x_{11}	Chilled-water valve position	%	`valve_position`	Valve prediction (F_2)
x_{12}	Rack PDU phase current	A	`i_branch`	F_6; power balance
x_{13}	Rack IT power	kW	`p_rack`	Power balance

Symbol	Variable	Unit	Prolog name	Diagnostic relevance
x_{14}	Mean CPU utilisation of a rack	–	`cpu_util`	Expected heat
x_{15}	Server CPU temperature	°C	`t_cpu`	F_5
x_{16}	Server CPU clock frequency	GHz	`f_cpu`	Throttling
x_{17}	Server fan speed	rpm	`server_fan_rpm`	F_5
x_{18}	UPS input power	kW	`p_ups_in`	UPS efficiency (F_8)
x_{19}	UPS output power	kW	`p_ups_out`	UPS efficiency (F_8)
x_{20}	UPS operating mode	–	`ups_mode`	F_8
x_{21}	Battery string float voltage	V	`v_battery`	F_7
x_{22}	Highest cell internal resistance	mΩ	`r_cell`	F_7
x_{23}	Battery temperature	°C	`t_battery`	Ageing
x_{24}	Telemetry sample age	s	`sample_age`	S_2
x_{25}	Chilled-water mass flow	kg/s	`m_chw`	F_2; water-side balance
x_{26}	Rack outlet temperature, top	°C	`t_outlet_top`	Outlet stratification

2.5.2 Layers of the knowledge base

The knowledge base is organised in seven layers, each a set of Prolog facts or Horn clauses. Table 2.7 lists them. Two design choices deserve explanation.

Rules as data. Diagnostic rules are stored as facts `drule(Id, Head, Body, Source)` rather than as ordinary clauses. A small meta-interpreter proves them and returns a proof tree for every conclusion. Each rule remains a Horn clause, $\mathit{Head} \leftarrow \mathit{Body}$, so the logical reading is unchanged. Storing rules as data gives every rule an identifier and a source, which the explanation facility reports, and lets the rule table in the report be generated directly from the knowledge base.

Provenance on every threshold and rule. Every threshold fact (`threshold/4`) and every rule names a source fact (`source/3`), whose type is `standard`, `literature`, `assumption` or `method`. This keeps four kinds of information apart, as the project framework requires: measured data (none in this project), synthetic test cases, engineering assumptions and literature-supported rules.

2.5.3 Graded evidence and the open-world treatment of missing data

The deductive rules compare features with crisp thresholds. The abductive and consistency-based modules instead use graded symptoms. Each symptom receives an evidence strength between 0 and 1, scaled to the measurement noise estimated from the case itself (Equations (E55) and (E53)), and a status of present, absent, uncertain or unknown. A symptom whose inputs are missing is *unknown*, never absent. The closed-world assumption is therefore applied only where it is safe: in the deductive rules, and in the final "normal"

Table 2.7: Layers of the knowledge base.

Layer	Representation	Content
Structure	`component/2`, `serves/2`, `rated/4`	Components, topology and ratings
Reference values	`threshold/4`, `source/3`	Envelopes and limits with provenance
Engineering calculations	Horn clauses with arithmetic	The equations of Section 2.7
Diagnostic rules	`drule/4`	47 rules deriving conditions and faults
Causal knowledge	`evidence/3`, `requires/2`, `masks/2`	Fault signatures for abduction
Redundancy relations	`arr/3`, `fm/2`	Relations, support sets and fault models
Maintenance knowledge	`maintenance/5`, `plan_interaction/3`	Corrective plans and their ordering

conclusion. It is never used to infer that a symptom is absent merely because it could not be computed.

2.6 Diagnosis Theory: Deductive, Abductive and Consistency-Based

Let KB be the knowledge base and E the evidence: the observations of a case together with the conditions derived from them. Three theories of diagnosis are implemented. Console and Torasso (1991) show that they are points on one spectrum of logical definitions of model-based diagnosis.

Deductive diagnosis. A fault F_k is diagnosed when it follows from the knowledge base and the evidence:

$$KB \cup E \models F_k. \tag{2.2}$$

This is the classical reading of a rule-based expert system. It is efficient and transparent, but it needs a rule for every symptom pattern, and a crisp rule turns measurement noise directly into false findings.

Abductive diagnosis. Abduction reasons from effects to causes. A set of hypotheses $D \subseteq \mathcal{H}$, where $\mathcal{H} = \mathcal{F} \setminus \{F_N\}$, is an explanation when it entails the evidence and is consistent with it (Poole, 1989):

$$D \cup KB \models E, \qquad D \cup KB \cup E \not\models \bot. \tag{2.3}$$

Among the explanations, the parsimony principle prefers the smallest:

$$D^* = \arg\min_{D} |D|. \tag{2.4}$$

The implementation follows parsimonious covering theory (Peng & Reggia, 1990): causal knowledge states which symptoms each fault can explain, and the search finds minimal sets of faults that cover every clearly present symptom.

Consistency-based diagnosis. Consistency-based diagnosis reasons from a model of correct behaviour (de Kleer & Williams, 1987; Reiter, 1987). Let SD be the system description, $COMPS$ the set of components, OBS the observations and $AB(c)$ the statement that component c is abnormal. A set $\Delta \subseteq COMPS$ is a diagnosis when

$$SD \cup OBS \cup \{AB(c) \mid c \in \Delta\} \cup \{\neg AB(c) \mid c \in COMPS \setminus \Delta\} \not\models \bot. \tag{2.5}$$

A conflict set is a set of components that cannot all be healthy given the observations. Reiter (1987) proved that the minimal diagnoses are exactly the minimal hitting sets of the conflict sets. In this project the behaviour model is written as analytical redundancy relations, each of which holds when a named set of components is healthy, which links consistency-based diagnosis to the fault detection practice of control engineering (Cordier et al., 2004). Sensors are treated as components, so a sensor fault is a diagnosis like any other. Pure consistency-based diagnosis cannot exonerate a component, because it never predicts what a fault would look like. Fault models remove this limitation (Struss & Dressler, 1989): if a component is abnormal, a named relation must also be violated, so a component whose fault model predicts a violation that clearly did not happen is exonerated.

Combining the three. The complete system combines the modes as "abduction proposes, consistency disposes". Abduction produces ranked candidate explanations, and the best-ranked one that is also a consistent diagnosis in the sense of Equation (2.5) is accepted. Chapter 3 describes the implementation, and Chapter 4 compares the four configurations that the framework asks for.

2.7 Mathematical Formulation

This section gives the engineering and logical model in full. Equations carry the tags E1 to E56 used in the design documents and in the comments of `kb/calculations.pl`, so that each equation can be traced to the predicate that implements it. Equations with a letter suffix, such as E11a, were added during development. Two numbering schemes are used: engineering equations carry these E-tags, while logical formulas and evaluation metrics are numbered by chapter, such as (2.5). Every symbol is defined where it first appears and again in Table 2.9.

2.7.1 Sets and indices

$\mathcal{R} = \{r_1, \ldots, r_6\}$ is the set of racks, with $n_R = |\mathcal{R}| = 6$. $\mathcal{S}_r$ is the set of servers in rack r, with $|\mathcal{S}_r| = 20$.

2.7.2 IT power

Server power is close to linear in CPU utilisation $u_s \in [0, 1]$ (Fan et al., 2007):

$$P_s = P_{idle} + (P_{max} - P_{idle})\, u_s, \tag{E1}$$

$$P_r = \textstyle\sum_{s \in \mathcal{S}_r} P_s, \tag{E2}$$

$$P_{IT} = \textstyle\sum_{r \in \mathcal{R}} P_r, \tag{E3}$$

$$\varepsilon_{P,r} = \frac{P_r^{meas} - P_r}{P_r}. \tag{E4}$$

With $P_{idle} = 0.40\,\mathrm{kW}$ and $P_{max} = 0.90\,\mathrm{kW}$ per server, utilisations of 10%, 50% and 90% give row loads of 54, 78 and 102 kW. P_r^{meas} is the metered rack power and $\varepsilon_{P,r}$ the model residual.

2.7.3 Server and rack thermal model

$$\dot{m} = \rho_a \dot{V}, \tag{E5}$$

$$\Delta T_s = \frac{P_s}{\dot{m}_s\, c_{p,a}}, \tag{E6}$$

$$T_{out,r} = T_{in,r} + \frac{P_r}{\dot{m}_r\, c_{p,a}}, \tag{E7}$$

$$T_{cpu,s} = T_{in,r} + R_{th}(\omega_s)\, P_{cpu,s}, \tag{E8}$$

$$\phi_{f,s} = f_{cpu,s} / f_{nom,s}. \tag{E9}$$

$\dot{m}$ is mass flow, ρ_a air density, $\dot{V}$ volume flow and $c_{p,a}$ the specific heat of air. Equation E6 gives a rise of about 13 K across a server at nominal load. In E8, the junction-to-inlet thermal resistance R_{th} rises as the server fan speed ω_s falls, so a failed server fan (F_5) raises the CPU temperature even when the inlet is normal. A frequency ratio $\phi_{f,s} < 1$ shows throttling, which is a consequence of F_1, F_2, F_4 or F_5 and not a fault class of its own.

2.7.4 Rack inlet indicators

$$T_{in,max,r} = \max(T_{in,top,r},\, T_{in,mid,r}), \tag{E10}$$

$$\Delta T_{strat,r} = T_{in,top,r} - T_{in,mid,r}, \tag{E11}$$

$$\Delta T_{strat,out,r} = T_{out,top,r} - T_{out,bot,r} = 2\,(T_{out,top,r} - T_{out,r}), \tag{E11a}$$

$$h_r = \mathbf{1}[T_{in,max,r} > T_{rec,max}]\,, \tag{E12}$$

$$\phi_{hot} = \frac{1}{n_R} \textstyle\sum_r h_r. \tag{E13}$$

$\mathbf{1}[\cdot]$ is 1 when its condition holds and 0 otherwise, and $T_{rec,max} = 27\,^{\circ}\mathrm{C}$ (ASHRAE Technical Committee 9.9, 2021). A hot-rack fraction $\phi_{hot} \geq 0.5$ points to a row-level cause such as F_1 or F_2; a smaller fraction points to a rack-level cause such as F_4. Equation E11a assumes that the two halves of a rack carry similar airflow, so the lower-half outlet temperature is $2T_{out,r} - T_{out,top,r}$. Real recirculation heats the upper servers' inlet air, so their exhaust becomes hotter too and E11a rises with E11. A biased top inlet sensor changes E11 but not E11a, which is how F_4 is separated from S_1.

The supply and return heat indices (Sharma et al., 2002) and the rack cooling indices (Herrlin, 2005) summarise the thermal health of the row:

$$SHI = \frac{\sum_r (T_{in,r} - T_{sa})}{\sum_r (T_{out,r} - T_{sa})}, \qquad RHI = 1 - SHI, \tag{E14–E15}$$

$$RCI_{HI} = \left[1 - \frac{\sum_r (T_{in,r} - T_{rec,max})\,\mathbf{1}[T_{in,r} > T_{rec,max}]}{n_R\,(T_{all,max} - T_{rec,max})}\right] \times 100\%, \tag{E16}$$

$$RCI_{LO} = \left[1 - \frac{\sum_r (T_{rec,min} - T_{in,r})\,\mathbf{1}[T_{in,r} < T_{rec,min}]}{n_R\,(T_{rec,min} - T_{all,min})}\right] \times 100\%. \tag{E17}$$

The mixing model behind F_4 treats the air at an inlet as a blend of supply air and exhaust, with recirculation fraction $\beta_r \in [0, 1]$:

$$T_{in,r} = (1 - \beta_r)\, T_{sa} + \beta_r\, T_{out,r}, \qquad \beta_r = \frac{T_{in,r} - T_{sa}}{T_{out,r} - T_{sa}}. \tag{E18–E19}$$

Because $\beta_r \geq 0$ and $T_{out,r} > T_{sa}$, the mixing model implies that a top inlet can never be colder than the middle inlet. A top sensor that reads colder must therefore be faulty.

2.7.5 CRAH air side and fan

$$Q_a = \dot{m}_a\, c_{p,a}\, (T_{ra} - T_{sa}), \tag{E20}$$
$$\dot{V}_a = \dot{V}_{rated}\, (N/N_{rated}), \tag{E21}$$
$$\Delta p_{fan} = \Delta p_{rated}\, (N/N_{rated})^2, \tag{E22}$$
$$P_{f,exp} = P_{f,rated}\, (N/N_{rated})^3, \tag{E23}$$
$$\rho_f = P_f / P_{f,exp}, \tag{E24}$$
$$P_f = \dot{V}_a\, \Delta p_{tot} / \eta_f, \tag{E25}$$
$$\Delta p_{filt} = K_{filt}\, \dot{V}_a^2, \tag{E26}$$
$$\lambda_{filt} = \frac{\Delta p_{filt} / \dot{V}_a^2}{K_{filt,clean}}, \tag{E27}$$
$$\delta_{sa} = T_{sa} - T_{sp}. \tag{E28}$$

Equations E21 to E23 are the fan affinity laws (ASHRAE, 2020). A healthy fan has a power ratio ρ_f within about 5% of 1. A ratio below 0.85 shows a partial failure such as belt slip, and below 0.5 a fan that is not moving air at all (F_1). The filter loading ratio λ_{filt} normalises the filter resistance by its clean value, so it does not depend on fan speed; values well above 1 indicate F_3. A negative supply-air deviation δ_{sa} with row-wide overheating supports F_1, because the coil over-cools the small airflow that remains.

2.7.6 Chilled-water side and coil

$$\Delta T_w = T_{wr} - T_{ws}, \tag{E29}$$
$$Q_w = \dot{m}_w\, c_{p,w}\, \Delta T_w, \tag{E30}$$
$$Q_{coil} = \varepsilon\, C_{min}\, (T_{ra} - T_{ws}), \tag{E31}$$
$$NTU = \frac{UA}{C_{min}}, \qquad C^* = \frac{C_{min}}{C_{max}}, \tag{E32}$$
$$\varepsilon = \frac{1 - \exp[-NTU(1 - C^*)]}{1 - C^* \exp[-NTU(1 - C^*)]}, \tag{E33}$$
$$\dot{m}_w^{exp} = \dot{m}_{w,max}\, \mathcal{R}_v^{\theta_v - 1}, \tag{E34}$$
$$\rho_w = \dot{m}_w^{meas} / \dot{m}_w^{exp}. \tag{E34a}$$

Equations E31 to E33 are the effectiveness–NTU model of the coil in counterflow form (Bergman et al., 2011), where $C = \dot{m} c_p$ is a heat capacity rate. Equation E34 is the characteristic of an equal-percentage valve with rangeability $\mathcal{R}_v$ and opening fraction θ_v.

The ratio ρ_w compares measured flow with the flow the valve opening predicts. A ratio far below 1 with the valve open is the signature of F_2. The water temperature rise ΔT_w is *not* a reliable signature of F_2: low flow makes it larger, not smaller. An earlier version of the knowledge base assumed the opposite, and Section 2.8 describes how the simulator exposed the error.

2.7.7 Energy balances and analytical redundancy

$$Q_a = P_{IT} + P_f + Q_{env}, \tag{E35}$$

$$Q_w = Q_a, \tag{E36}$$

$$r_1 = \frac{Q_a - (P_{IT} + P_f)}{P_{IT}}, \tag{E37}$$

$$r_2 = \frac{Q_w - Q_a}{Q_a}. \tag{E38}$$

Equation E35 is the first law over the row, with fan heat in the airstream and a small envelope gain Q_{env}. Equation E36 states that the coil is adiabatic. A residual counts as violated only when it exceeds what sensor uncertainty can explain. For a product $Q = \dot{m} c_p \Delta T$, first-order propagation gives

$$\frac{u_Q}{Q} = \sqrt{\left(\frac{u_{\dot{m}}}{\dot{m}}\right)^2 + \left(\frac{\sqrt{2}\,u_T}{\Delta T}\right)^2}, \qquad |r_i| > \vartheta_i = k_c\, u_{r_i}, \tag{E39–E40}$$

where u denotes a standard uncertainty and k_c a coverage factor. With the sensor accuracies of the modelled row, $u_{r_2} \approx 0.07$. The knowledge base uses $\vartheta_2 = 0.2$, close to $k_c = 3$, because chilled-water loss alone can produce $|r_2| \approx 0.14$.

Two neighbour-consistency relations check individual temperature sensors against sensors that see the same air:

$$r_{S,r} = T_{in,r} - \text{median}_{j \in \mathcal{N}(r)}\, T_{in,j}, \tag{E41}$$

$$r_{sa} = T_{sa} - \text{median}_{r \in \mathcal{R}}\, T_{in,mid,r}. \tag{E41a}$$

A large r_{sa} can come from a biased supply-air sensor or from a real airflow shortfall, which warms the inlets with room and hot-aisle air. The two are separated by recomputing E38 with the supply-air reading replaced by the median middle inlet:

$$r_2' = \frac{Q_w - \rho_a \dot{V}_a c_{p,a} \left(T_{ra} - \text{median}_r\, T_{in,mid,r}\right)}{\rho_a \dot{V}_a c_{p,a} \left(T_{ra} - \text{median}_r\, T_{in,mid,r}\right)}. \tag{E41b}$$

If the sensor is biased, the substitution removes the error and $|r_2'|$ falls within the threshold. If airflow is really lost, the estimated airflow is still wrong and the balance stays broken. The sensor-fault hypothesis is accepted only if assuming it restores consistency, which is consistency-based reasoning applied to a single hypothesis.

2.7.8 Electrical distribution and battery

$$P = \sqrt{3}\, V_{LL}\, I \cos\varphi, \tag{E42}$$

$$L_b = I_b / I_{brk,b}, \tag{E43}$$

$$IU = \frac{\max_p |I_p - \bar{I}|}{\bar{I}}, \tag{E44}$$

$$\eta_{UPS} = P_{UPS,out} / P_{UPS,in}, \qquad P_{UPS,loss} = P_{UPS,in} - P_{UPS,out}, \tag{E45–E46}$$

$$pPUE = \frac{P_{IT} + P_f + P_{UPS,loss}}{P_{IT}}, \tag{E47}$$

$$V_{cell} = V_{str} / n_{cell}, \qquad \rho_R = R_{cell} / R_{cell,0}, \tag{E48–E49}$$

$$t_{run} = H \left(\frac{C_H}{I_{dis}\, H} \right)^{k_p}, \tag{E50}$$

$$\mathcal{L}(T_b) = \mathcal{L}_{25} \cdot 2^{-(T_b - 25)/10}. \tag{E51}$$

A branch loading ratio L_b above 0.8 is an overload (F_6) under the continuous-load rule (National Fire Protection Association, 2023). Phase imbalance IU is a common root cause. On static bypass (F_8), the UPS passes utility power straight through and η_{UPS} rises from about 0.95 to about 1. Each rack's metered power should also equal the sum of its six phase powers, computed with E42 per phase. That relation separates a real overload from a faulty current sensor. A cell resistance ratio ρ_R well above 1 indicates battery degradation (F_7) (IEEE, 2005). Equation E50 is Peukert's law for runtime, and E51 the common rule of thumb that VRLA life halves for every 10 K above 25 °C; both constants are to be confirmed against manufacturer data.

2.7.9 Evidence, noise estimation and ranking

$$a_x = t_{now} - t_x, \tag{E52}$$

$$\delta_x = \frac{x - x_{ref}}{x_{ref}}, \qquad z_x = \frac{x - \mu_x}{\sigma_x}, \tag{E53}$$

$$\hat{x}_k = \alpha\, x_k + (1 - \alpha)\, \hat{x}_{k-1}, \tag{E54}$$

$$e_j = \min\left(1, \max\left(0,\ \tfrac{1}{2} + \frac{d_j}{4\sigma_j}\right)\right), \tag{E55}$$

$$E_k = \textstyle\sum_j w_{jk}\, e_j, \qquad \mathrm{Conf}(k) = \frac{E_k}{E_k + E_{\neg k} + \epsilon}. \tag{E56}$$

A sample older than 60 s is stale (S_2). In E55, d_j is the signed distance of a feature past its threshold and σ_j its noise, so a symptom has evidence 0.5 exactly at its threshold and counts as present only one standard deviation beyond it. The noise is estimated from the case itself: the six middle inlets all see the same supply air, so their spread measures temperature-sensor noise. The robust estimate $\sigma_T = \max(0.15\,\mathrm{K},\ 1.4826\,\mathrm{MAD})$, using the median absolute deviation, cannot be distorted by one biased sensor (Rousseeuw & Croux, 1993). Electrical features use a noise scale estimated in the same way from the rack power balances. In E56, w_{jk} is the weight of symptom j for hypothesis k and $E_{\neg k}$ the evidence against k from expected symptoms that are clearly absent. $\mathrm{Conf}(k)$ ranks competing hypotheses. It is not a probability, and the model makes no probabilistic claim. Equation E54 is a noise filter for repeated readings and is not used, because each test case is a single snapshot.

2.7.10 Where each equation is used

Not every equation is computed by the code. Table 2.8 sorts the 60 equations by use. It is generated by `tests/equation_usage.py` from the equation references in the code comments, so it cannot drift from the implementation. 44 equations are used by the code: 30 by the knowledge base or the inference engines, 12 only by the simulator to generate the data, and E39–E40 only to set the residual threshold that the rules apply. The remaining 16 are descriptive. They complete the engineering model of the row and its standard performance indices, such as the heat indices E14–E15, the rack cooling indices E16–E17, the first-law balance E35–E37, phase imbalance E44, partial PUE E47 and the battery runtime and life models E50–E51, but no diagnostic rule needs them yet. Several would be cheap to add as further evidence, which Section 6.7 lists as future work. Counting only the equations the code uses, the framework's requirement of at least 40 is still met.

Table 2.8: Where each engineering equation is used, generated from the equation references in the code. Reasoning: computed or applied by the knowledge base or an inference engine. Simulator: used to generate the synthetic data. Threshold derivation: used offline to set a threshold that the rules apply. Descriptive: part of the model, but computed by no code. The file-by-file list is in `docs/14_equation_usage.md`.

Used by	Count	Equations
Reasoning and simulator	12	E5, E18, E21, E23, E24, E27, E29, E30, E34, E42, E45, E49
Reasoning only	18	E4, E10, E11, E11a, E12, E13, E20, E28, E34a, E38, E41, E41a, E41b, E43, E52, E53, E55, E56
Simulator only	12	E1, E2, E3, E6, E7, E8, E9, E26, E31, E32, E33, E48
Threshold derivation only	2	E39, E40
Descriptive only	16	E14, E15, E16, E17, E19, E22, E25, E35, E36, E37, E44, E46, E47, E50, E51, E54
All equations	60	

2.7.11 Closed-world normality and evaluation metrics

Normal operation is concluded under the closed-world assumption when no fault can be derived and every inlet is within the envelope:

$$F_N \iff (\forall F_k \in \mathcal{H} :\ KB \cup E \not\models F_k) \wedge (\forall r :\ T_{in,max,r} \le T_{rec,max})\,. \tag{2.6}$$

Performance is measured with the metrics required by the framework. With TP, TN, FP and FN the true and false positives and negatives of a fault class,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \tag{2.7}$$

$$F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{Coverage} = \frac{N_{correct}}{N_{test}}, \quad \text{EMR} = \frac{1}{N_{test}} \sum_i \mathbf{1}[\hat{Y}_i = Y_i], \tag{2.8}$$

$$\text{ExplainabilityRate} = \frac{N_{valid\ explanations}}{N_{diagnoses}}, \qquad \text{PlanValidity} = \frac{N_{valid\ plans}}{N_{plans}}, \tag{2.9}$$

where Y_i and $\hat{Y}_i$ are the true and diagnosed fault sets of case i. Mean and maximum reasoning time complete the set. Chapter 4 defines exactly how each is counted for multiple-fault cases.

Table 2.9: Nomenclature.

Symbol	Meaning	Unit
a_x	Sample age of variable x	s
C, C_{min}, C_{max}, C^*	Heat capacity rate $\dot{m}c_p$; smaller, larger; their ratio	kW/K
C_H, H	Battery rated capacity at rated discharge time H	Ah, h
$c_{p,a}$, $c_{p,w}$	Specific heat of air (1.006) and water (4.186)	kJ/(kg K)
d_j, e_j	Distance past threshold; graded evidence of symptom j	–, –
E_k, $E_{\neg k}$	Weighted evidence for and against hypothesis k	–
$f_{cpu,s}$, $f_{nom,s}$	Measured and nominal CPU clock	GHz
h_r	Hot-rack indicator	–
I, I_p, $\bar{I}$, I_b, $I_{brk,b}$	Line, phase and mean phase current; branch current and rating	A
I_{dis}	Battery discharge current	A
IU	Phase current imbalance	–
K_{filt}, $K_{filt,clean}$	Filter resistance coefficient, current and clean	Pa s^2/m^6
k_c, k_p	Coverage factor; Peukert exponent	–
L_b	Branch loading ratio	–
$\mathcal{L}(T_b)$, $\mathcal{L}_{25}$	Battery service life at T_b and at 25 °C	years
$\dot{m}$, $\dot{m}_a$, $\dot{m}_r$, $\dot{m}_s$, $\dot{m}_w$	Mass flow: general, CRAH air, rack, server, water	kg/s
$\dot{m}_{w,max}$, $\dot{m}_w^{exp}$, $\dot{m}_w^{meas}$	Water flow: valve fully open, predicted, measured	kg/s
N, N_{rated}	CRAH fan speed and rated speed	%
n_R, n_{cell}	Number of racks; number of cells in series	–
NTU	Number of transfer units	–
P, P_s, P_r, P_{IT}	Three-phase power; server, rack and row IT power	kW
P_{idle}, P_{max}, $P_{cpu,s}$	Server idle and full-load power; CPU power	kW
P_f, $P_{f,exp}$, $P_{f,rated}$	Fan power: measured, expected, rated	kW
$P_{UPS,in}$, $P_{UPS,out}$, $P_{UPS,loss}$	UPS input, output and loss power	kW
$pPUE$	Partial power usage effectiveness of the row	–
Q_a, Q_w, Q_{coil}, Q_{env}	Heat flows: air side, water side, coil, envelope	kW
R_{cell}, $R_{cell,0}$	Cell internal resistance, current and baseline	mΩ
R_{th}	CPU junction-to-inlet thermal resistance	K/kW
$\mathcal{R}_v$	Valve rangeability	–
r_1, r_2, r_2'	Energy balance residuals; r_2 with substituted supply air	–
$r_{S,r}$, r_{sa}	Neighbour-consistency residuals	K
RCI_{HI}, RCI_{LO}	Rack cooling indices	%
SHI, RHI	Supply and return heat indices	–
$T_{in,r}$, $T_{in,top,r}$, $T_{in,mid,r}$, $T_{in,max,r}$	Rack inlet temperature: general, top, middle, maximum	°C
$T_{out,r}$, $T_{out,top,r}$, $T_{out,bot,r}$	Rack outlet temperature: mean, upper half, lower half	°C
T_{sa}, T_{ra}, T_{sp}	CRAH supply, return and setpoint temperatures	°C
T_{ws}, T_{wr}	Chilled-water supply and return temperatures	°C
$T_{cpu,s}$, T_b	CPU temperature; battery temperature	°C

Symbol	Meaning	Unit
$T_{rec,max}$, $T_{rec,min}$, $T_{all,max}$, $T_{all,min}$	ASHRAE limits: 27, 18, 32 and 15	°C
t_{run}, t_{now}, t_x	Battery runtime; evaluation time; sample timestamp	h, s, s
u_s	Server CPU utilisation	–
u_Q, u_T, $u_{\dot{m}}$, u_{r_i}	Standard uncertainties	as quantity
UA	Coil overall heat-transfer conductance	kW/K
V_{LL}, V_{str}, V_{cell}	Line voltage; battery string and cell voltage	V
$\dot{V}$, $\dot{V}_a$, $\dot{V}_{rated}$	Volume flow: general, CRAH, rated	m^3/s
w_{jk}	Weight of symptom j for hypothesis k	–
x, x_{ref}, $\hat{x}_k$	Variable, reference value, filtered value	as variable
α	Smoothing weight	–
β_r	Recirculation fraction of rack r	–
Δp_{fan}, Δp_{rated}, Δp_{tot}, Δp_{filt}	Pressure rise or drop: fan, rated, system, filter	Pa
ΔT_s, ΔT_w	Temperature rise across a server; of the chilled water	K
$\Delta T_{strat,r}$, $\Delta T_{strat,out,r}$	Inlet and outlet stratification	K
δ_{sa}, δ_x	Supply-air deviation; relative deviation	K, –
ε, $\varepsilon_{P,r}$, ϵ	Coil effectiveness; rack power residual; small constant	–
η_f, η_{UPS}	Fan and UPS efficiency	–
θ_v	Valve opening fraction	–
ϑ_i	Violation threshold of residual r_i	–
λ_{filt}	Filter loading ratio	–
μ_x, σ_x, z_x, σ_j, σ_T	Mean, standard deviation, z-score; feature and temperature noise	as variable
ρ_a	Air density, 1.2	kg/m^3
ρ_f, ρ_w, ρ_R	Fan power, water flow and cell resistance ratios	–
φ	Phase angle; $\cos\varphi$ is the power factor	rad
ϕ_{hot}, $\phi_{f,s}$	Hot-rack fraction; CPU frequency ratio	–
ω_s	Server fan speed	rpm

2.8 Knowledge Acquisition

The knowledge in the system came from four kinds of source: engineering standards, literature, manufacturer data and explicitly stated engineering assumptions (Figure 2.3). Each piece of knowledge was converted into one of the representations of Table 2.7 and tagged with its source, so that every rule has an identifiable provenance. Table 2.10 gives representative entries. The complete rule and provenance tables are generated from the knowledge base itself and reproduced in Chapter 3.

Engineering assumptions. Where no source gave a value for this particular row, the value was set by engineering reasoning and recorded as a source of type `assumption`. Examples are the 3 K inlet stratification limit, chosen above the 1 to 2 K seen in sealed racks, and the 0.85 fan power ratio, chosen well outside the ±5% scatter of a healthy fan.

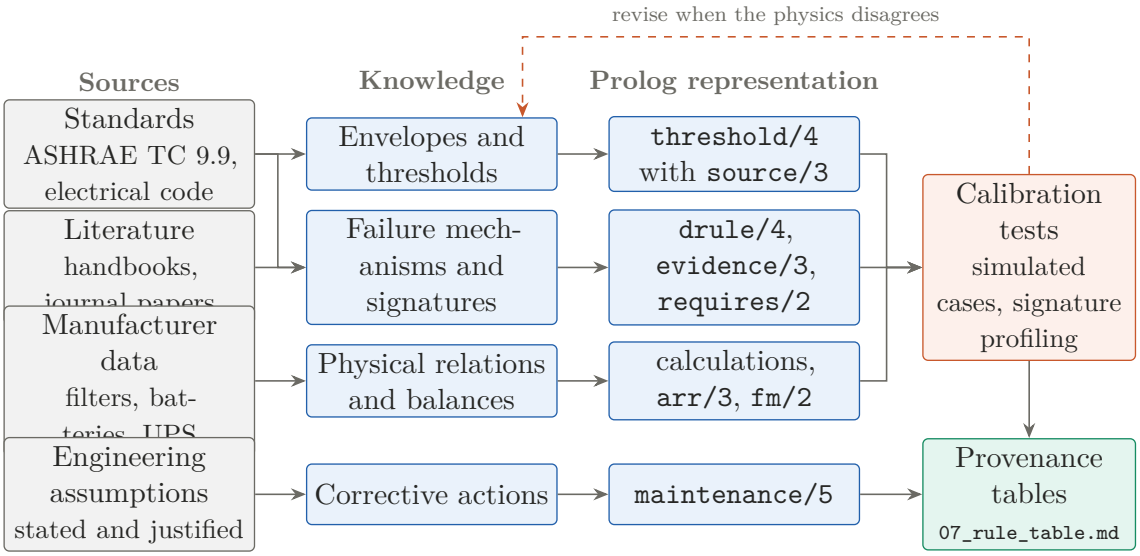


Figure 2.3: Knowledge-acquisition workflow, from sources to Prolog representation, with feedback from calibration tests.

Table 2.10: Representative knowledge-acquisition entries.

Knowledge	Source	Converted into
Recommended inlet envelope 18 to 27 °C	ASHRAE Technical Committee 9.9 (2021)	Threshold facts; rule r01
Fan power scales with speed cubed	ASHRAE (2020)	Equation E23; rules r07, r08, r48, r49
Continuous load at most 80% of rating	National Fire Protection Association (2023)	Threshold fact; rule r17
Rising resistance indicates capacity fade	IEEE (2005)	Equation E49; rule r18
Blanking panels prevent recirculation	Acton et al. (2018), practice 5.1.4	Maintenance plan for F_4
Server power linear in utilisation	Fan et al. (2007)	Equation E1
Minimal diagnoses are minimal hitting sets	Reiter (1987)	Consistency-based module
Parsimonious covering	Peng & Reggia (1990)	Abductive module
Fault models exonerate components	Struss & Dressler (1989)	`fm/2` facts

Of the 47 rules, 36 rest at least partly on such an assumption. Chapter 4 quantifies how sensitive the results are to each one, and shows that the large effects occur only when a threshold moves inside the scatter of normal operation.

Validation against a physics simulator. Because no measured fault data were available, the knowledge was tested against cases generated by a steady-state simulator of the row, built from the equations of Section 2.7 (Chapter 4). This exposed three errors that hand-made test cases had not revealed:

1. The first rule for F_2 expected the chilled-water temperature rise to fall to zero. Physically, low flow makes it rise. The rule was rewritten to compare measured flow with the valve prediction (Equation E34a).

2. A single-feed rack design ran at 81% of its breaker rating in normal operation at high load, which would have made every overload rule fire falsely. The design was changed to dual feeds, as in real data centres.

3. A rule that compared inlet and outlet stratification directly failed, because server fans speed up when their inlet air is hot. The final rules compare outlet stratification on its own with a fixed threshold (Equation E11a).

In each case the knowledge was revised to agree with the physics, which is the feedback loop shown in Figure 2.3. Thresholds were checked only against a calibration dataset. A separate held-out dataset was never used to adjust anything, so that the reported performance is not inflated by tuning.

Chapter 3

Design and Development of the Prolog Expert System

This chapter describes how the knowledge of Chapter 2 was turned into a working expert system. It covers the architecture (Section 3.1), the design of the knowledge base (Section 3.2), the predicates and rules (Section 3.3), the three inference mechanisms and their combination (Sections 3.4 to 3.7), the explanation facility (Section 3.8) and the corrective-maintenance planner (Section 3.9). All diagnostic reasoning is implemented in SWI-Prolog. Python is used only to generate synthetic test data and to format results.

3.1 System Architecture

Figure 3.1 shows the architecture. It follows the ten components that the project framework asks for:

1. a user and input interface: test cases loaded as Prolog facts, and queries such as `why/1`;
2. observation and preprocessing predicates: noise estimation and graded symptoms;
3. engineering-calculation predicates for the equations of Section 2.7;
4. the knowledge base;
5. a deductive inference engine;
6. an abductive diagnosis module;
7. a consistency-based diagnosis module;
8. an explanation facility;
9. a corrective-maintenance planner;
10. an evaluation and reporting module.

The knowledge base is kept separate from the engines that use it, which is the defining property of an expert system (Jackson, 1999). Table 3.1 lists the source files. The knowledge base and engines together are about 1,900 lines of Prolog, and the test and reporting code about 460 more. A single entry file, `main.pl`, loads everything.

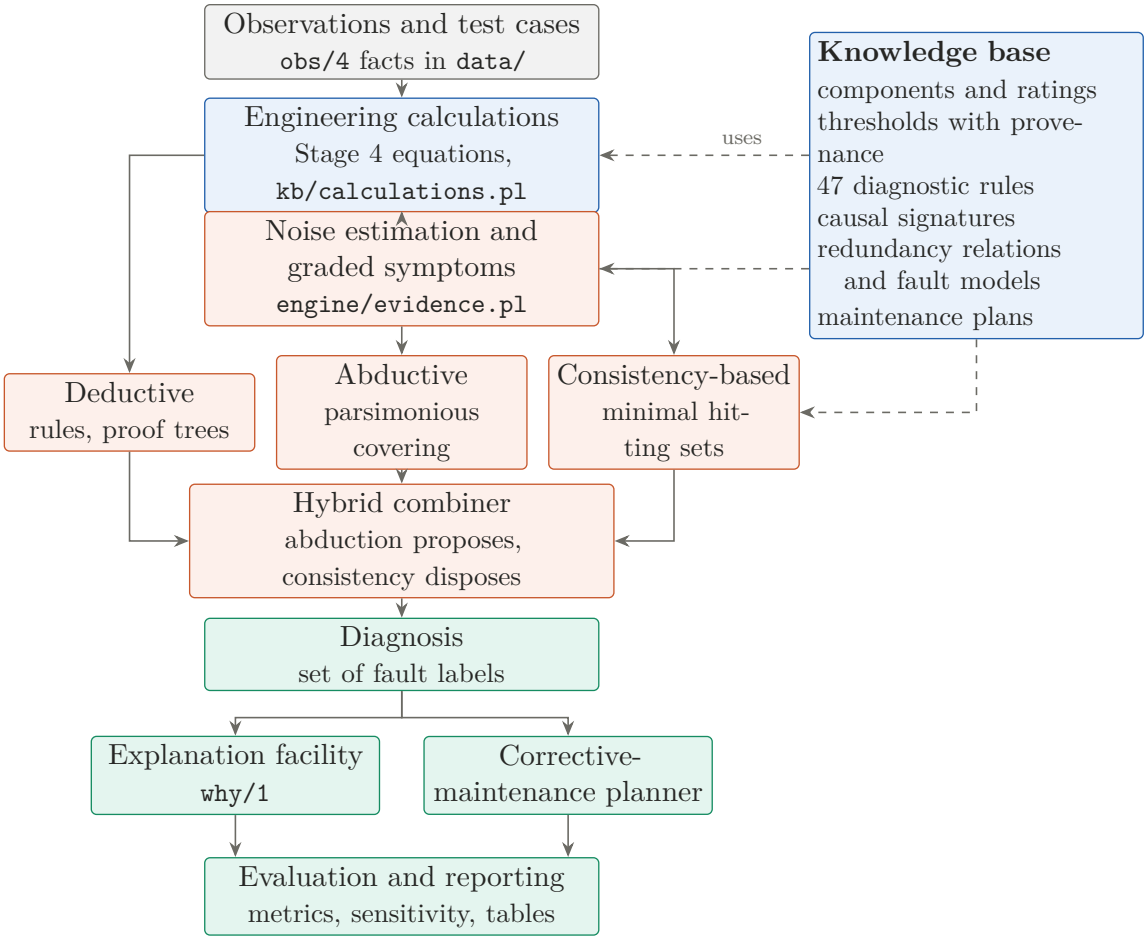


Figure 3.1: Architecture of the Prolog expert system.

Table 3.1: Source modules.

File	Role
`kb/components.pl`	Components, topology, ratings and the variable dictionary
`kb/thresholds.pl`, `kb/sources.pl`	Reference values and their provenance
`kb/calculations.pl`	Engineering equations as predicates
`kb/rules.pl`	Diagnostic rules stored as `drule/4` facts
`kb/symptoms.pl`	Graded symptom definitions with noise models
`kb/causal.pl`	Causal signatures for abduction
`kb/arr.pl`	Redundancy relations and fault models
`kb/maintenance.pl`	Corrective-maintenance plans and plan interactions
`engine/prover.pl`	Proof-recording meta-interpreter
`engine/deductive.pl`	Deductive diagnosis and closed-world conclusions
`engine/evidence.pl`	Noise estimation and evidence grading
`engine/abductive.pl`	Parsimonious covering
`engine/consistency.pl`	Conflicts, minimal hitting sets and fault models
`engine/hybrid.pl`	The four reasoning configurations
`engine/explain.pl`	The explanation facility
`tests/*.pl`	Test runner, metrics, sensitivity and table export

3.2 Knowledge-Base Design

The knowledge base follows the seven layers of Table 2.7. Its contents are summarised in Table 3.2.

Table 3.2: Contents of the knowledge base.

Element	Count	Representation
Diagnostic rules deriving conditions	29	`drule/4`
Diagnostic rules deriving faults	18	`drule/4`
Closed-world conclusions	2	`engine/deductive.pl`
Thresholds	24	`threshold/4`
Sources: standard, literature, method, assumption	3, 1, 3, 17	`source/3`
Engineering-calculation predicates	20	`calc_pred/1`
Graded, yes/no and composite symptoms	20, 2, 2	`sdef/7` and others
Causal evidence links	31	`evidence/3`
Required symptoms, masking links, extra explanations	12, 2, 4	`requires/2` and others
Analytical redundancy relations	20	`arr/3`
Fault models	14	`fm/2`
Maintenance plans	12	`maintenance/5`
Plan interactions	4	`plan_interaction/3`

3.2.1 Facts

Structural knowledge is stated as ground facts, for example `component(crah_1, crah)`, `serves(crah_1, rack_a04)` and `rated(crah_1, fan_power_rated, 5.5, kw)`. Observations are facts `obs(Case, Variable, Entity, Value)`, so every case is a self-contained set of facts and the same knowledge base diagnoses all cases.

3.2.2 Thresholds and provenance

Each threshold is a fact with its value, unit and source, and each source has a type:

```
threshold(t_inlet_rec_max, 27.0, degC, ashrae_tc99_2021).
threshold(strat_out_max,    3.5, kelvin, a7_outlet_strat).
source(a7_outlet_strat, assumption,
       'Engineering assumption: a top-to-bottom outlet difference above 3.5 K
        confirms recirculation. Normal racks stay below 3 K.').
```

Rules and symptoms refer to thresholds by name through `threshold/2`, never by value. A threshold can therefore be changed in one place, and the sensitivity analysis of Chapter 4 can perturb it without touching any rule. The complete threshold and provenance tables are generated from these facts (Tables 3.7 and 3.8).

3.2.3 Engineering calculations

Each equation of Section 2.7 that the reasoning needs is a Horn clause with arithmetic, commented with its equation tag. For example, the fan affinity law and the fan power ratio, Equations E23 and E24, are:

```
%   Eq. 23  P_f,exp = P_f,rated * (N / N_rated)^3      fan affinity law
expected_fan_power(C, U, P) :-
    obs(C, fan_speed, U, N),
    rated(U, fan_power_rated, Pr, _),
    rated(U, fan_speed_rated, Nr, _),
    P is Pr * (N / Nr) ** 3.

%   Eq. 24  rho_f = P_f,measured / P_f,exp
fan_power_ratio(C, U, Ratio) :-
    obs(C, fan_power, U, Pm),
    expected_fan_power(C, U, Pe),
    Pe > 0,
    Ratio is Pm / Pe.
```

A calculation fails when one of its inputs is missing, and a rule that uses it then cannot fire. This is how incomplete evidence is handled in the deductive mode.

3.3 Predicates and Rules

Table 3.3 lists the main predicates. The diagnostic rules are listed in full in Table 3.4, which is generated from `kb/rules.pl`. Each rule is a Horn clause stored as data:

```
%   R24  F4 from exhaust evidence alone, when the top inlet sensor is
%        suspect because it reads low.
drule(r24, fault(C, hot_air_recirculation(R)),
      [ cond(C, outlet_stratified(R)), cond(C, inlet_not_stratified(R)),
        serves(U, R), cond(C, supply_air_normal(U)) ],
      eu_coc_dc).
```

It reads: rack R has hot-air recirculation if its exhaust is stratified, its inlet reading is not, and its cooling unit supplies normal air. The rule exists because a low-reading top sensor can hide real recirculation (Table 2.5). The exhaust then carries the evidence that the inlet sensor does not.

Table 3.4: Diagnostic rules, generated from `kb/rules.pl`. In each rule `A` is the case being diagnosed and `\+` is negation as failure.

Rule	Concludes	If all of these hold	Source type
r01	condition `rack_inlet_hot(B)`	`component(B, rack), inlet_max(A, B, C), threshold(t_inlet_rec_max, D), C>D`	standard
r02	condition `row_inlet_hot`	`hot_rack_fraction(A, B), threshold(row_hot_fraction, C), B>=C`	assumption
r03	condition `localised_inlet_hot(B)`	`cond(A, rack_inlet_hot(B)), hot_rack_fraction(A, C), threshold(row_hot_fraction, D), C<D`	assumption
r04	condition `inlet_stratified(B)`	`inlet_stratification(A, B, C), threshold(strat_max, D), C>D`	assumption
r05	condition `supply_air_hot(B)`	`component(B, crah), obs(A, t_supply_air, B, C), threshold(t_supply_setpoint, D), threshold(t_supply_band, E), C>D+E`	assumption

Rule	Concludes	If all of these hold	Source type
r06	condition supply_air_normal(B)	component(B, crah), obs(A, t_supply_air, B, C), threshold(t_supply_setpoint, D), threshold(t_supply_band, E), C=<D+E	assumption
r07	condition fan_power_low(B)	component(B, crah), fan_power_ratio(A, B, C), threshold(fan_ratio_min, D), C<D	assumption
r08	condition fan_power_normal(B)	component(B, crah), fan_power_ratio(A, B, C), threshold(fan_ratio_min, D), C>=D	literature
r09	condition valve_open(B)	obs(A, valve_position, B, C), threshold(valve_open_min, D), C>=D	assumption
r10	condition chw_flow_low(B)	chw_flow_ratio(A, B, C), threshold(chw_flow_ratio_min, D), C<D	assumption
r20	fault crah_fan_failure(B)	cond(A, row_inlet_hot), cond(A, fan_power_low(B))	literature
r21	fault chw_supply_loss(B)	cond(A, supply_air_hot(B)), cond(A, valve_open(B)), cond(A, chw_flow_low(B)), cond(A, fan_power_normal(B))	assumption
r11	condition outlet_stratified(B)	outlet_stratification(A, B, C), threshold(strat_out_max, D), C>D	assumption
r12	condition outlet_not_stratified(B)	outlet_stratification(A, B, C), threshold(strat_out_max, D), C=<D	assumption
r13	condition inlet_not_stratified(B)	inlet_stratification(A, B, C), threshold(strat_max, D), C=<D	assumption
r14	condition filter_loaded(B)	component(B, crah), filter_loading(A, B, C), threshold(filter_loading_max, D), C>D	assumption
r15	condition server_fan_stopped(B)	obs(A, server_fan_rpm, B, C), threshold(server_fan_min, D), C<D	assumption
r16	condition cpu_hot(B)	obs(A, t_cpu, B, C), threshold(cpu_hot_min, D), C>=D	assumption
r17	condition branch_overloaded(B)	branch_loading(A, B, C), threshold(branch_loading_max, D), C>D	standard
r18	condition cell_resistance_high(B)	component(B, battery), cell_resistance_ratio(A, B, C), threshold(cell_r_ratio_max, D), C>D	assumption
r19	condition ups_efficiency_bypass_like(B)	component(B, ups), ups_efficiency(A, B, C), threshold(ups_eff_bypass_min, D), C>=D	assumption
r40	condition mid_inlet_deviates(B)	component(B, rack), mid_inlet_deviation(A, B, C), threshold(neighbour_dev_max, D), abs(C)>D, \+cond(A, inlet_rise_corroborated(B))	assumption
r45	condition inlet_rise_corroborated(B)	mid_inlet_deviation(A, B, C), top_inlet_deviation(A, B, D), C*D>0, threshold(neighbour_dev_max, E), abs(D)>E	assumption
r46	condition top_inlet_sensor_suspect(B)	component(B, rack), inlet_stratification(A, B, C), threshold(neighbour_dev_max, D), C<-D, \+cond(A, mid_inlet_deviates(B))	method
r41	condition top_inlet_sensor_suspect(B)	cond(A, inlet_stratified(B)), cond(A, outlet_not_stratified(B)), \+cond(A, mid_inlet_deviates(B))	assumption

Rule	Concludes	If all of these hold	Source type
r42	condition top_inlet_sensor_suspect(B)	cond(A, outlet_stratified(B)), cond(A, inlet_not_stratified(B)), \+cond(A, mid_inlet_deviates(B))	assumption
r43	condition supply_sensor_inconsistent(B)	component(B, crah), cond(A, fan_power_nominal(B)), supply_air_deviation(A, B, C), threshold(neighbour_dev_max, D), abs(C)>D, coil_balance_residual(A, B, E), threshold(balance_resid_max, F), abs(E)>F, coil_balance_residual_corrected(A, B, G), abs(G)=<F	method
r48	condition fan_power_nominal(B)	component(B, crah), fan_power_ratio(A, B, C), threshold(fan_ratio_reduced, D), C>=D	assumption
r49	condition fan_power_reduced(B)	component(B, crah), fan_power_ratio(A, B, C), threshold(fan_ratio_reduced, D), C<D	assumption
r47	condition supply_air_cold(B)	component(B, crah), obs(A, t_supply_air, B, C), threshold(t_supply_setpoint, D), threshold(t_supply_band, E), C<D-E	assumption
r44	condition telemetry_stale(B)	obs(A, sample_age, B, C), threshold(sample_age_max, D), C>D	assumption
r36	fault crah_fan_failure(B)	cond(A, row_inlet_hot), cond(A, supply_air_cold(B)), \+cond(A, supply_sensor_inconsistent(B))	assumption
r38	fault crah_fan_failure(B)	cond(A, fan_power_reduced(B)), cond(A, row_inlet_hot)	assumption
r37	fault chw_supply_loss(B)	cond(A, supply_air_hot(B)), cond(A, valve_open(B)), cond(A, fan_power_normal(B)), \+obs(A, m_chw, B, C), \+cond(A, supply_sensor_inconsistent(B))	assumption
r22	fault hot_air_recirculation(B)	cond(A, localised_inlet_hot(B)), serves(C, B), cond(A, supply_air_normal(C)), cond(A, inlet_stratified(B)), \+cond(A, top_inlet_sensor_suspect(B))	standard
r23	fault hot_air_recirculation(B)	cond(A, inlet_stratified(B)), cond(A, outlet_stratified(B)), serves(C, B), cond(A, supply_air_normal(C))	standard
r24	fault hot_air_recirculation(B)	cond(A, outlet_stratified(B)), cond(A, inlet_not_stratified(B)), serves(C, B), cond(A, supply_air_normal(C))	standard
r25	fault filter_clogging(B)	cond(A, filter_loaded(B)), cond(A, fan_power_normal(B))	assumption
r26	fault server_fan_failure(B)	cond(A, server_fan_stopped(B)), cond(A, cpu_hot(B))	assumption
r27	fault branch_overload(B)	cond(A, branch_overloaded(B))	standard
r28	fault battery_degradation(B)	cond(A, cell_resistance_high(B))	assumption
r29	fault ups_on_bypass(B)	component(B, ups), obs(A, ups_mode, B, bypass)	assumption
r35	fault ups_on_bypass(B)	component(B, ups), \+obs(A, ups_mode, B, C), cond(A, ups_efficiency_bypass_like(B))	assumption

Rule	Concludes	If all of these hold	Source type
r50	fault sensor_bias(t_inlet_mid, B)	cond(A, mid_inlet_deviates(B))	assumption
r51	fault sensor_bias(t_inlet_top, B)	cond(A, top_inlet_sensor_suspect(B))	assumption
r52	fault sensor_bias(t_supply_air, B)	cond(A, supply_sensor_inconsistent(B))	method
r53	fault telemetry_dropout(B)	cond(A, telemetry_stale(B))	assumption
r30	fault normal	no fault rule fires, and every rack inlet is within the recommended envelope	method
r31	unexplained_abnormality	no fault rule fires, but some rack inlet is outside the envelope	method

3.4 Inference Mechanisms

Figure 3.2 shows how the three inference mechanisms work together on one case. The readings are first used to estimate measurement noise. Every symptom is then graded. Deductive rules, abduction and consistency checking run on the result, and the hybrid combiner chooses the final diagnosis.

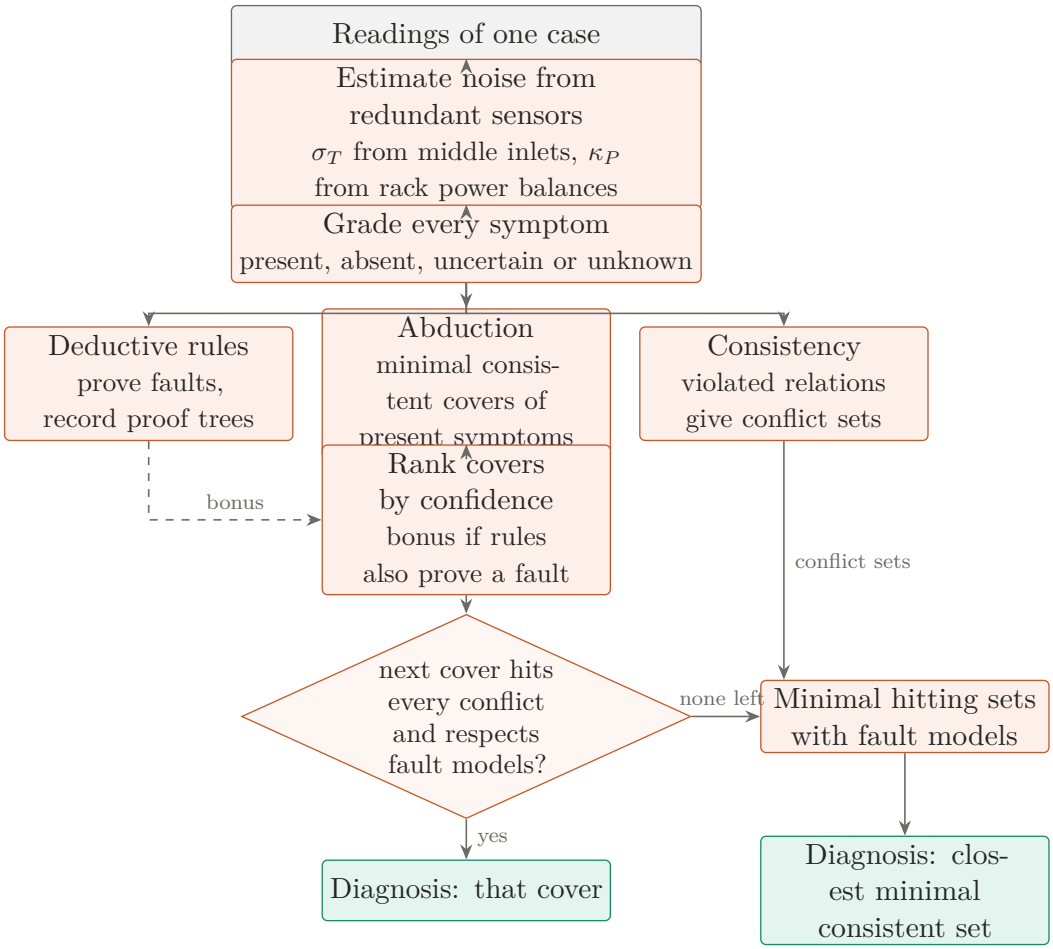


Figure 3.2: Diagnostic reasoning workflow of the complete system: abduction proposes, consistency disposes.

Table 3.3: Main predicates.

Predicate	Purpose
`obs(C, V, E, X)`	Observation: variable V of entity E in case C has value X
`threshold(N, X)`	Value of threshold N, after any sensitivity perturbation
`drule(Id, H, B, S)`	Diagnostic rule $H \leftarrow B$ with identifier and source
`prove(G, P)`	Proves goal G and returns its proof tree P
`diagnose(C, Ds)`	Deductive diagnosis with proofs
`case_symptoms(C, Ss)`	Graded symptoms with evidence and status
`abduce(C, R)`	Abductive candidates, minimal explanations and verdicts
`conflicts(Ss, Ks)`	Conflict sets from violated redundancy relations
`cbd_diagnoses(Ss, Ds)`	Minimal consistent diagnoses
`diagnose_mode(M, C, Ls)`	Diagnosis in configuration M
`why(C)`	Full explanation of the complete system's diagnosis
`maintenance(K, Pr, As, R, S)`	Plan for fault class K: priority, actions, rationale, source

3.4.1 Graded, noise-adaptive evidence

For each symptom definition, `engine/evidence.pl` computes the feature value, compares it with its threshold and grades the evidence with Equation E55. The noise σ_j of each feature is propagated from the datasheet accuracy of its sensors and scaled by the noise level estimated from the case. For temperature features this is the robust spread of the six middle inlets, and for electrical features the spread of the six rack power balances. A symptom is *present* once its value is one standard deviation past the threshold, *absent* once it is one standard deviation on the normal side, and *uncertain* in between. If an input is missing the symptom is *unknown*, which is never treated as absent. The graded symptoms of a case are computed once and cached, so that the abductive and consistency-based modules see exactly the same evidence.

3.4.2 The four configurations

The framework asks for four configurations to be compared, and `engine/hybrid.pl` provides them through `diagnose_mode/3`:

- **Deductive only:** the rules of Table 3.4.
- **Deductive and abductive:** the best parsimonious explanation, with a bonus of 0.1 to the confidence of hypotheses that the rules also prove.
- **Deductive and consistency-based:** the deductive faults are kept if they form a consistent diagnosis, after dropping any fault whose component hits no conflict. Otherwise the minimal consistent diagnosis closest to them replaces them.
- **Complete system:** abduction proposes and consistency disposes. The best-ranked abductive explanation that is also a consistent diagnosis is accepted. If none is consistent, the minimal consistent diagnosis closest to the top explanation is used.

When several consistent diagnoses tie, the one with fewer sensor components wins, because assuming a physical fault is the safer maintenance choice.

3.4.3 Design parameters set by judgement

Besides the thresholds, five parameters of the reasoning were set by engineering judgement rather than derived from a source. They are collected in `engine/params.pl`, so that each can be changed in one place. Table 3.5 states each one with its rationale. Section 4.7.6 measures how much the results depend on them.

Table 3.5: Reasoning parameters set by judgement, with their rationale. Their effect is measured in Table 4.19.

Parameter	Default	Rationale
Deductive bonus	0.1	A hypothesis that a sourced rule also proves deserves a small preference. It is kept well below the typical gap between competing confidences, so that it only breaks near-ties.
Evidence cut-offs	$k = 1$: $e \geq 0.75$, $e \leq 0.25$	A symptom is present one noise standard deviation past its threshold, and absent one standard deviation before it. One standard deviation is the usual compromise between reacting to noise and missing small faults.
Search bound	4 faults	The largest combination in the test design has four faults (Table 4.4). Larger sets cost exponentially more search and are implausible in one snapshot.
Causal weights	`kb/causal.pl`	1.0 for the symptom that defines a fault, 0.4 to 0.8 for strong secondary effects and 0.2 to 0.3 for weak or shared ones, following the fault signatures of Table 2.4.
Noise of features without redundancy	tied to σ_T	Cooling-unit and server features have no redundant sensor from which to estimate their noise. They are scaled with the temperature-sensor noise, on the assumption that instruments degrade together.

3.5 Deductive Reasoning

Deductive diagnosis implements Equation (2.2). Because rules are stored as data, a small meta-interpreter is needed to prove them (Bratko, 2012). It returns a proof tree for every conclusion:

```
derived(cond(_, _)).
derived(fault(_, _)).

prove(\+ G, naf(G)) :-                % negation as failure
    !,
    \+ prove(G, _).
prove(G, by(Id, G, SubProofs)) :-     % a derived goal: try each rule
    derived(G),
    !,
```

```
    drule(Id, G, Body, _),
    prove_body(Body, SubProofs).
prove(G, leaf(G)) :-                    % a fact, calculation or comparison
    call(G).

prove_body([], []).
prove_body([G|Gs], [P|Ps]) :-
    prove(G, P),
    prove_body(Gs, Ps).
```

Conditions and faults are proved through the rules. Everything else, such as observations, thresholds, calculations and comparisons, is called directly and recorded as a leaf. `diagnose/2` collects every fault that can be proved, keeps the first proof of each, and applies the two closed-world conclusions: `normal` when no fault rule fires and every inlet is within the envelope, and `unexplained_abnormality` when no fault rule fires but some inlet is not.

Negation as failure is used in two ways. It guards rules against known confounders. Rule r22, for example, concludes recirculation only if it cannot prove that the top inlet sensor is suspect. It also expresses the closed-world conclusions. Both uses stay within the conditions under which negation as failure is sound (Clark, 1978): the negated goal is ground when it is called.

Three groups of rules deserve comment:

- **Backup rules** use independent evidence when the primary reading is missing. Rule r36 concludes a CRAH fan failure from row-wide overheating with unusually cold supply air, which does not need the fan power reading.
- **Sensor-fault rules** test a sensor against sensors that see the same air. Rule r43 goes further and accepts a supply-air sensor fault only if replacing its reading with the median middle inlet closes the coil energy balance (Equation (E41b)).
- **Physical-constraint rules** use the mixing model. Rule r46 flags a top inlet sensor that reads colder than the middle sensor, which Equation (E18–E19) rules out.

3.6 Abductive Reasoning

Abduction implements Equations (2.3) and (2.4) by parsimonious covering (Peng & Reggia, 1990). The causal knowledge for each hypothesis says which symptoms it predicts, with what weight, and which of them it requires. Recirculation, for example, is described as:

```
evidence(hot_air_recirculation(R), outlet_stratified(R),  1.0).
evidence(hot_air_recirculation(R), inlet_stratified(R),   0.8).
evidence(hot_air_recirculation(R), mid_inlet_deviates(R), 0.3).
requires(hot_air_recirculation(R), outlet_stratified(R)).
requires(hot_air_recirculation(R), inlet_stratified(R)).
masks(sensor_bias(t_inlet_top, R), inlet_stratified(R)).
```

`abduce/2` proceeds in four steps:

1. It collects the clearly present symptoms that need explaining.
2. It generates candidate hypotheses that explain at least one of them, and discards any that require a clearly absent symptom that no other candidate could mask.

3. It searches subsets of the candidates in increasing size, up to four, and keeps every subset that covers all present symptoms and is consistent. A member may require a clearly absent symptom only if another member masks it. The first size that yields a solution gives the minimal explanations.

4. It ranks the minimal explanations by summed confidence (Equation E56). Explanations with equal scores are ordered by their number of sensor faults, fewest first, the same rule the consistency-based mode uses. It records a verdict for every candidate: part of an explanation, not needed, or rejected together with the absent symptom that ruled it out.

Uncertain and unknown symptoms never need explaining, so a reading near the noise level cannot force an extra fault into the answer. Masking lets the module find a hidden fault: when the top inlet reads low over a recirculating rack, the explanation {recirculation, top-sensor bias} is consistent although recirculation alone is not.

3.7 Consistency-Based Reasoning

Consistency-based diagnosis implements Equation (2.5). Each analytical redundancy relation names the symptom that signals its violation and the components that must be healthy for it to hold. Each fault model names a relation that must be violated if a component is abnormal:

```
arr(a_out_strat(R), outlet_stratified(R),
    [rack_seal(R), sensor(t_outlet_top, R), sensor(t_outlet, R)]).
arr(a_rack_power(R), rack_power_mismatch(R), [sensor(p_rack, R) | Phases]) :-
    component(R, rack),
    findall(sensor(i_branch, B), phase_of_rack(R, B), Phases).

fm(rack_seal(R), a_out_strat(R)).
fm(sensor(i_branch, B), a_rack_power(R)) :- phase_of_rack(R, B).
```

Every clearly violated relation yields a conflict set. `cbd_diagnoses/2` then searches for minimal hitting sets (Reiter, 1987). It picks a conflict that the current candidate does not yet hit and branches on its members. When a candidate hits every conflict but breaks a fault model, because a component's predicted violation clearly did not happen, it branches on adding a component that could mask the effect. Candidates are capped at four components, and supersets of other diagnoses are removed.

Fault models are what let the module tell sensor faults from equipment faults (Struss & Dressler, 1989). A branch overload produces the conflict {branch, current sensor}. A faulty current sensor would also break the rack power balance, so when that balance holds, the sensor is exonerated and only the overload remains. In the same way, a high top-inlet reading over a rack whose exhaust is not stratified exonerates the rack seal, so the diagnosis is a sensor fault.

3.8 Explanation Facility

The explanation facility answers the question "why was this diagnosis reached?". `why/1` follows the workflow of Figure 3.3. For every reported fault that the rules prove, it reads the proof tree and reports the observed readings, the derived engineering quantities, the

thresholds with their sources, the derived conditions, the rules fired with their provenance and the full reasoning trace. For a fault found only by abduction and consistency checking, it reports the graded supporting symptoms, each with its value, threshold, estimated noise, evidence strength and source. It then gives the maintenance plan and any plan interactions, the competing hypotheses with their confidence and the reason each was used, not needed or rejected, and the conflict sets from the consistency check.

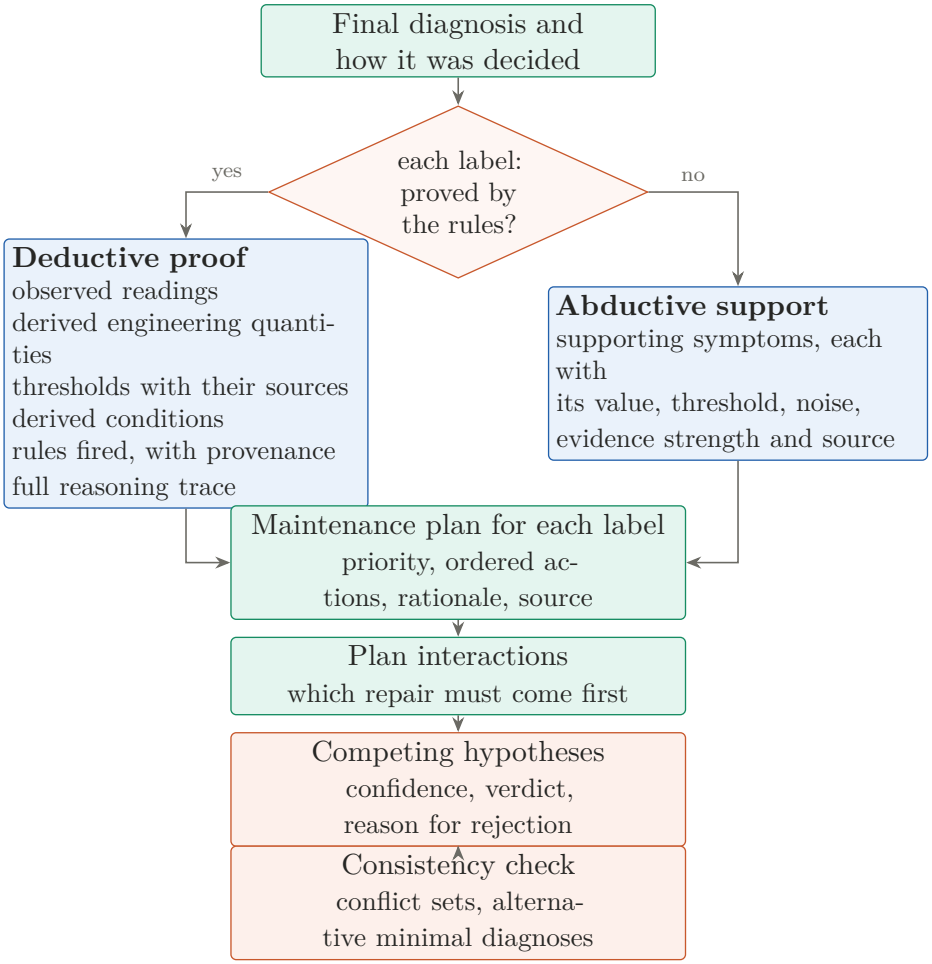


Figure 3.3: Explanation-generation workflow of the explanation facility.

Listing 3.1 shows the explanation of a held-out test case in which recirculation at rack A04 is hidden by a top inlet sensor that reads low. The output is reproduced unedited. It shows each of the eight elements that the framework requires of an explanation: observed evidence, derived conditions, rules activated, intermediate reasoning, the diagnosis, competing hypotheses, the reasons for rejecting weaker ones and the recommended action.

Listing 3.1: Unedited output of `why(h188)`: recirculation hidden by a low-reading sensor.

```
=== Case h188: hot-air recirculation + sensor bias masking, high load  [data origin: synthetic] ===

FINAL DIAGNOSIS (complete system): [hot_air_recirculation(rack_a04),sensor_bias(t_inlet_top,rack_a04)]
  Decided by: the best-ranked abductive explanation, confirmed consistent with every conflict and fault
     model

DIAGNOSIS: hot_air_recirculation(rack_a04)

  Observed evidence:
    - CRAH supply-air temperature of crah_1 = 19.734 degC
```

```
    - rack outlet temperature of rack_a04 = 38.715 degC
    - rack outlet temperature, top of rack_a04 = 41.208 degC
    - rack inlet temperature, middle of rack_a04 = 19.867 degC
    - rack inlet temperature, top of rack_a04 = 20.725 degC

  Derived engineering quantities:
    - outlet_stratification(h188,rack_a04,4.986)
    - inlet_stratification(h188,rack_a04,0.858)

  Reference values:
    - strat_out_max = 3.5 kelvin  [source: a7_outlet_strat]
    - strat_max = 3.0 kelvin  [source: a5_stratification]
    - t_supply_setpoint = 20.0 degC  [source: a1_supply_setpoint]
    - t_supply_band = 2.0 kelvin  [source: a1_supply_setpoint]

  Derived conditions:
    - outlet_stratified(rack_a04)  (by r11)
    - inlet_not_stratified(rack_a04)  (by r13)
    - supply_air_normal(crah_1)  (by r06)

  Rules activated:
    - r24  [source: eu_coc_dc, standard]
    - r11  [source: a7_outlet_strat, assumption]
    - r13  [source: a5_stratification, assumption]
    - r06  [source: a1_supply_setpoint, assumption]

  Reasoning trace:
    fault(hot_air_recirculation(rack_a04))  [r24]
      cond(outlet_stratified(rack_a04))  [r11]
        outlet_stratification(h188,rack_a04,4.986)
        threshold(strat_out_max,3.5)
        4.986>3.5
      cond(inlet_not_stratified(rack_a04))  [r13]
        inlet_stratification(h188,rack_a04,0.858)
        threshold(strat_max,3)
        0.858=<3
      serves(crah_1,rack_a04)
      cond(supply_air_normal(crah_1))  [r06]
        component(crah_1,crah)
        obs(h188,t_supply_air,crah_1,19.734)
        threshold(t_supply_setpoint,20)
        threshold(t_supply_band,2)
        19.734=<20+2

  Recommended corrective maintenance for rack_a04  [priority: high]:
    1. Inspect the rack for missing blanking panels and unsealed cable openings
    2. Check containment doors and roof panels for gaps
    3. Install blanking panels and seal the openings
    4. Re-measure top and middle inlet temperatures to confirm the difference is below 3 K
  Rationale: Hot exhaust air re-entering the front of the rack heats the upper servers while the cooling
     unit itself is healthy.  [source: eu_coc_dc]

DIAGNOSIS: sensor_bias(t_inlet_top,rack_a04)

  Observed evidence:
    - rack outlet temperature of rack_a04 = 38.715 degC
    - rack outlet temperature, top of rack_a04 = 41.208 degC
    - rack inlet temperature, middle of rack_a04 = 19.867 degC
    - rack inlet temperature, top of rack_a04 = 20.725 degC

  Derived engineering quantities:
    - outlet_stratification(h188,rack_a04,4.986)
    - inlet_stratification(h188,rack_a04,0.858)

  Reference values:
    - strat_out_max = 3.5 kelvin  [source: a7_outlet_strat]
    - strat_max = 3.0 kelvin  [source: a5_stratification]

  Derived conditions:
    - top_inlet_sensor_suspect(rack_a04)  (by r42)
    - outlet_stratified(rack_a04)  (by r11)
    - inlet_not_stratified(rack_a04)  (by r13)
```

```
Rules activated:
  - r51  [source: a7_outlet_strat, assumption]
  - r42  [source: a7_outlet_strat, assumption]
  - r11  [source: a7_outlet_strat, assumption]
  - r13  [source: a5_stratification, assumption]

Reasoning trace:
  fault(sensor_bias(t_inlet_top,rack_a04))  [r51]
    cond(top_inlet_sensor_suspect(rack_a04))  [r42]
      cond(outlet_stratified(rack_a04))  [r11]
        outlet_stratification(h188,rack_a04,4.986)
        threshold(strat_out_max,3.5)
        4.986>3.5
      cond(inlet_not_stratified(rack_a04))  [r13]
        inlet_stratification(h188,rack_a04,0.858)
        threshold(strat_max,3)
        0.858=<3
      not cond(h188,mid_inlet_deviates(rack_a04))

Recommended corrective maintenance for t_inlet_top on rack_a04  [priority: high]:
  1. Check the sensor against a calibrated reference instrument
  2. Check the sensor mounting and position
  3. Recalibrate or replace the sensor
  4. Do not act on readings from this sensor until it is verified
Rationale: One sensor disagrees with independent sensors that measure the same air, so the sensor is
   more likely wrong than the equipment.  [source: a13_neighbour]

Plan interactions (order the work as follows):
  - Fix the top inlet sensor on rack_a04 first. The last step of the recirculation plan re-measures with
    that sensor.

Clearly present symptoms: [outlet_stratified(rack_a04),top_reads_low(rack_a04)]

Competing hypotheses (Conf is Stage 4 Eq. 56, a ranking score, not a probability):
  - sensor_bias(t_inlet_top,rack_a04)  Conf 0.99: part of a minimal explanation
  - hot_air_recirculation(rack_a04)  Conf 0.55: part of a minimal explanation

Consistency check: 1 conflict set(s) from violated redundancy relations.
  - at least one of [rack_seal(rack_a04),sensor(t_outlet_top,rack_a04),sensor(t_outlet,rack_a04)] is
   abnormal
```

3.9 Corrective-Maintenance Planner

Every fault class has a plan in `kb/maintenance.pl`, with a priority, ordered actions, an engineering rationale and a source:

```
maintenance(branch_overload, urgent,
    [ 'Identify the servers on the overloaded phase from the rack power records',
      'Rebalance server power supplies across the three phases',
      'Check for equipment added without a capacity review',
      'Confirm every phase is below 80% of its breaker rating' ],
    'A phase above its continuous rating can trip its breaker and drop every
     server on that circuit.',
    nec_continuous).
```

Each plan ends with a verification step that confirms the repair worked, such as re-measuring a temperature or a current. Figure 3.4 shows how plans are assembled. A diagnosed label is split into its fault class and entity, and the plan for the class is instantiated for the entity.

Table 3.6 lists the plan issued for every diagnosis. It is generated from the knowledge base, like the rule table.

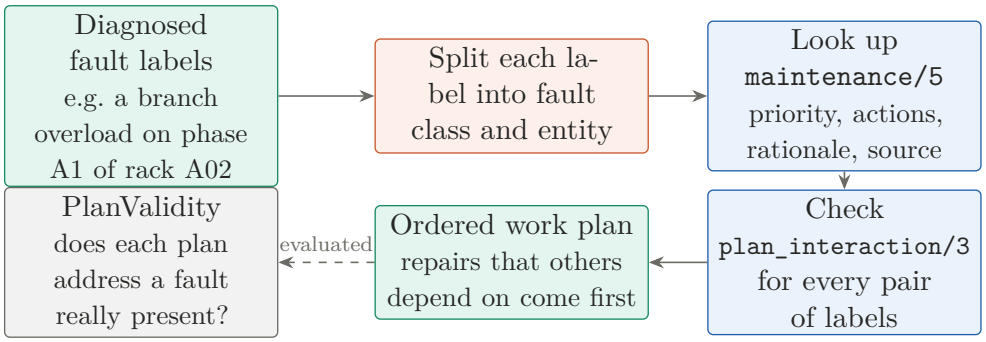


Figure 3.4: Corrective-maintenance planning workflow.

Table 3.6: Maintenance-action mapping, generated from `kb/maintenance.pl`: the corrective plan issued for each diagnosis, in order.

Diagnosis	Priority	Ordered actions	Source
`crah_fan_failure`	urgent	1. Check the fan drive fault log and the motor current 2. Inspect the motor, belt or coupling, and bearings; repair or replace the failed part 3. Bring redundant cooling online or raise the output of neighbouring units during the repair 4. After repair, confirm fan power matches the affinity-law prediction and rack inlets fall below 27 C	`ashrae_hb_fans`
`chw_supply_loss`	urgent	1. Confirm chilled-water plant status: pumps running and isolation valves open 2. Compare commanded and actual valve stroke; repair or replace a seized actuator 3. Vent trapped air from the coil 4. Confirm water flow matches the valve position and supply air is back at setpoint	`a4_chw_flow`
`hot_air_recirculation`	high	1. Inspect the rack for missing blanking panels and unsealed cable openings 2. Check containment doors and roof panels for gaps 3. Install blanking panels and seal the openings 4. Re-measure top and middle inlet temperatures to confirm the difference is below 3 K	`eu_coc_dc`
`normal`	routine	1. No corrective action. Continue routine monitoring.	`ashrae_tc99_2021`
`unexplained_abnormality`	high	1. Verify the affected sensors before acting 2. Escalate to a technician for manual investigation	`closed_world`
`filter_clogging`	high	1. Replace the CRAH air filters 2. Check the filter differential pressure gauge against a reference 3. Review the filter replacement interval and recent dust sources such as building work 4. After replacement, confirm filter pressure drop and fan speed are back to normal	`a8_filter`
`server_fan_failure`	high	1. Check the server management controller event log for the failed fan module 2. Migrate or reduce the workload on the server until it is repaired 3. Hot-swap the failed fan module 4. Confirm CPU temperature and clock frequency return to normal	`a9_server_fan`

Diagnosis	Priority	Ordered actions	Source
`branch_overload`	urgent	1. Identify the servers on the overloaded phase from the rack power records 2. Rebalance server power supplies across the three phases 3. Check for equipment added without a capacity review 4. Confirm every phase is below 80% of its breaker rating	`nec_continuous`
`battery_degradation`	high	1. Confirm with a capacity discharge test as IEEE Std 1188 recommends 2. Check battery room temperature and its cooling 3. Replace weak blocks or the whole string, depending on the test result 4. Record a new resistance baseline after replacement	`a11_battery`
`ups_on_bypass`	urgent	1. Read the UPS alarm log to find why it transferred to bypass 2. Call the UPS service engineer to repair the inverter or rectifier fault 3. Postpone any upstream electrical work while the load has no battery protection 4. Return the UPS to double conversion after the repair and confirm normal efficiency	`a12_ups`
`sensor_bias`	high	1. Check the sensor against a calibrated reference instrument 2. Check the sensor mounting and position 3. Recalibrate or replace the sensor 4. Do not act on readings from this sensor until it is verified	`a13_neighbour`
`telemetry_dropout`	high	1. Check the monitoring agent, controller and network link for this telemetry group 2. Restore the data feed 3. Treat the equipment status as unknown until fresh data arrives 4. Inspect critical equipment on site if the feed cannot be restored quickly	`a14_telemetry`

Plan interactions. When two diagnoses interact, their plans must be ordered. The verification step of the recirculation plan re-measures the rack's top inlet temperature. If the same rack's top inlet sensor is also diagnosed as biased, that step would rely on a faulty sensor. `plan_interaction/3` records such dependencies, and the planner then says to repair the sensor first, as Listing 3.1 shows. Similar interactions cover the supply-air sensor, whose reading the cooling-unit plans rely on, and stale telemetry, which leaves any plan for the affected equipment unverifiable.

Plan validity. Whether each issued plan addresses a fault that is really present is measured as PlanValidity in Chapter 4. A plan for a fault that is not present wastes maintenance effort. A missing plan for a fault that is present leaves it unrepaired.

Generated tables

Tables 3.7 and 3.8 are generated from the knowledge base by `tests/export_latex.pl`, like Table 3.4.

Table 3.7: Thresholds, generated from `kb/thresholds.pl`.

Threshold	Value	Unit	Source	Source type
t_inlet_rec_max	27.0	degC	ashrae_tc99_2021	standard
t_inlet_rec_min	18.0	degC	ashrae_tc99_2021	standard
t_supply_setpoint	20.0	degC	a1_supply_setpoint	assumption
t_supply_band	2.0	kelvin	a1_supply_setpoint	assumption
row_hot_fraction	0.5	ratio	a2_row_fraction	assumption
valve_open_min	50.0	percent	a3_valve_open	assumption
chw_flow_ratio_min	0.3	ratio	a4_chw_flow	assumption
strat_max	3.0	kelvin	a5_stratification	assumption
fan_ratio_min	0.5	ratio	a6_fan_ratio	assumption
strat_out_max	3.5	kelvin	a7_outlet_strat	assumption
filter_loading_max	1.7	ratio	a8_filter	assumption
server_fan_min	1000.0	rpm	a9_server_fan	assumption
cpu_hot_min	85.0	degC	a10_cpu	assumption
branch_loading_max	0.8	ratio	nec_continuous	standard
cell_r_ratio_max	1.3	ratio	a11_battery	assumption
ups_eff_bypass_min	0.98	ratio	a12_ups	assumption
neighbour_dev_max	2.0	kelvin	a13_neighbour	assumption
balance_resid_max	0.2	ratio	eq40_uncertainty	method
sample_age_max	60.0	s	a14_telemetry	assumption
t_supply_hot_limit	22.0	degC	a1_supply_setpoint	assumption
fan_ratio_reduced	0.85	ratio	a17_fan_reduced	assumption
t_supply_cold_limit	18.0	degC	a1_supply_setpoint	assumption
power_resid_max	0.1	ratio	a15_power_balance	assumption
sensor_noise_floor	0.15	kelvin	a16_noise	assumption

Table 3.8: Provenance of rules and thresholds, generated from `kb/sources.pl`.

Source	Type	Citation or justification
ashrae_tc99_2021	standard	ASHRAE TC 9.9 (2021). Thermal guidelines for data processing environments (5th ed.). ASHRAE.
ashrae_hb_fans	literature	ASHRAE (2020). 2020 ASHRAE Handbook - HVAC Systems and Equipment, fans chapter. Fan affinity laws.
eu_coc_dc	standard	Acton et al. (2018). 2018 Best Practice Guidelines for the EU Code of Conduct on Data Centre Energy Efficiency, v9.1.0 (European Commission JRC), practice 5.1.4.
a1_supply_setpoint	assumption	Design assumption: CRAH supply-air setpoint 20 C with a 2 K tolerance band.
a2_row_fraction	assumption	Engineering assumption: overheating at half or more of the racks indicates a row-level cause.

Source	Type	Citation or justification
a3_valve_open	assumption	Engineering assumption: a valve open 50% or more should produce a measurable water temperature rise.
a4_chw_flow	assumption	Engineering assumption: measured water flow below 30% of the valve-characteristic prediction means chilled-water supply is lost.
a5_ stratification	assumption	Engineering assumption: a top-to-middle inlet difference above 3 K indicates hot-air recirculation.
a6_fan_ratio	assumption	Engineering assumption: fan power below 50% of the affinity-law prediction means the fan is not moving air.
closed_world	method	Closed-world assumption: normal operation is concluded when no fault rule fires and all inlets are within the envelope.
nec_continuous	standard	NFPA (2023). NFPA 70 National Electrical Code, continuous-load rule: 80% of breaker rating. Local codes may differ.
a7_outlet_strat	assumption	Engineering assumption: a top-to-bottom outlet difference above 3.5 K confirms recirculation. Normal racks stay below 3 K.
a8_filter	assumption	Engineering assumption: filter resistance 1.7 times its clean value calls for replacement (check against manufacturer final-resistance data).
a9_server_fan	assumption	Engineering assumption: a server fan below 1000 rpm has failed. The controller never runs it below about 5400 rpm.
a10_cpu	assumption	Engineering assumption: CPU temperature of 85 C or more, 10 K below the 95 C throttling limit, confirms lost server cooling.
a11_battery	assumption	Engineering assumption: cell internal resistance 30% above its baseline indicates degradation (check against IEEE Std 1188 and manufacturer guidance).
a12_ups	assumption	Engineering assumption: UPS efficiency of 0.98 or more is only reached on static bypass. Double conversion runs at about 0.95.
a13_neighbour	assumption	Engineering assumption: a temperature sensor more than 2 K from the median of sensors that see the same air is suspect (Stage 4 Eq. 41).
eq40_uncertainty	method	Residual threshold from uncertainty propagation, Stage 4 Eqs. 39 and 40, with a coverage factor of about 3.
a14_telemetry	assumption	Engineering assumption: a sample older than 60 s is stale, given a polling interval of about 30 s.
eq18_mixing	method	Physical constraint from the mixing model, Stage 4 Eq. 18: the top inlet cannot be colder than the middle inlet.

Source	Type	Citation or justification
a15_power_balance	assumption	Engineering assumption: metered rack power and the sum of its six phase currents agree within 10% when both meters are healthy (Stage 4 Eq. 42).
a16_noise	assumption	Engineering assumption: temperature-sensor noise is at least 0.15 K (typical datasheet accuracy). Higher noise is estimated from the spread of middle inlets, which all see the same supply air.
a17_fan_reduced	assumption	Engineering assumption: fan power below 85% of the affinity-law prediction shows a partial failure such as belt slip. Healthy fans stay within 5%.

Chapter 4

Dataset, Experimental Methodology and Evaluation

This chapter describes the data the expert system was tested on (Sections 4.1 and 4.2), how the experiments were designed and validated (Sections 4.3 and 4.4), how performance was measured (Section 4.5) and what the results show (Sections 4.6 and 4.7). Every table of results in this chapter is generated by the test code from the current knowledge base and datasets, so the numbers can be reproduced exactly (Chapter 5).

4.1 Dataset and Data Dictionary

4.1.1 Data origin

Every test case in this project is synthetic. No measured fault data from a real data centre were available, and no value in the dataset is presented as a real measurement. The project framework requires four kinds of information to be kept apart, and they are kept apart as follows:

- **Measured data:** none.
- **Synthetic test cases:** generated by a physics simulator, or written by hand for four smoke tests. Every case carries the origin `synthetic` in its `case/3` fact.
- **Engineering assumptions:** recorded as sources of type `assumption` in the knowledge base (Table 3.8), and listed for the simulator in Table 4.2.
- **Literature-supported rules:** rules whose source is of type `standard` or `literature`.

4.1.2 The simulator

The test cases were generated by a steady-state simulator of the row, written in Python with only the standard library. It implements the engineering model of Section 2.7. Each case is built in four steps:

1. **Workload.** The row load is drawn as low, nominal or high, at mean utilisations of 10 to 20%, 40 to 60% or 80 to 90%. Each rack varies around the row value and each server around its rack.

2. **Fault injection into the physics.** A fault changes a physical parameter, never a reading directly. A fan failure, for example, reduces the airflow the fan delivers, and every temperature then follows from the physics. Table 4.1 lists the parameter changed for each fault.

3. **Steady-state solution.** The coupled server, rack, cooling-unit and coil equations are solved by damped fixed-point iteration. The cooling-unit fan follows the airflow demand of the servers, and the valve controller finds the smallest opening that holds the 20 °C setpoint.

4. **Sensor layer.** Sensor bias, Gaussian measurement noise, frozen readings for stale telemetry and randomly missing readings are applied last.

Because faults act on the physics, their symptoms arise as consequences rather than being written in. The diagnostic rules are therefore tested against behaviour they were not built from, which is what exposed the three knowledge-base errors described in Section 2.8.

Table 4.1: Physical parameter changed for each injected fault.

Fault	Parameter changed	Clear range	Borderline range
F_1 CRAH fan failure	Airflow fraction delivered; fan power over prediction	0 to 0.1; 0.05 to 0.15	0.3 to 0.5; 0.4 to 0.6
F_2 chilled-water loss	Fraction of design water flow available	0.02 to 0.2	0.25 to 0.4
F_3 filter clogging	Filter loading ratio, Equation E27	3 to 6	1.8 to 2.5
F_4 recirculation	Extra recirculation fraction at the top of one rack	0.25 to 0.5	0.12 to 0.2
F_5 server fan failure	Server airflow and reported fan speed near zero	–	–
F_6 branch overload	Phase 1 share of feed A, set for a target current	27 to 30 A	24.5 to 26.5 A
F_7 battery degradation	Cell resistance ratio, with low float voltage	1.5 to 2.5	1.2 to 1.4
F_8 UPS on bypass	Mode flag; efficiency rises to 0.99	–	–
S_1 sensor bias	Offset on one inlet or supply-air sensor	3 to 8 K	–
S_2 telemetry dropout	One telemetry group frozen, sample age 300 to 1800 s	–	–

4.1.3 Data dictionary

The variables, units and sensors are defined in Table 2.6. Each case has about 460 readings: nine for the cooling unit, six for each rack, one current for each of the six phase circuits of each rack, three for each of the 120 servers, six for the UPS and battery, and one sample age for each telemetry group. The simulator writes every dataset in three forms: Prolog facts for the expert system, a CSV file in long format with units for analysis outside Prolog, and a JSON file holding the hidden ground truth of each case, such as the injected severities.

Table 4.2: Main simulator assumptions.

Assumption	Value	Reason
Server power	0.40 kW idle, 0.90 kW at full load, 55% in the CPU	Equation E1; idle power about half of peak (Barroso & Hölzle, 2007)
Server airflow	0.055 kg/s at nominal fan speed	Gives a 13 to 16 K rise across a server
CPU throttling	Clock reduced to keep the CPU at or below 95 °C	Below the 102 °C maximum of a server processor (Intel Corporation, 2024)
Cooling-unit fan control	Supplies 10% more air than the servers demand	Common practice to limit recirculation
Coil heat transfer	15 kW/K air side; 10 kW/K water side at 3.2 kg/s, scaling with flow to the power 0.8	Gives a 5 to 7 K water temperature rise in normal operation
Airflow shortfall	40% drawn from the hot aisle, 60% from room air at 25 °C	Without it a fan failure has no steady state
Power distribution	Two feeds per rack sharing the load equally	Standard 2N practice
Measurement noise	0.15 K on air temperatures; 1% on power and current	Within class A tolerance (International Electrotechnical Commission, 2022) and accuracy class 1 (International Electrotechnical Commission, 2020)
Time	Steady state only	A limitation: transients are not modelled

4.1.4 Datasets

Four datasets were used (Table 4.3). The calibration and held-out splits have the same composition but different random seeds. A fixed seed always reproduces the same cases.

Table 4.3: Datasets.

Dataset	Cases	Seed	Use
Smoke tests	4	–	Hand-written cases for the explanation demonstrations
Calibration	255	20261020	Checking thresholds and signatures during development
Held-out	255	777	Reported performance; never used to adjust anything
Sensitivity sweeps	24 × 60	4242 to 4545	Sensitivity to noise, missing data, multiple faults and sensor bias

4.2 Test Cases

The calibration and held-out splits each contain seven categories, which together cover the eight test categories that the project framework requires (Table 4.4). Test H, explanation and maintenance, is assessed on every case through ExplainabilityRate and PlanValidity, and demonstrated in full by the explanation traces of Section 3.8.

Table 4.4: Test categories and the framework tests they cover.

Category	Cases	Test	Contents
Normal	30	A	10 cases each at low, nominal and high load
Single fault	120	B, C, D	15 cases of each physical fault F_1 to F_8
Instrumentation	30	E	15 sensor-bias and 15 telemetry-dropout cases
Multiple	30	F	6 cases of each combination in Table 2.5
Incomplete	15	G	A random single fault or normal case with 15% of readings removed
High noise	15	G	A random single fault with five times the nominal sensor noise
Borderline	15	G	Fault severities near or below the detection thresholds, and high-load normal cases
All categories	255	H	Explanation and maintenance plan for every diagnosis

The borderline category deliberately includes faults injected below the detection thresholds. Its exact-match score is therefore expected to be low, and it is reported to measure how gracefully detection degrades, not as a target to maximise.

4.3 Experimental Design

Figure 4.1 shows the experimental workflow. Three design decisions protect the results against optimistic bias.

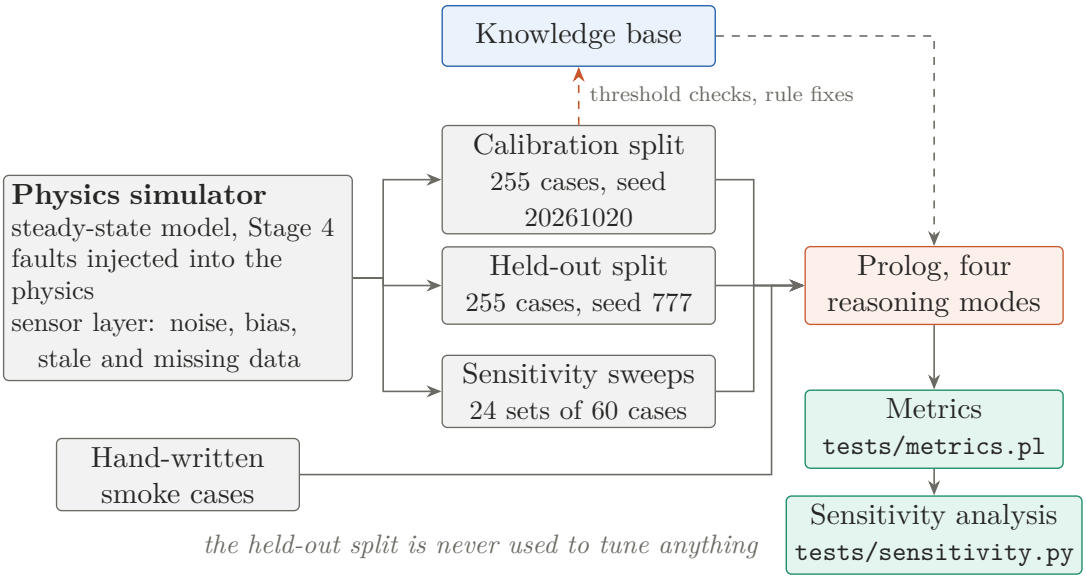


Figure 4.1: Experimental and evaluation workflow, with the calibration and held-out splits kept apart.

Calibration and held-out splits. Thresholds were set from engineering reasoning and then checked against the calibration split, by profiling each signature in normal and fault cases. The held-out split was generated afterwards with a different seed and was never used to choose or adjust anything. Its results are the reported performance.

Faults in the physics, not the readings. Because the simulator injects faults into the physical model, the knowledge base cannot simply reproduce the generator's rules. Symptoms that the knowledge base did not anticipate appear in the data, as the three errors found during development show.

Four configurations on identical data. The four reasoning configurations of Section 3.4 run on exactly the same cases, so differences between them are due to the reasoning alone. This comparison is the ablation study that the framework asks for.

Reasoning time is measured as wall-clock time per case, after a warm-up pass and with the symptom cache cleared for each case, so that every configuration pays the full cost of its own evidence computation.

4.4 Validation

Verification asks whether the system implements its design correctly. Validation asks whether the design is right for the problem. Both were addressed at several levels:

- **Equations against physics.** The simulator's normal cases were checked against independent hand calculations. A 78 kW row needs about 4.9 m^3/s of air at a 13 K rise, and the fan figures imply a plausible pressure rise of about 225 Pa. The coil model was tuned so that normal operation gives the 5 to 7 K water temperature rise of real chilled-water coils.

- **Rules against simulated physics.** Each new rule was first run against the calibration split. Rules that disagreed with the physics were revised rather than the data, which exposed the three errors of Section 2.8.
- **Generalisation.** The held-out split measures performance on cases that played no part in development. Held-out accuracy is slightly higher than calibration accuracy for every configuration (Tables 4.5 and 4.6), which makes overfitting to the calibration split unlikely. Both splits come from the same simulator, however, so this tests generalisation across cases of one model, not to a different system. Section 4.8 tests robustness to the model itself.
- **Explanations.** Explanation traces were inspected for representative cases of every category. The trace of Listing 3.1 shows each element that the framework requires.
- **Regression.** The four smoke cases and both splits are rerun after every change to the knowledge base.

The main limitation of this validation is that both the knowledge base and the simulator rest on the same engineering model. Agreement between them shows that the reasoning is consistent with the model, not that the model is consistent with a real data centre. Section 4.8 measures how much the results depend on the model being exactly right, by regenerating the held-out cases with deliberately changed physics. Validation on measured data remains future work (Chapter 7).

4.5 Metrics

The metrics follow Section 2.7. Because a case can contain several faults, each metric is counted as follows:

- **Exact match** is the share of cases whose predicted set of fault labels equals the true set exactly. A label names both the fault and the affected entity, so diagnosing recirculation at the wrong rack counts as wrong.
- **Coverage** is the share of cases in which every true fault was predicted, whether or not extra faults were predicted too.
- **Precision, recall and F1-score** are computed for each fault class from label-level counts: a predicted label that matches a true label is a true positive. Macro averages are taken over the eleven classes.
- **Class accuracy** is the framework's Equation (2.7) at case level: for each class, the share of cases in which the presence or absence of that class was predicted correctly. Its mean over the classes is reported.
- **ExplainabilityRate** is the share of cases in which every reported label has a justification, meaning a deductive proof, a clearly present supporting symptom or a hit conflict set, together with a maintenance plan and sourced thresholds. The system only reports a label for one of these reasons, so this rate cannot fall below 100%. It confirms that every output is traceable, not that the explanation is the real reason for the output. Explanation fidelity, below, tests that.

- **Explanation fidelity** removes the observations an explanation cites and diagnoses the case again (Section 4.6.5). Unlike ExplainabilityRate, it can fail.
- **PlanValidity** is the share of issued maintenance plans that address a fault really present. A routine plan counts as valid only for a truly normal case. One plan is issued per label, and a plan counts as valid when its label is correct, so PlanValidity is in effect label-level precision. It does not test whether a plan's actions are the right ones. That rests on the plans' sources and verification steps (Section 3.9), and ideally on review by a practitioner.
- **Reasoning time** is the mean and maximum wall-clock time per case.

Confidence scores are used only to rank hypotheses. No probability is claimed, so probabilistic metrics such as calibration curves are not reported.

4.6 Results

4.6.1 Overall performance

Table 4.5 gives the held-out results of the four configurations, and Table 4.6 the calibration results for comparison. Each exact-match rate is given with its Wilson 95% confidence interval. Because all four configurations diagnose the same cases, they are compared pairwise with an exact McNemar test on the cases where they disagree (Table 4.7). Six pairs are compared, so each table also gives p-values adjusted by Holm's step-down method, which keeps the chance of any false positive among the six at 5%. A difference is called significant below only if it remains so after this correction.

Table 4.5: Comparison of reasoning modes, held-out split (255 cases). 95% CI: Wilson confidence interval of the exact-match rate. Explain.: ExplainabilityRate. Plan val.: PlanValidity.

Mode	Exact	95% CI	Coverage	Macro F1	Mean ms	Max ms	Explain.	Plan val.
Deductive only	91.0%	86.8–93.9	95.7%	0.924	0.18	1.13	100.0%	85.1%
Deductive + abductive	94.5%	91.0–96.7	96.1%	0.958	0.91	2.03	100.0%	94.2%
Deductive + consistency-based	93.3%	89.6–95.8	97.3%	0.950	0.89	5.34	100.0%	91.4%
Complete system	93.7%	90.1–96.1	96.5%	0.954	1.03	10.00	100.0%	92.9%

Four findings stand out.

1. **Every configuration is accurate on the core categories.** All four diagnose every normal, single-fault and instrumentation case of the held-out split correctly (Table 4.8). Over five held-out seeds, they miss 3 to 6 of the 900 such cases (Section 4.6.2).
2. **Abduction gives the largest gain.** Adding abduction raises exact-match accuracy from 91.0% to 94.5%. It corrects 11 cases that deduction gets wrong and loses 2 ($p = 0.022$). On one split of 255 cases that is not enough to survive the correction for six comparisons (Holm $p = 0.135$), but over five seeds the gain is clearly significant

Table 4.6: Comparison of reasoning modes, calibration split (255 cases). 95% CI: Wilson confidence interval of the exact-match rate. Explain.: ExplainabilityRate. Plan val.: PlanValidity.

Mode	Exact	95% CI	Coverage	Macro F1	Mean ms	Max ms	Explain.	Plan val.
Deductive only	90.2%	85.9–93.3	95.3%	0.925	0.18	0.68	100.0%	86.7%
Deductive + abductive	93.3%	89.6–95.8	94.5%	0.954	0.95	2.52	100.0%	94.4%
Deductive + consistency-based	91.8%	87.7–94.6	95.3%	0.942	0.84	2.48	100.0%	91.6%
Complete system	93.3%	89.6–95.8	95.3%	0.952	0.96	3.20	100.0%	93.8%

Table 4.7: Paired comparison of reasoning modes, held-out split: exact two-sided McNemar test on the cases where the two modes disagree. Only A: cases the first mode gets right and the second wrong. Holm: p adjusted for the six comparisons by Holm's method. Bold: below 0.05.

Mode A	Mode B	Only A	Only B	p	Holm
Deductive only	Deductive + abductive	2	11	**0.022**	0.135
Deductive only	Deductive + consistency-based	2	8	0.109	0.461
Deductive only	Complete system	3	10	0.092	0.461
Deductive + abductive	Deductive + consistency-based	5	2	0.453	1.000
Deductive + abductive	Complete system	2	0	0.500	1.000
Deductive + consistency-based	Complete system	2	3	1.000	1.000

(Holm $p < 0.001$, Section 4.6.2). The gain comes almost entirely from the imperfect-evidence categories, where parsimony removes spurious findings. The complete system's gain over deduction is likewise not significant on either single split (Holm $p = 0.461$ held-out, 0.231 calibration), but is over five seeds (Holm $p = 0.009$).

3. **One split cannot separate the model-based configurations; five seeds can.** On the held-out split their exact-match rates differ by two to five cases, their confidence intervals overlap, and no pairwise difference between them approaches significance ($p \geq 0.45$). Pooled over five held-out seeds, deductive + abductive reasoning is significantly more accurate than each of the other configurations (Section 4.6.2). Consistency checking is the only mode that catches the smallest supply-sensor bias in the calibration split, and it refuses explanations that leave a violated relation unexplained. Under heavy noise, however, it occasionally takes a chance violation of the power balance at face value and reports a meter fault.

4. **Every output is traceable, and the explanations are faithful.** ExplainabilityRate is 100% in every configuration, as it must be by construction. The fidelity test, which can fail, finds that removing the evidence an explanation cites removes the diagnosis in 96 to 100% of cases (Section 4.6.5). Reasoning takes about 1 ms per case on average and at most 10 ms.

Table 4.8: Exact matches by test category and reasoning mode, held-out split.

Category	Test	Cases	Deductive	Ded. + abd.	Ded. + cons.	Complete
Normal operation	A	30	30	30	30	30
Single faults	B–D	120	120	120	120	120
Instrumentation	E	30	30	30	30	30
Multiple faults	F	30	30	29	30	29
Incomplete evidence	G	15	13	14	14	14
High noise	G	15	3	9	5	7
Borderline	G	15	6	9	9	9

4.6.2 Variability across seeds

The held-out split is one random draw of 255 cases. To measure how much the results depend on that draw, four more held-out splits were generated with the same design and the seeds 1001, 2002, 3003 and 4004. None of them was used to choose anything. Table 4.9 gives the exact match of each configuration on each seed, and pooled over all 1275 cases. They are generated by `tests/seeds.py`.

The standard deviation across seeds is 1.2 to 1.6 percentage points. That is the same size as the differences between the model-based configurations on any one split, which is why one split cannot separate them. The order is stable where it matters most. On every seed, deductive + abductive reasoning is ahead of the complete system, and the complete system is ahead of deductive + consistency-based reasoning. Deduction alone is last on four seeds, but on seed 3003 it is narrowly first, which shows how much a single split can mislead.

Table 4.9: Exact match on five held-out splits of 255 cases with different seeds. Seed 777 is the held-out split reported in Table 4.5. SD: sample standard deviation over the five seeds. Pooled: all 1275 cases, with the Wilson 95% confidence interval.

Mode	777	1001	2002	3003	4004	Mean	SD	Pooled (95% CI)
Deductive only	91.0	90.6	92.2	91.4	89.0	90.8	1.2	90.8 (89.1–92.3)
Deductive + abductive	94.5	94.9	94.1	91.0	93.3	93.6	1.6	93.6 (92.1–94.8)
Deductive + consistency-based	93.3	91.4	92.5	90.2	92.2	91.9	1.2	91.9 (90.3–93.3)
Complete system	93.7	93.7	93.3	90.6	92.9	92.9	1.3	92.9 (91.3–94.2)

Table 4.10: Paired comparison of reasoning modes pooled over the five held-out seeds (1275 cases): exact two-sided McNemar test on the cases where the two modes disagree. Holm: p adjusted for the six comparisons by Holm's method. Bold: below 0.05.

Mode A	Mode B	Only A	Only B	p	Holm
Deductive only	Deductive + abductive	17	52	**< 0.001**	**< 0.001**
Deductive only	Deductive + consistency-based	14	28	**0.044**	0.071
Deductive only	Complete system	21	47	**0.002**	**0.009**
Deductive + abductive	Deductive + consistency-based	29	8	**0.001**	**0.004**
Deductive + abductive	Complete system	9	0	**0.004**	**0.012**
Deductive + consistency-based	Complete system	8	20	**0.036**	0.071

Pooling the paired cases gives the statistical power that one split lacks (Table 4.10). Every pairwise difference is then below 0.05, and four of the six remain significant after Holm's correction: deductive + abductive reasoning over each of the other three configurations, and the complete system over deduction alone. The two comparisons of deductive + consistency-based reasoning with deduction and with the complete system do not survive the correction (Holm $p = 0.071$), so its position between them is suggestive only. Deductive + abductive reasoning is the most accurate configuration, at 93.6%. The complete system is never right where deductive + abductive reasoning is wrong, and is wrong in 9 cases where it is right. In each of the 9, it keeps the correct fault and adds a false sensor fault, and 8 of them are high-noise cases. On these data, the consistency check therefore costs more exact matches under noise than it gains. Its benefits show in the controlled experiments, such as detecting the smallest sensor biases (Section 4.7). Graded consistency checking (Section 6.7) is the change most likely to remove its cost.

The core categories are not perfect on every seed either. Over the five seeds, 3 to 6 of the 900 normal, single-fault and instrumentation cases are misdiagnosed, depending on the configuration. The perfect score on seed 777 is therefore partly the luck of one draw.

4.6.3 Results by fault class

Table 4.11 gives the per-class results of the complete system and Table 4.12 its confusion matrix. Every single-label case of the held-out split is diagnosed correctly: the confusion matrix is diagonal. The errors that remain occur in the multiple-fault and imperfect-evidence categories, and show in two classes. Sensor bias has a precision of 0.656, because 11 sensor faults were reported that were not present, mostly in high-noise cases. Normal has

a precision of 0.829, because seven faulty cases were reported as normal, mostly borderline faults injected below the detection thresholds. The class-accuracy column should be read with care. Each class is absent from most cases, so true negatives dominate it: sensor bias still has a class accuracy of 96.5% despite its precision of 0.656. Precision, recall and F1-score are the informative measures.

Table 4.11: Per-class results of the complete system, held-out split. Counts are of fault labels; class accuracy is at case level (Equation (2.7)). Because each class is absent from most cases, class accuracy is dominated by true negatives and is high even for a weak class; precision, recall and F1 are the informative measures.

Class	TP	FP	FN	Precision	Recall	F1	Class acc.
F_N normal	34	7	0	0.829	1.000	0.907	97.3%
F_1 CRAH fan failure	27	1	0	0.964	1.000	0.982	99.6%
F_2 chilled-water loss	23	0	0	1.000	1.000	1.000	100.0%
F_3 filter clogging	25	0	3	1.000	0.893	0.943	98.8%
F_4 recirculation	36	2	2	0.947	0.947	0.947	98.4%
F_5 server fan failure	24	0	0	1.000	1.000	1.000	100.0%
F_6 branch overload	23	0	2	1.000	0.920	0.958	99.2%
F_7 battery degradation	25	0	2	1.000	0.926	0.962	99.2%
F_8 UPS on bypass	23	0	0	1.000	1.000	1.000	100.0%
S_1 sensor bias	21	11	0	0.656	1.000	0.792	96.5%
S_2 telemetry dropout	15	0	0	1.000	1.000	1.000	100.0%
Macro average				0.945	0.971	0.954	

Table 4.12: Confusion matrix of the complete system, held-out split, single-label cases (normal, single-fault and instrumentation categories). Rows are the true class and columns the predicted class. *Mult.* means more than one label was predicted and *Unex.* an unexplained abnormality.

True \ predicted	F_N	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	S_1	S_2	Mult.	Unex.
F_N	30	·	·	·	·	·	·	·	·	·	·	·	·
F_1	·	15	·	·	·	·	·	·	·	·	·	·	·
F_2	·	·	15	·	·	·	·	·	·	·	·	·	·
F_3	·	·	·	15	·	·	·	·	·	·	·	·	·
F_4	·	·	·	·	15	·	·	·	·	·	·	·	·
F_5	·	·	·	·	·	15	·	·	·	·	·	·	·
F_6	·	·	·	·	·	·	15	·	·	·	·	·	·
F_7	·	·	·	·	·	·	·	15	·	·	·	·	·
F_8	·	·	·	·	·	·	·	·	15	·	·	·	·
S_1	·	·	·	·	·	·	·	·	·	15	·	·	·
S_2	·	·	·	·	·	·	·	·	·	·	15	·	·

4.6.4 Where the system fails

- **High noise.** At five times the nominal noise, the outlet stratification of Equation E11a, which doubles a difference of two readings, scatters by about 2 K. The deductive

rules then report recirculation and sensor faults that are not there. The noise-adaptive grading and parsimony of the abductive mode remove most of these, but not all.

- **Borderline faults** injected below the thresholds are reported as normal. This is the intended behaviour of a threshold-based system, and its sensitivity is quantified in Section 4.7.
- **Missing key readings.** When the only reading that reveals a fault is missing, such as the cell resistance for battery degradation, no mode can detect it.
- **One physical interaction is not modelled.** A failed server fan inside a recirculating rack reduces the exhaust stratification. One such case in the held-out split is reported as a sensor fault rather than recirculation.

4.6.5 Explanation fidelity

ExplainabilityRate shows that every output can be traced, but not that the trace is the reason for the output. An explanation could cite evidence the diagnosis does not depend on. Explanation fidelity tests this directly, in the spirit of deletion tests for explanations in machine learning. For every fault label a configuration reports on the held-out split, the observations its explanation cites are removed and the case is diagnosed again. A removed observation is missing, which the system treats as unknown, as it would a failed sensor. The cited observations are those the explanation facility reports for the label: the observed evidence of its deductive proof, the readings behind every clearly present symptom it predicts, and the readings behind every violated redundancy relation that involves its component. Two outcomes are measured:

- **Necessity:** the label disappears when its cited observations are removed;
- **Control:** the label survives when the same number of uncited observations, chosen at random, are removed instead. This shows that the label does not vanish whenever any readings go missing.

Table 4.13: Explanation fidelity on the held-out split. Necessity: share of reported fault labels that disappear when the observations their explanation cites are removed. The next three columns say what replaced them. Control: share that survive when the same number of uncited observations are removed at random.

		Cited		Replaced by			
Mode	Labels	(mean)	Necessity	normal	unexpl.	other	Control
Deductive only	277	6.2	95.7%	177	2	86	100.0%
Deductive + abductive	250	6.4	100.0%	189	0	61	100.0%
Deductive + consistency-based	263	6.5	100.0%	184	0	79	100.0%
Complete system	256	6.5	100.0%	187	0	69	99.6%

Table 4.13 gives the results. An explanation cites about six of the roughly 460 readings in a case. In the three model-based configurations, removing them removes the label every time, while removing six other readings leaves it in place in all but one case. Most labels are then replaced by a normal diagnosis, and the rest by a different label, such as the real fault that a false sensor alarm had been hiding.

The test also found two weaknesses, both reportable rather than fatal:

- **Single-proof explanations can be incomplete.** In 12 cases, deduction alone kept its UPS-bypass diagnosis after the cited reading, the UPS mode, was removed. Rule R35 then inferred bypass from the UPS efficiency, which is its purpose when the mode reading is missing. The diagnosis had two sufficient reasons, and the deductive trace showed only the first proof found. The model-based configurations cite the efficiency evidence as well, so they do not have this gap. Reporting every proof of a label, not only the first, would close it.
- **Noise estimates are not cited.** The one control failure is a false meter-fault label in a high-noise case of the complete system. The random removal took out one phase current of another rack, so that rack's power balance could not be computed. That changed the electrical noise estimate, which is a median over the racks, and the meter fault fell below the presence threshold. The explanation cites the readings a symptom is computed from, but not those that set its noise scale.

Fidelity is measured against the system's own decision, not against the truth: it shows that the explanations are faithful to the reasoning, not that the reasoning is right. Whether the explanations are useful to a maintenance engineer still has to be assessed with users (Section 6.6).

4.7 Sensitivity and Ablation Analysis

The comparison of the four configurations in Section 4.6 is the ablation study. This section varies the conditions instead: sensor noise, missing observations, multiple simultaneous faults, the size of a sensor bias and the value of each threshold. Each sweep level uses 60 new synthetic cases. One seed is used for all levels of a sweep, so neighbouring levels differ mainly in the swept factor. With 60 cases, one case moves accuracy by about 1.7 percentage points, so smaller differences between configurations are within the resolution of the experiment.

4.7.1 Sensor noise

Table 4.14: Sensitivity to sensor noise: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage.

Noise multiple	Ded.	+Abd.	+Cons.	Complete	Cov.	Macro F1	ms
1	100.0%	100.0%	100.0%	100.0%	100.0%	1.000	1.36
2	91.7%	98.3%	93.3%	98.3%	100.0%	0.985	1.22
3	58.3%	81.7%	60.0%	80.0%	96.7%	0.894	1.21
4	41.7%	73.3%	51.7%	70.0%	95.0%	0.867	1.24
5	25.0%	55.0%	36.7%	55.0%	90.0%	0.787	1.42
6	13.3%	50.0%	30.0%	43.3%	90.0%	0.776	2.84
8	5.0%	33.3%	16.7%	23.3%	81.7%	0.693	5.91

Noise degrades deduction fastest (Figure 4.2 and Table 4.14). At four times the nominal noise, deduction keeps 42% exact match while the abductive configuration keeps 73%.

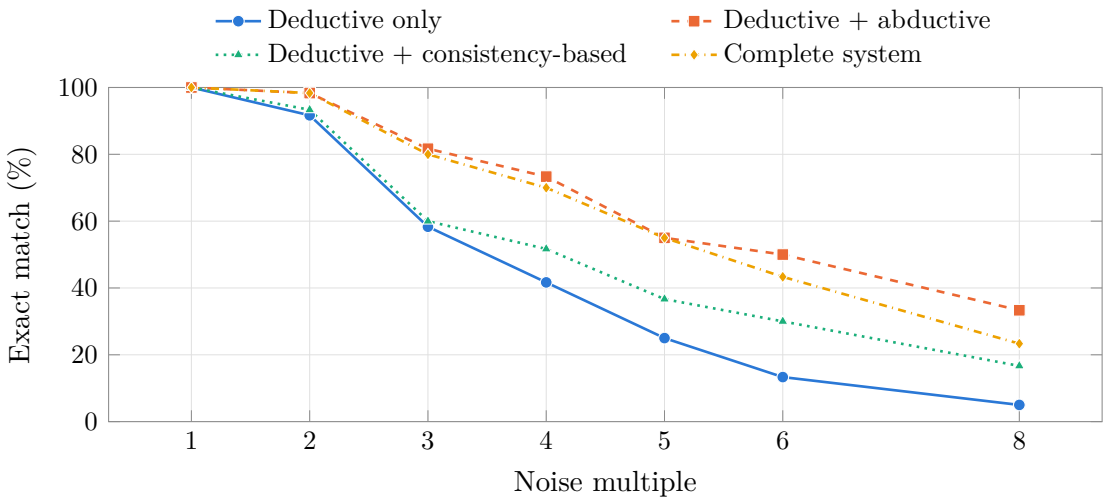


Figure 4.2: Exact-match accuracy against sensor noise, by reasoning mode. Each point is 60 synthetic cases.

Noise is a precision problem, not a recall problem. The deductive rules still find the true fault in most noisy cases, but crisp thresholds turn noise into additional false findings. Graded evidence, scaled to the noise estimated from the case itself, and parsimony keep those findings out of the abductive explanations.

4.7.2 Missing observations

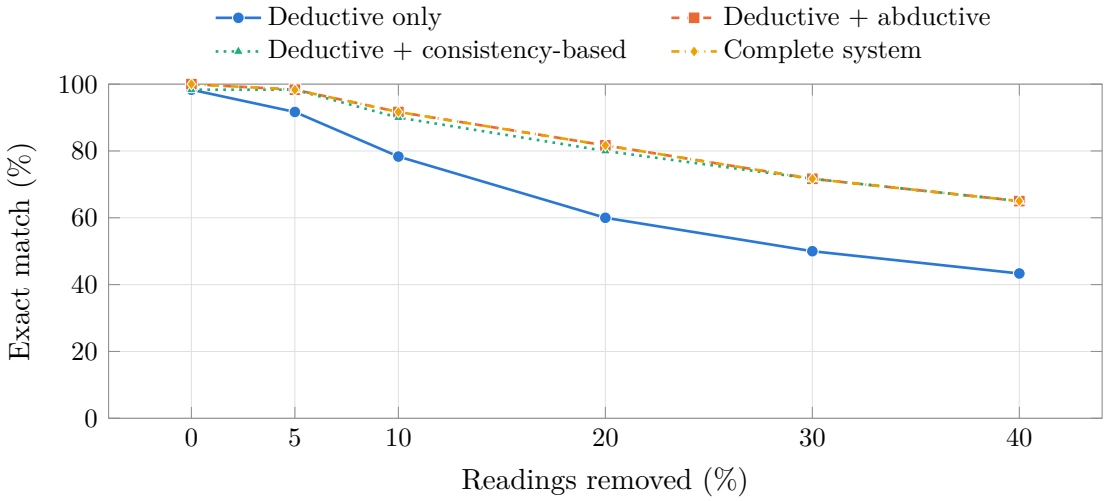


Figure 4.3: Exact-match accuracy against the share of readings removed, by reasoning mode. The three model-based configurations give almost identical results, so their lines overlap.

The lead of the three model-based configurations over deduction widens with the share of missing readings, from about 7 percentage points at 5% missing to about 22 points at 20% and beyond (Figure 4.3). A missing reading makes its symptom unknown rather than absent, so no hypothesis is rejected for lack of data, and a hypothesis can be accepted on its remaining evidence. A deductive rule fails as soon as one reading in its body is missing.

Table 4.15: Sensitivity to missing observations: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage.

Readings removed	Ded.	+Abd.	+Cons.	Complete	Cov.	Macro F1	ms
0%	98.3%	100.0%	98.3%	100.0%	100.0%	1.000	1.11
5%	91.7%	98.3%	98.3%	98.3%	98.3%	0.982	1.09
10%	78.3%	91.7%	90.0%	91.7%	91.7%	0.914	1.11
20%	60.0%	81.7%	80.0%	81.7%	81.7%	0.818	0.94
30%	50.0%	71.7%	71.7%	71.7%	71.7%	0.739	1.17
40%	43.3%	65.0%	65.0%	65.0%	65.0%	0.681	0.82

4.7.3 Multiple simultaneous faults

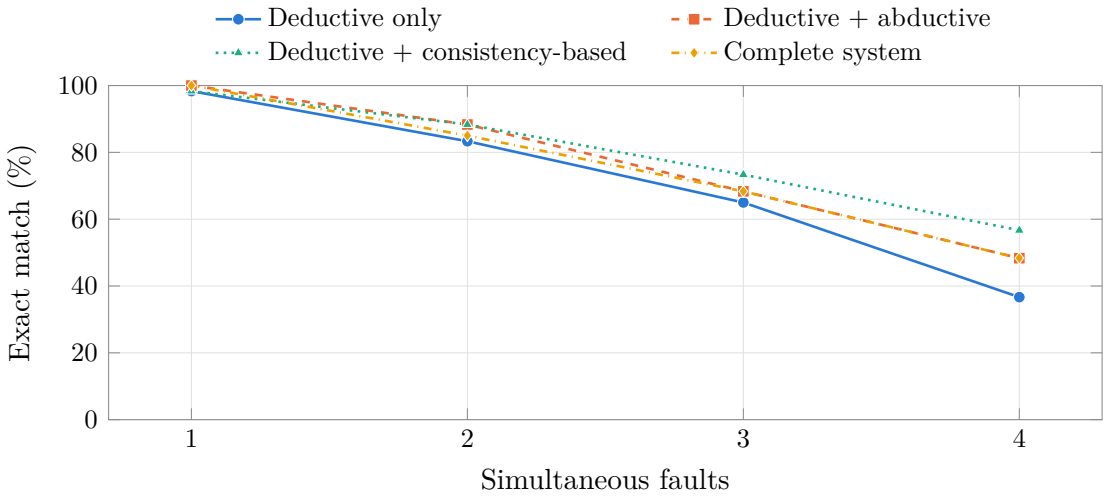


Figure 4.4: Exact-match accuracy against the number of simultaneous faults, by reasoning mode.

Exact-match accuracy falls as faults are combined (Figure 4.4). Precision stays close to 1, so the system rarely reports a fault that is not present, while recall falls as interacting faults share symptoms. Two cooling faults together, such as a fan failure and a chilled-water loss, overlap in what they show, and parsimony then prefers one fault to two. The consistency-based configuration handles four faults best, because a fault model forces it to keep a second fault whose own relation was violated. Reasoning remains fast at every fault count (Figure 4.5). The timing curves are wall-clock measurements and include some noise from other processes on the test machine, such as the dip at two faults. They should therefore be read only as an order of magnitude: a few milliseconds per case at every fault count, far below the sampling interval of data centre monitoring. Differences of a fraction of a millisecond between configurations or fault counts are not meaningful.

4.7.4 Contradictory evidence: sensor bias

A biased sensor contradicts its neighbours and the energy balances. Biases below the 2 K neighbour-deviation threshold are rarely detected, which is the expected limit of the design, and biases of 4 K or more are detected almost always (Figure 4.6). At 3 K the

Table 4.16: Sensitivity to the number of simultaneous faults: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage.

Simultaneous faults	Ded.	+Abd.	+Cons.	Complete	Cov.	Macro F1	ms
1	98.3%	100.0%	98.3%	100.0%	100.0%	1.000	1.35
2	83.3%	88.3%	88.3%	85.0%	93.3%	0.983	1.43
3	65.0%	68.3%	73.3%	68.3%	76.7%	0.951	1.40
4	36.7%	48.3%	56.7%	48.3%	48.3%	0.846	1.73

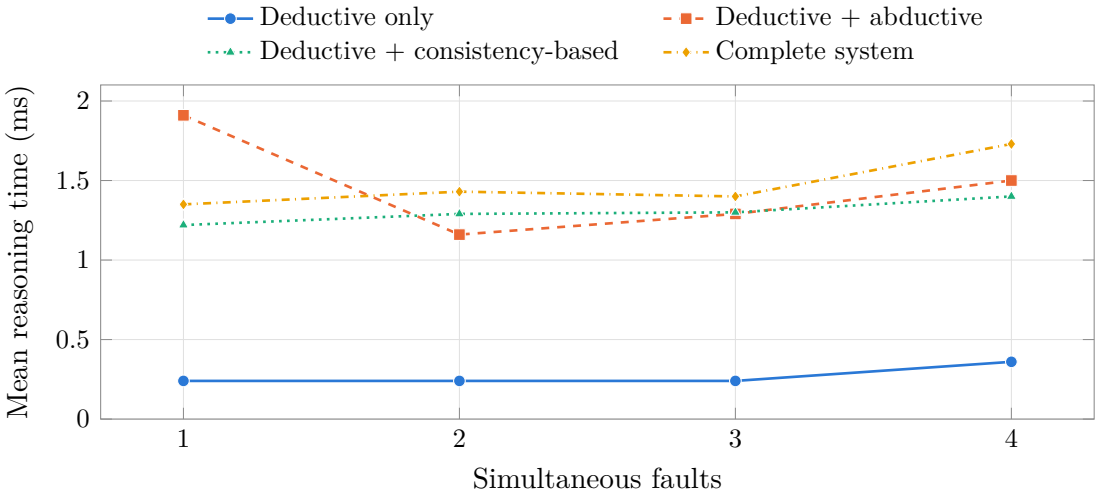


Figure 4.5: Mean reasoning time per case against the number of simultaneous faults. Times are wall-clock and include measurement noise.

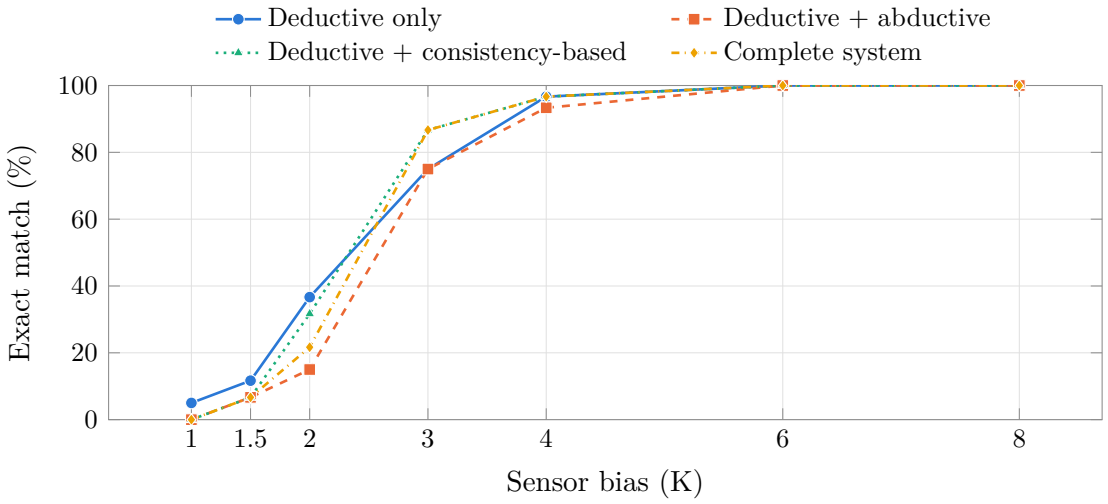


Figure 4.6: Exact-match accuracy against the size of an injected temperature-sensor bias.

Table 4.17: Sensitivity to the size of a sensor bias, a source of contradictory evidence: exact match by reasoning mode, with coverage, macro F1 and mean time in milliseconds of the complete system. Each level has 60 synthetic cases. Ded.: deductive only; +Abd.: deductive and abductive; +Cons.: deductive and consistency-based; Cov.: coverage.

Sensor bias	Ded.	+Abd.	+Cons.	Complete	Cov.	Macro F1	ms
1 K	5.0%	0.0%	0.0%	0.0%	0.0%	0.000	0.96
1.5 K	11.7%	6.7%	6.7%	6.7%	6.7%	0.121	1.06
2 K	36.7%	15.0%	31.7%	21.7%	21.7%	0.317	1.05
3 K	75.0%	75.0%	86.7%	86.7%	86.7%	0.929	1.11
4 K	96.7%	93.3%	96.7%	96.7%	96.7%	0.983	1.13
6 K	100.0%	100.0%	100.0%	100.0%	100.0%	1.000	1.18
8 K	100.0%	100.0%	100.0%	100.0%	100.0%	1.000	1.63

consistency-based configurations do best, because a single sensor fault is then the only minimal diagnosis that removes every conflict. At 2 K the graded modes are deliberately more cautious than deduction, because they require one noise standard deviation of margin past the threshold.

4.7.5 Threshold changes

Each threshold was perturbed on its own on the held-out split (Table 4.18). Eleven of the 23 thresholds change nothing at any level, because backup rules or other symptoms cover them, or because no case lies near them. At ±1 K or ±10%, all but three change the complete system's accuracy by less than 3 percentage points.

Table 4.18: Threshold sensitivity on the held-out split: change in exact-match accuracy of the complete system, in percentage points, when one threshold is perturbed. Temperature thresholds move by −2, −1, +1 and +2 K; other thresholds are scaled by 0.8, 0.9, 1.1 and 1.2. The last column is the worst change for deduction alone. A dot means no change.

Threshold	Base	−2 / 0.8	−1 / 0.9	+1 / 1.1	+2 / 1.2	Ded. worst
`t_inlet_rec_max`	27.0 °C	·	·	·	·	·
`t_inlet_rec_min`	18.0 °C	·	·	·	·	·
`t_supply_band`	2.0 K	·	·	·	·	-7.5
`row_hot_fraction`	0.5	·	·	·	·	·
`valve_open_min`	50.0 %	·	·	·	·	·
`chw_flow_ratio_min`	0.3	-0.8	-0.8	·	·	·
`strat_max`	3.0 K	-77.3	-2.7	·	-0.8	-90.2
`fan_ratio_min`	0.5	·	·	·	·	·
`strat_out_max`	3.5 K	-50.2	-1.2	-4.3	-8.2	-76.5
`filter_loading_max`	1.7	0.8	·	-0.8	-2.0	-1.2
`server_fan_min`	1000.0 rpm	·	·	·	·	·
`cpu_hot_min`	85.0 °C	·	·	·	·	·
`branch_loading_max`	0.8	0.8	0.8	-3.9	-9.0	-9.0
`cell_r_ratio_max`	1.3	-30.2	0.8	-0.4	-0.8	-43.9
`ups_eff_bypass_min`	0.98	-84.3	-83.9	·	·	-1.2
`neighbour_dev_max`	2.0 K	-85.1	-0.4	·	·	-87.8
`balance_resid_max`	0.2	·	·	·	·	·
`sample_age_max`	60.0 s	·	·	·	·	·
`t_supply_hot_limit`	22.0 °C	-12.5	·	·	·	·
`fan_ratio_reduced`	0.85	·	·	·	-45.5	-7.1

Threshold	Base	−2 / 0.8	−1 / 0.9	+1 / 1.1	+2 / 1.2	Ded. worst
`t_supply_cold_limit`	18.0 °C	·	·	·	-11.4	·
`power_resid_max`	0.1	-0.4	·	0.4	0.4	·
`sensor_noise_floor`	0.15 K	·	·	·	·	·

The large drops share one cause: each occurs when the perturbed threshold moves inside the scatter of normal operation. Moving the inlet stratification limit to 1 K, inside the 1 to 2.2 K of sealed racks, costs 77 points. Scaling the reduced-fan-power limit to 1.02, inside the ±5% scatter of a healthy fan, costs 46 points. This is evidence that each threshold sits in a margin between the normal envelope and the fault envelope, and the table measures how wide each margin is.

One result is a warning about consistency checking. Scaling the UPS-efficiency threshold to 0.88 puts every healthy double-conversion UPS, which runs near 0.955, above it. Deduction loses only 1.2 points, because its UPS rule relies on the mode flag. The complete system loses 84 points, because the mis-set relation produces a conflict in every case and the consistency module then rejects every correct explanation. A redundancy relation acts as a hard constraint, so a wrong relation threshold corrupts every diagnosis. Relation thresholds should therefore be derived from sensor uncertainty (Equation (E39–E40)) and validated on normal data before deployment.

4.7.6 Reasoning parameters

The threshold study does not cover the five reasoning parameters set by judgement (Table 3.5). Each was changed on its own, and the three model-based configurations were run on both splits and on four stress sets from this section: four times the nominal noise, 20% of readings missing, four simultaneous faults, and a 2 K sensor bias, which lies in the transition between missed and detected biases. Table 4.19 gives the results. Deduction uses none of these parameters and is the reference. The study is generated by `tests/param_sensitivity.py`.

- **Causal weights hardly matter.** Setting every weight to 1, or multiplying each by a random factor between 0.5 and 1.5 in five independent draws, changes at most one case on any set. Parsimony, not the weights, decides most explanations, and the weights only rank explanations of the same size.
- **The deductive bonus hardly matters up to 0.2.** At 0.5 it gains two held-out cases and six 2 K-bias cases, because abduction then defers to the rules, which detect small biases better (Section 4.7). It leaves the calibration split unchanged.
- **The evidence cut-off is a real trade-off.** A looser cut-off, $k = 0.5$, catches four to nine more 2 K biases but loses seven to nine cases under heavy noise. A stricter one, $k = 2$, gains three to six noisy cases and loses two to five small biases. The default $k = 1$ lies between them: on the held-out split it is at most two cases behind the stricter cut-off and three or four ahead of the looser one.
- **The search bound must cover the largest fault combination.** A bound of 2 or 3 loses every four-fault case of the abductive configurations, and 12 of the consistency-based one. A bound of 5 changes nothing, except that consistency-based search then accepts larger, wrong diagnoses in four of the four-fault cases.

Table 4.19: Sensitivity to the reasoning parameters set by judgement. Each parameter is changed alone. The first two rows give exactly matched cases at the defaults; the other rows give the change in exactly matched cases, for deductive + abductive / deductive + consistency-based / complete system. For the perturbed weights, the range over five random draws is given in brackets. Defaults: bonus 0.1, cut-off $k = 1$, search bound 4, weights from `kb/causal.pl`, noise of unredundant features tied to σ_T. Raw counts are in `docs/15_parameter_sensitivity.md`.

Parameter	Value	Calibration	Held-out	Noise ×4	20% missing	Four faults	2 K bias
Deduction only		230	232	25	36	22	22
Model-based	default	238/234/238	241/238/239	44/31/42	49/48/49	29/34/29	9/19/13
Deductive bonus	0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
	0.05	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
	0.2	0/0/0	+1/0/+1	+1/0/+1	0/0/0	0/0/0	0/0/0
	0.5	0/0/0	+2/0/+2	+1/0/+1	0/0/0	0/0/0	+6/0/+6
Cut-off k	0.5	−3/−1/−2	−3/−4/−4	−7/−7/−9	0/0/0	0/0/0	+6/+4/+9
	1.5	0/+1/0	+1/−1/+1	+2/+4/+4	0/0/0	0/0/0	−2/−2/−3
	2.0	+1/+1/+1	+1/+2/+2	+3/+6/+5	0/0/0	0/0/0	−2/−4/−5
Search bound	2	0/0/0	0/+1/0	+1/+1/0	0/0/0	−29/−12/−29	0/0/0
	3	0/0/0	0/0/0	+1/0/0	0/0/0	−29/−12/−29	0/0/0
	5	0/0/0	0/0/0	0/0/0	0/0/0	0/−4/0	0/0/0
Causal weights	all 1	0/0/0	0/0/0	0/0/0	−1/0/−1	0/0/0	0/0/0
	×U[.5,1.5]	−1/0/−1	0/0/0	0/0/0	[0,+1]/0/[0,+1]	0/0/0	0/0/0
Noise scaling	fixed	+1/+1/+1	0/0/0	−1/−3/−4	0/0/0	0/0/0	0/0/0

- **Tying the noise of unredundant features to the temperature sensors helps under noise.** With fixed nominal noise instead, the model-based configurations lose one to four cases at four times the noise, and gain one calibration case.

The defaults were set before this study and were not changed after it. The held-out split must not be used to tune them, and on the calibration split no variant except the search bound and the loosest cut-off moves any configuration by more than one case, so the calibration data give no reason to change them either. The results support the claim that the conclusions of this chapter do not depend on the exact values chosen by judgement, except for the evidence cut-off, which sets the balance between robustness to noise and sensitivity to small faults.

4.7.7 Summary

Table 4.20 summarises the findings of the sensitivity and ablation analysis.

4.8 Robustness to Model Mismatch

Every result so far was obtained on cases generated by the same engineering model that the knowledge base encodes. This section asks how much the diagnosis depends on that model being exactly right.

Method. The 255 held-out cases were regenerated with the same seed, and so with the same faults, severities and random draws, but with deliberately changed simulator physics. Table 4.21 lists the two profiles. They change the coil heat transfer, server power and airflow, CPU thermal behaviour, the share of airflow shortfall drawn from the hot aisle,

Table 4.20: Summary of the sensitivity analysis.

Question	Finding
Threshold changes	Robust within ±1 K or ±10%. Large losses only when a threshold enters normal scatter. Consistency checking is sensitive to wrong relation thresholds.
Missing observations	The model-based configurations lead deduction by about 7 points at 5% missing, widening to about 22 points at 20% and beyond.
Contradictory evidence	Sensor biases of 3 K and more are reliably found; consistency checking is best at the detection limit.
Noisy data	Abduction with noise-adaptive evidence is the most robust configuration.
Multiple faults	Precision stays near 1; recall falls with the number of faults because interacting faults share symptoms.
Reasoning parameters	Causal weights and the deductive bonus hardly matter. The evidence cut-off trades robustness to noise against sensitivity to small faults. The search bound must cover the largest fault combination.
Reasoning time	A few milliseconds per case in every sweep.
Explanation quality	Every diagnosis in every sweep comes with a complete explanation, as it must by construction. Explanation fidelity was measured on the held-out split only (Section 4.6.5), not at each sweep level.

natural recirculation, room temperature, fan-control margin and sensor noise. Equipment ratings that a real knowledge base would take from nameplates, such as the fan's rated power and the valve characteristic, were left unchanged. The knowledge base was not modified. Because each case keeps its faults and draws, the results are compared with the nominal split case by case.

Mild mismatch. With the physics changed by about 10 to 25%, the configurations with abduction lose about four points of exact-match accuracy, keeping 90 to 91%, while deduction alone falls from 91.0% to 80.8% (Table 4.22). Every configuration loses significantly, but graded, noise-adaptive evidence makes the model-based configurations markedly less sensitive to model error than crisp rules.

Strong mismatch. With changes of about 25 to 50%, every configuration collapses, to between 26% and 40%. Even 17 of the 30 normal cases are misdiagnosed (Table 4.23). Inspection of the failures shows that the reasoning is intact and that three envelope thresholds are to blame. With less airflow per server and more natural recirculation, healthy racks stratify by 3 to 4 K, above the 3 K and 3.5 K stratification thresholds. With hotter-running CPUs, healthy servers exceed the 85 °C CPU threshold. The system then sees recirculation where there is none, or symptoms that no hypothesis explains. The thresholds encode the normal envelope of the nominal model, and the strong profile moves that envelope.

Re-baselining from normal operating data. A real site has plenty of data from normal operation, even when it has no fault data. To test whether that is enough, normal-

Table 4.21: Simulator physics in the two model-mismatch profiles.

Parameter	Nominal	Mild	Strong
Coil air-side conductance (kW/K)	15	12	10
Coil water-side conductance at design flow (kW/K)	10	12	14
Server idle and full-load power (kW)	0.40, 0.90	0.35, 0.95	0.30, 1.05
Server airflow at nominal fan speed (kg/s)	0.055	0.050	0.045
CPU share of server power	0.55	0.50	0.60
CPU thermal resistance (K/kW)	90	90	110
Share of airflow shortfall from the hot aisle	0.4	0.5	0.6
Natural recirculation at the top of a sealed rack	0.10	0.12	0.15
Fan-control airflow margin	1.10	1.15	1.25
Room air temperature (°C)	25	26	27
Sensor noise, multiple of nominal	1.0	1.25	1.5

Table 4.22: Model mismatch: exact match of each reasoning mode when the held-out cases are regenerated with changed simulator physics and diagnosed with the unchanged knowledge base. The last pair of columns re-baselines three envelope thresholds from normal operating data only. p: exact McNemar test against the nominal split, paired case by case. Confidence intervals are in `docs/12_model_mismatch.md`.

Mode	Nominal	Mild	p	Strong	p	Re-baselined	p
Deductive only	91.0%	80.8%	$<$ **0.001**	26.3%	$<$ **0.001**	75.3%	$<$ **0.001**
Deductive + abductive	94.5%	91.0%	**0.004**	40.0%	$<$ **0.001**	83.1%	$<$ **0.001**
Deductive + consistency-based	93.3%	86.3%	$<$ **0.001**	33.7%	$<$ **0.001**	83.5%	$<$ **0.001**
Complete system	93.7%	90.2%	**0.004**	39.6%	$<$ **0.001**	82.4%	$<$ **0.001**

Table 4.23: Model mismatch by test category: exact matches of the complete system.

Category	Cases	Nominal	Mild	Strong	Re-baselined
Normal	30	30	30	13	30
Single faults	120	120	117	57	111
Instrumentation	30	30	29	6	29
Multiple faults	30	29	28	14	17
Incomplete	15	14	13	5	13
High noise	15	7	4	1	3
Borderline	15	9	9	5	7

only cases were generated under the strong physics with a different seed (20261020). Each of the three envelope thresholds was shifted by how far the largest normal value moved, which keeps its original margin above normal operation. No fault labels were used. The shifts were +2.3 K and +2.2 K for the inlet and outlet stratification thresholds and +26.7 K for the CPU threshold. Re-baselining restores the complete system to 82.4% and the other model-based configurations to about 83%, against 75.3% for deduction. Normal and instrumentation cases are again almost all correct. Multiple faults, at 17 of 30, and high-noise cases recover only partly. The re-baselined CPU threshold also lies above the throttling limit, so CPU temperature no longer helps to detect a failed server fan.

Implication. The diagnostic reasoning transfers across models, but fixed absolute thresholds do not. A deployment would therefore need its envelope thresholds calibrated against the site's own normal operation before use. Defining those thresholds relative to a learned baseline, rather than as fixed values, is the most direct improvement this experiment suggests (Section 6.7).

4.9 Comparison with a Data-Driven Baseline

Fault diagnosis in data centres has mostly been data-driven (Chapter 2), and this project offers explainability instead. That trade-off should be shown, not asserted. Standard classifiers were therefore trained on the raw readings, with no engineering knowledge, and tested on the same cases as the expert system. The framework forbids replacing the symbolic system with machine learning, not comparing against it, so the classifiers are a baseline only and play no part in the expert system.

Method. Each case becomes one vector of every reading, with a missing reading left missing. The labels are the fault labels seen in training, such as chw_supply_loss(crah_1), learned as one binary output each. A case with no predicted fault is predicted normal. A random forest of 300 trees and a single decision tree were trained with the standard settings of scikit-learn (Pedregosa et al., 2011), without tuning, in two regimes: on the calibration split, the 255 cases against which the knowledge base was checked, and on 2550 further simulated cases, far more labelled fault data than a real site would have. The second regime favours the classifiers in two ways: they see labelled examples of nearly every fault at every location, and they are trained and tested on the same simulator. The script is `tests/baseline.py`.

Table 4.24 gives the results. Four findings stand out.

- **With the data the knowledge base was checked against, the classifiers fail.** Trained on the 255 calibration cases, they reach 52 to 58% exact match. A fault label names its location, so there are dozens of labels with a few examples each, and 23 held-out cases contain a label never seen in training. No instrumentation case and only 2 of the 30 multiple-fault cases are diagnosed correctly.

- **With ten times as much labelled data, they come closer but stay significantly behind.** Trained on 2550 cases, the decision tree reaches 89.4% on the held-out split and 87.1% over five seeds, against 94.5% and 93.6% for deductive + abductive reasoning. The random forest, at 82.0%, does worse than the single tree with these untuned settings. The classifiers do better than the expert system

Table 4.24: Exact match of the expert system and of data-driven classifiers trained on the raw readings, on the same test cases (255 held-out, 1275 over five seeds, 255 per mismatch profile). The classifiers are a baseline for comparison only. Bold: significantly different from deductive + abductive reasoning on the same cases (exact McNemar, $p < 0.05$). Details in `docs/17_baseline_comparison.md`.

System	Training data	Held-out	Five seeds	Mild mismatch	Strong mismatch
Deductive + abductive	none	94.5	93.6	91.0	40.0
Complete system	none	93.7	92.9	90.2	39.6
Random forest	calibration (255)	**53.3**	**52.1**	**34.9**	**23.9**
Decision tree	calibration (255)	**53.7**	**57.9**	**38.0**	**29.0**
Random forest	2550 extra cases	**82.0**	**82.0**	**48.6**	**30.6**
Decision tree	2550 extra cases	**89.4**	**87.1**	**50.2**	38.0

on borderline and high-noise cases: the forest gets 12 and 13 of 15 right against 9 and 9. Boundaries learned from labelled data fit the simulator's noise better than thresholds set by engineering judgement, and that is the price of sourced, explainable thresholds.

- **Under model mismatch, the classifiers collapse first.** With the physics changed by 10 to 25%, the classifiers trained on 2550 cases fall to about 50%, while deductive + abductive reasoning keeps 91.0%. The classifiers have learned the exact envelope of the nominal simulator, not the physics behind it. Under strong mismatch every system fails, and the best classifier, at 38.0%, is not significantly different from the expert system, at 40.0%.
- **The classifiers explain nothing.** They give no proof, no competing hypotheses and no maintenance plan, they need labelled examples of every fault at every location, and they cannot diagnose a fault at a location absent from their training data.

On these data, then, the knowledge-based system is more accurate than standard classifiers, needs no labelled fault data, and transfers far better to a changed model, while also explaining every diagnosis. The classifiers' advantage on borderline and noisy cases is the strongest argument for learning thresholds from data within the symbolic system, as Section 6.7 proposes, rather than for replacing it.

Chapter 5

Expert-System Software Development, Documentation and Reproducibility

Source code alone does not show how a system was built or whether its results can be trusted. This chapter documents the development lifecycle (Section 5.1), the implementation environment (Section 5.2), the software structure and its design decisions (Section 5.3), the main problems found and fixed during debugging (Section 5.4), how the software was verified and validated (Section 5.5), and exactly how to run it and reproduce every result (Sections 5.6 and 5.7).

5.1 Development Process

Development followed the lifecycle that the project framework prescribes:

Requirements → Knowledge acquisition → Predicate design → Rule design → Implementation → Debugging → Testing → Validation.

In practice the later stages fed back into the earlier ones. Table 5.1 summarises what each stage produced. The design documents in the `docs/` folder record each stage as it was carried out, and much of Chapters 2 to 4 is drawn from them.

The system grew incrementally. A first prototype covered three faults and normal operation, end to end from readings to explanation and maintenance plan. The simulator was built next, so that every later rule could be tested against physically consistent data before it was accepted. Rules for the remaining faults followed. The abductive and consistency-based modules were added last, once the deductive layer was stable, and were evaluated against it.

5.2 Implementation Environment

Table 5.2 lists the software used. The expert system needs only SWI-Prolog. Python, with its standard library alone, generates the synthetic data and formats results. LaTeX is needed only to build this report.

Table 5.1: Development lifecycle and its products.

Stage	Work done	Product
Requirements	System boundary, fault universe, observation vector, test categories	`docs/01` to `docs/03`
Knowledge acquisition	Standards, literature and engineering assumptions, each with provenance	`docs/04`, `kb/sources.pl`
Predicate design	Representation of facts, observations, thresholds and calculations	`kb/components.pl`, `kb/calculations.pl`
Rule design	Deductive rules, causal signatures, redundancy relations and fault models	`kb/rules.pl`, `kb/causal.pl`, `kb/arr.pl`
Implementation	Meta-interpreter, abductive and consistency-based engines, explanation facility	`engine/`
Testing	Physics simulator, calibration and held-out splits, sensitivity sweeps	`simulator/`, `tests/`
Validation	Generalisation to the held-out split; explanation inspection; sensitivity analysis	Chapter 4

Table 5.2: Implementation environment.

Component	Version	Role
Operating system	Windows 11 Pro, build 26200	Development and testing
SWI-Prolog	10.0.2, 64-bit	All diagnostic reasoning; no extra packages
Python	3.13.1, standard library only	Simulator, sensitivity driver, reproduction check
scikit-learn	1.9.1, with NumPy 2.5.4; optional	Data-driven baseline only (Section 4.9)
MiKTeX	25.12, pdfTeX 4.23	Building the report
Biber	2.21	Bibliography processing
Strawberry Perl	5.42.3	Needed by the MiKTeX tool chain

The Prolog code uses only standard built-ins and the SWI-Prolog libraries that load automatically, such as `lists`, `apply`, `yall` and `solution_sequences`. It should run unchanged on Linux and macOS, although it has been tested only on Windows.

5.3 Software Structure

The source modules are listed in Table 3.1 and their main predicates in Table 3.3. The framework asks how several design questions were answered. Each is summarised below, with a pointer to the fuller treatment.

Why each major predicate exists. Each predicate implements one component of the architecture in Figure 3.1. Calculation predicates implement single equations and carry their equation tags. `prove/2` exists so that rules can be stored as data and still yield proof trees. `case_symptoms/2` exists so that the abductive and consistency-based modules see identical, cached evidence. `diagnose_mode/3` exists so that the four configurations can be compared on the same data.

How facts are represented. Structure, ratings, thresholds and sources are ground facts. Each test case is a separate set of `obs/4` facts, so the same knowledge base diagnoses every case unchanged (Section 3.2).

How rules were constructed. Every rule began as a row of the fault taxonomy or a physical relation from Section 2.7. It was written as a Horn clause over named thresholds, given an identifier and a source, and accepted only after it had been run against the calibration split (Section 3.3).

How conflicts are handled. Conflicting conclusions are resolved at four levels:

- When several rules prove the same fault, it is reported once, with the first proof.
- Rules that could fire on a confounder are guarded by negation as failure. The recirculation rule r22, for example, is blocked if the top inlet sensor can be shown to be suspect.
- Competing hypotheses are resolved by abduction, through parsimony and confidence ranking (Section 3.6).
- Explanations that contradict a redundancy relation or a fault model are rejected by the consistency check (Section 3.7).

When two diagnoses call for maintenance plans that depend on each other, the plan interactions order the work (Section 3.9).

How explanations are generated. The explanation facility reads the proof trees of the deductive rules and the evidence records of the abductive and consistency-based modules. It never reconstructs reasoning after the fact (Section 3.8).

How maintenance actions are encoded. Each fault class has one `maintenance/5` fact, with priority, ordered actions, rationale and source, instantiated for the diagnosed entity (Section 3.9).

Generated documentation. Four kinds of documentation are generated from the code and data rather than written by hand, so they cannot drift from the implementation: the rule, threshold and provenance tables (`tests/export_latex.pl`), the evaluation tables (`tests/metrics_latex.pl`), the sensitivity tables and figures (`tests/sensitivity.py`), and the explanation listing, which is unedited program output.

5.4 Debugging

The most instructive problems found during development fall into three groups.

5.4.1 Knowledge errors found by the simulator

Because the simulator injects faults into the physics rather than the readings, it revealed errors in the knowledge itself that hand-made test cases had not. The three most significant are described in Section 2.8: a wrong signature for chilled-water loss, a power design that made overload rules fire on healthy racks, and a stratification rule broken by server fan control. Three further problems were found the same way:

- **Partial fan failures were missed.** A slipping belt that leaves the fan drawing 55% of its expected power passed the 0.5 threshold for a failed fan, so the fan-failure hypothesis was rejected and the hot row went unexplained. Healthy fans stay within 5% of the prediction, so a second threshold at 0.85 was added for a partial failure (rules r38, r48 and r49).
- **A real airflow loss was blamed on a sensor.** A partial fan failure warms the inlets relative to the supply air, which looked like a biased supply-air sensor. The sensor-fault rule now accepts the hypothesis only if assuming it closes the coil energy balance (Equation (E41b)).
- **Noise was estimated from too few sensors.** At first the noise of every feature was scaled from the spread of six temperature readings. That estimate is itself noisy, and it understated the noise of the power meters. The electrical noise is now estimated separately, from the redundancy of the rack power balances.

5.4.2 Robustness problems found by the sensitivity analysis

The threshold sweep perturbed the sensor noise floor, which is stored in kelvin, by -1 and $-2\,\mathrm{K}$. That made the floor negative, so the estimated noise could be zero or negative. Every symptom then became clearly present, and the abductive and consistency-based searches grew without bound. Two fixes followed. The sweep now scales the noise floor instead of shifting it, and the evidence grading clamps the noise to a small positive value, so that no input can make the reasoning diverge.

5.4.3 Implementation problems

- **Library name clashes.** SWI-Prolog autoloads library predicates on first use. Names such as `run_tests/0` and `explain/1` would have clashed with library predicates, so the project's own predicates are called `run_suite/0` and `why/1`.
- **Facts split across files.** The test cases are spread over several data files. Without `multifile` declarations, loading the second file would have silently replaced the facts of the first.
- **Generated LaTeX.** In `format/2`, a tilde introduces a directive, so the non-breaking space `~` in a generated caption must be written as a doubled tilde. The first version of the evaluation exporter failed on this.
- **Reproducibility of the data.** The reproduction check of Section 5.7 found that a later change to the simulator had swapped the order of two random draws for a sensor bias. The simulator still ran, but it no longer regenerated the published datasets. The original order was restored, so the datasets now regenerate exactly from their seeds, and the sensitivity sweeps were rerun with the corrected code.

5.5 Verification and Validation

Verification checks that the software does what its design says. It was carried out at four levels:

- four hand-written smoke cases with known answers, rerun after every change;
- the calibration split, against which every new rule was run before it was accepted;
- a static check of every figure file, followed by visual inspection of every compiled figure and table;
- the reproduction check of Section 5.7, which regenerates the datasets and recomputes every reported count.

Validation checks that the design solves the right problem. It is discussed in Section 4.4: equations were checked against hand calculations, rules against simulated physics, generalisation against the held-out split, and explanations against the requirements of the framework. Its main limitation is that the knowledge base and the simulator share one engineering model, so validation on measured data remains necessary.

5.6 Execution Instructions

All commands are run from the project folder, `dc-expert-system`.

Load the system and ask questions.

```
swipl main.pl
?- run_suite.                       % deductive test suite, all splits
?- run_suite(single_fault).         % one category, listing each failure
?- why(h188).                       % full explanation of one case
?- why_deductive(f01).              % deductive proof only
?- diagnose_mode(full, h188, D).    % modes: deductive, ded_abd, ded_cbd, full
```

In Visual Studio Code with the Prolog Runner extension, opening `run.pl` and pressing Run executes the test suite and explains the four smoke cases.

Regenerate the datasets. This step is optional, because the datasets are included.

```
python simulator/dc_sim.py
python simulator/dc_sim.py --seed 777 --prefix h --split holdout --out data/
    holdout_cases.pl --csv data/holdout_cases.csv --truth data/holdout_truth.json
```

Regenerate the results and the report tables.

```
swipl -g "evaluation_all('docs/08_evaluation_results.md', 'report/tables')" -t halt
    main.pl tests/metrics.pl tests/metrics_latex.pl
swipl -g "export_latex_tables('report/tables')" -t halt main.pl tests/export_latex.pl
python tests/sensitivity.py                 # full sensitivity analysis, about 12
    minutes
python tests/sensitivity.py --tables-only   # tables only, from the saved results
swipl -g "fidelity_report('docs/13_explanation_fidelity.md', 'report/tables')" -t
    halt tests/fidelity.pl
python tests/equation_usage.py              # equation-usage table, from the code's
    Eq. references
python tests/param_sensitivity.py           # reasoning-parameter sweep, about a
    minute
python tests/seeds.py                       # four extra held-out seeds, about a
    minute
python tests/baseline.py                    # data-driven baseline; needs scikit-
    learn
```

Build the report. From the `report` folder:

```
pdflatex main.tex
biber main
pdflatex main.tex
pdflatex main.tex
```

5.7 Reproducibility

The framework requires that another researcher can reproduce the system. Five elements make that possible:

- **Software.** The versions are listed in Table 5.2. No library outside SWI-Prolog and the Python standard library is required, except scikit-learn for the optional baseline comparison.

- **Data.** The datasets, with their data dictionary (Table 2.6), units, ranges, generation procedure (Section 4.1) and assumptions (Table 4.2), are included. They can also be regenerated from their seeds.
- **Execution.** Exact commands are given in Section 5.6.
- **Expected output.** Table 5.3 gives the expected results.
- **Source code.** The complete Prolog source of the expert system, the simulator and the test code is included.

One-command reproduction check. The script `tests/reproduce.py` checks reproducibility automatically. It regenerates both datasets from their seeds into a temporary folder and checks that they are identical to the included files, using recorded SHA-256 checksums. It then runs all four reasoning modes on every split, including the model-mismatch splits of Section 4.8 and the four extra held-out seeds of Section 4.6.2, and compares the exact-match counts with the recorded values. It also runs the explanation-fidelity test of Section 4.6.5 and checks its counts. With `-tables` it also regenerates every generated table in the report, and runs `tests/mismatch.py`, which regenerates the mismatch splits and re-baselines the thresholds, and `tests/param_sensitivity.py`, which sweeps the reasoning parameters.

```
python tests/reproduce.py            # 35 checks; exit code 0 if all pass
python tests/reproduce.py --tables   # also regenerate the report tables
```

On the development machine all 35 checks pass in about five minutes.

Table 5.3: Expected exact-match counts, checked by `tests/reproduce.py`.

Split	Deductive	Ded. + abd.	Ded. + cons.	Complete
Smoke (4 cases)	4	4	4	4
Calibration (255)	230	238	234	238
Held-out (255)	232	241	238	239

What is not exactly reproducible. Reasoning times are wall-clock measurements and vary with the machine and its load, so only their order of magnitude, about a millisecond per case, should be expected to reproduce. Everything else, including every diagnosis, count and table, is deterministic.

Version control. The project folder is a git repository. The submitted version should be committed and tagged, so that the exact state of the code, data and report behind these results can always be recovered.

Chapter 6

Assessment Framework and Research Deliverables

This chapter maps the project against the assessment criteria and minimum requirements of the project framework (Sections 6.1 and 6.2), states its research contributions (Section 6.3), indexes the required tables, figures and deliverables (Sections 6.4 and 6.5), and sets out its limitations and the improvements they suggest (Sections 6.6 and 6.7). Where a requirement is not yet fully met, this chapter says so.

6.1 Assessment Criteria

The framework lists twelve criteria. Table 6.1 gives, for each, where the evidence is and what it shows.

Table 6.1: Evidence against the assessment criteria.

Criterion	Evidence	What it shows
Engineering correctness	Sections 2.2, 2.7, 4.4	Physical model of the row with 60 tagged equations, 44 of them used by the code (Table 2.8); simulator checked against hand calculations; three knowledge errors found and corrected against the physics
Depth of KRR treatment	Sections 2.4 to 2.6	Horn clauses, negation as failure and the closed-world assumption; formal deductive, abductive and consistency-based diagnosis with fault models
Quality of knowledge representation	Section 2.5; Tables 2.7, 3.2	Seven layers; rules stored as data with identifiers and provenance; graded symptoms that treat missing data as unknown
Correctness of Prolog rules	Table 3.4; Section 4.4	Every rule run against the calibration split before acceptance; all single-label cases of the held-out split diagnosed correctly, and 3 to 6 of 900 missed over five seeds

Criterion	Evidence	What it shows
Diagnostic accuracy	Tables 4.5, 4.11	91.0% to 94.5% exact match on unseen data across the four configurations; macro F1 up to 0.958; over five held-out seeds, 90.8% to 93.6% with a standard deviation of 1.2 to 1.6 points (Table 4.9)
Multiple-fault capability	Tables 4.8, 4.16	29 or 30 of 30 multiple-fault cases on the held-out split, a single sample of 30; precision near 1 for up to four simultaneous faults
Robustness to imperfect evidence	Tables 4.14, 4.15, 4.17, 4.22	Graceful degradation under noise and missing data; sensor biases of 3 K and more detected; robust to mild model error, and to strong model error after re-baselining
Explanation quality	Section 3.8; Listing 3.1	ExplainabilityRate of 100%, by construction; explanation fidelity of 96 to 100% (Table 4.13), a test that can fail; every element the framework requires shown in a real trace; not yet assessed by human users
Maintenance-plan validity	Table 3.6; Section 3.9	A sourced plan for every diagnosis; plan interactions ordered; PlanValidity 85% to 94% on the held-out split, which in effect measures label precision, not the plans' actions
Computational tractability	Tables 4.5, 4.16	About 1 ms per case, and a few milliseconds with four simultaneous faults, as orders of magnitude on one machine
Documentation quality	Chapters 3 and 5	Design documents for every stage; rule, threshold, provenance, maintenance and result tables generated from the code
Reproducibility	Section 5.7	Datasets regenerate exactly from their seeds; a 35-check script reproduces every reported count

6.2 Minimum Technical Requirements

Table 6.2 checks the minimum technical requirements of the framework. All of them are met.

6.3 Research Contributions

The individual techniques used in this project are established in the literature. Its contributions lie in how they are combined and applied to data centre cooling and power infrastructure, and in the evidence it provides about that combination.

1. **An explainable hybrid diagnostic system for data centre infrastructure.** The system combines deductive rules, parsimonious-covering abduction and

Table 6.2: Minimum technical requirements.

Requirement	Status	Evidence
Expert system in Prolog	Met	All reasoning in SWI-Prolog; Python only generates data and formats results
Explicit mathematical model	Met	Section 2.7
At least 40 equations	Met	60 tagged engineering equations, 44 of them computed by the reasoning or the simulator (Table 2.8), plus the logical and evaluation formulas
Engineering-justified fault classes	Met	Table 2.4 and Section 2.3
Deductive, abductive and consistency-based reasoning	Met	Sections 3.5 to 3.7
Explanation	Met	Section 3.8
Maintenance recommendation	Met	Section 3.9; Table 3.6
Documented dataset	Met	Section 4.1
Normal, single and multiple faults	Met	Table 4.4
Noisy and incomplete evidence	Met	Table 4.4; Section 4.7
Instrumentation faults	Met	Sensor bias and telemetry dropout classes
Quantitative evaluation	Met	Section 4.6
Software documentation	Met	Chapter 5
Reproducibility	Met	Section 5.7
TikZ/LaTeX diagrams	Met	Eight TikZ diagrams and five pgfplots charts
Complete Prolog source	Met	Included in full
70 to 90 scholarly sources	Met	77 entries, including standards and manufacturer data; all checked against online records and verified by the author
APA 7th edition via EndNote	Met	APA 7th edition style applied; all 77 entries managed in an EndNote library, `dc-expert-system.enlx`, linked to `references.bib` by citation key

consistency-based diagnosis with fault models, as "abduction proposes, consistency disposes". It diagnoses eight equipment faults and two instrumentation faults across the cooling and power path, and explains every conclusion.

2. **Noise-adaptive graded evidence from in-case redundancy.** Symptom evidence is graded against measurement noise estimated from the case itself: from the robust spread of sensors that see the same air, and from the redundancy of the rack power balances. At four times the nominal noise, this lifts exact-match accuracy from 42% to 73% without any retuning.

3. **Separating sensor faults from equipment faults.** Treating sensors as components with fault models, and accepting a sensor-fault hypothesis only when assuming it restores an energy balance (Equation (E41b)), separates sensor faults from equipment faults that produce similar symptoms. Every single sensor fault in the held-out split is diagnosed correctly.

4. **A quantified comparison of the three reasoning modes.** The four configurations required by the framework were compared on identical data and under controlled noise, missing data, multiple faults, sensor bias and threshold changes. On one split, no difference survives correction for the six comparisons. Pooled over five held-out seeds, four of the six do: deductive + abductive reasoning is significantly the most accurate configuration, and consistency checking costs exact matches under heavy noise. The controlled experiments show where each mode helps and where it costs accuracy, including the fragility of consistency checking to a mis-set redundancy relation. A model-mismatch experiment shows that the reasoning transfers to a changed model, while fixed thresholds must be re-baselined from normal operating data. A comparison with standard classifiers shows that the knowledge-based system is more accurate on these data, needs no labelled faults and transfers better to a changed model.

5. **A physics-based test methodology for knowledge validation.** Injecting faults into a physical model rather than into readings, with separate calibration and held-out splits, exposed errors in the knowledge that hand-made cases did not, and guards the reported results against tuning bias.

6. **Documentation that cannot drift from the code.** Rule, threshold, provenance, maintenance and result tables are generated from the knowledge base and the test runs, and a reproduction script checks every reported count.

6.4 Required Tables and Figures

The framework lists the tables and figures a report should contain. Table 6.3 shows where each one is.

Table 6.3: Index of the required tables and figures.

Required item	Location
Fault taxonomy table	Table 2.4
Observation-variable dictionary	Table 2.6
Engineering-equation dictionary	Section 2.7; nomenclature in Table 2.9

Required item	Location
Predicate and knowledge-representation table	Tables 2.7, 3.2 and 3.3
Diagnostic-rule table	Table 3.4
Rule-provenance table	Tables 3.7 and 3.8
Dataset and test-case table	Tables 4.3 and 4.4
Confusion matrix	Table 4.12
Performance metrics	Tables 4.5, 4.6 and 4.11
Explanation-quality assessment	ExplainabilityRate in Table 4.5; fidelity in Table 4.13; trace in Listing 3.1
Maintenance-action mapping	Table 3.6
Comparison of reasoning modes	Tables 4.5 and 4.8; Section 4.7
Industrial system schematic	Figure 2.1
Expert-system architecture	Figure 3.1
Knowledge-acquisition workflow	Figure 2.3
Fault-propagation structure	Figure 2.2
Diagnostic reasoning workflow	Figure 3.2
Explanation-generation workflow	Figure 3.3
Maintenance-planning workflow	Figure 3.4
Experimental and evaluation workflow	Figure 4.1

6.5 Software Deliverables

Table 6.4 maps the ten deliverables of the framework to the submitted files.

Table 6.4: Deliverables.

Deliverable	Files
Final report	`report/main.pdf`, built from `report/`
Complete Prolog source code	`main.pl`, `run.pl`, `kb/`, `engine/`
Documented knowledge base	`kb/` with comments; generated tables in Chapter 3; `docs/07_rule_table.md`
Dataset and data dictionary	`data/` as Prolog, CSV and JSON; Table 2.6; `docs/06_dataset_and_simulator.md`
Test-case suite	`data/cases.pl`, the calibration and held-out splits, `tests/run_suite.pl`
Evaluation results	`docs/08_evaluation_results.md`, `docs/10_sensitivity_results.md`, `data/sweeps/results.json`
Explanation traces	`why/1` for any case; `report/listings/why_h188.txt`
Corrective-maintenance outputs	Part of every `why/1` trace; Table 3.6
Software-development documentation	Chapter 5; `docs/01` to `docs/10`; `README.md`
Reproducibility instructions	Section 5.7; `tests/reproduce.py`; `README.md`

6.6 Limitations

Data and validation.

- Every test case is synthetic. The knowledge base and the simulator rest on the same engineering model, so agreement between them shows internal consistency, not accuracy on a real data centre. The model-mismatch experiment measures sensitivity to model error, but only for the kinds of error it was given.
- Fixed envelope thresholds encode the normal behaviour of the nominal model. Under strong model mismatch they fail until they are re-baselined from the site's normal operating data.
- The model is steady state. Real faults develop over time, and a chilled-water loss, for example, heats a room over minutes.
- The scope is one row with one cooling unit. Humidity, the chiller plant and the generator are outside the boundary.
- The modelled row has no cooling redundancy. With one unit, a fan or chilled-water failure overheats the whole row, which makes F_1 and F_2 easy to detect. With N+1 units, as real rows normally have, the healthy units would partly mask the failure, and the row-level symptoms the rules rely on would be weaker and more local.

Knowledge.

- 17 of the 24 sources in the knowledge base are engineering assumptions rather than citations, and 36 of the 47 rules rest on one. The sensitivity analysis shows that the results are robust to moderate changes in them, but each still needs a source or a written justification.
- The confidence weights of the causal knowledge were set by engineering judgement and not fitted to data. Confidence is a ranking, not a probability. Replacing the weights by uniform or randomly perturbed ones changes at most one case (Table 4.19).
- One physical interaction is not modelled: a failed server fan inside a recirculating rack reduces the exhaust stratification.

Reasoning.

- Under heavy noise, consistency checking can turn a chance violation of a balance into a reported sensor fault, which lowers sensor-fault precision to 0.656.
- A wrongly set redundancy-relation threshold acts as a hard constraint and can corrupt every diagnosis of the complete system.
- The noise of the cooling-unit instruments is inferred from the temperature sensors, which assumes that they degrade together. The assumption helps under heavy noise, but has not been checked against real instruments.
- Explanations and diagnoses are capped at four simultaneous faults. The cap must be at least the largest fault combination expected: a cap of three loses every four-fault case.

- The evidence cut-off of one standard deviation trades robustness to noise against sensitivity to small faults (Section 4.7.6). A site would have to choose it for its own noise level.

Evaluation and reporting.

- Explanation quality was assessed by criteria and by a fidelity test, not by maintenance engineers using the system. The fidelity test shows that a deductive trace reports only the first proof of a label, and that no explanation cites the readings behind its noise estimate.
- PlanValidity follows from the correctness of the label, so it does not test whether the actions in a plan are right. Their validity rests on their sources and has not been reviewed by a practitioner.
- Reasoning times were measured on one machine.

6.7 Future Improvements

The limitations suggest nine improvements, roughly in order of importance:

1. **Validation on measured data,** ideally logged telemetry from a real facility with recorded faults, to test the knowledge against behaviour outside the shared model.
2. **Time-series reasoning.** Repeated readings with the noise filter of Equation E54 would reduce false findings under noise, and would let the system recognise faults by how they develop.
3. **Thresholds relative to a learned baseline.** The model-mismatch experiment shows that re-baselining three envelope thresholds from normal operating data restores most of the lost accuracy. Making every envelope threshold relative to a baseline learned on site would make this automatic.
4. **Sources for every assumption,** or, where none exists, thresholds estimated from normal operating data with stated confidence.
5. **Graded consistency checking,** in which a relation's violation counts in proportion to its evidence strength, so that one marginal violation cannot veto a well-supported explanation.
6. **More of the engineering model as evidence.** The heat and rack cooling indices (E14–E17), the first-law residual (E37) and phase imbalance (E44) are already part of the model and can be computed from the same readings, but no rule uses them yet (Table 2.8).
7. **A wider scope:** several rows, redundant cooling units, humidity, and the chiller plant. Redundant units need the airflow and heat-transfer share of each unit in the model, and an abductive choice of which unit has failed, with each unit's own fan power and valve readings as the discriminating evidence.
8. **User evaluation of explanations** by data centre maintenance engineers.

9. **Integration with live monitoring,** so that diagnoses and plans are produced as telemetry arrives.

Chapter 7

Conclusion and Final Project Specification

7.1 Summary of the Developed System

This project developed an explainable expert system, implemented in SWI-Prolog, that diagnoses faults in the cooling and power infrastructure of a data centre row and recommends corrective maintenance. Its knowledge base represents the row's components, a 60-equation engineering model, 24 sourced thresholds, 47 diagnostic rules, causal signatures for abduction, 20 analytical redundancy relations with fault models, and a corrective plan for every diagnosis. Three inference mechanisms reason over it: deductive rules executed by a proof-recording meta-interpreter, abduction by parsimonious covering, and consistency-based diagnosis by minimal hitting sets with fault models. They are combined as "abduction proposes, consistency disposes". An explanation facility reports, for every diagnosis, the evidence, the rules, the competing hypotheses and the reasons the weaker ones were rejected. A physics simulator generated the synthetic datasets used to test the system.

7.2 Diagnostic Performance

On 255 held-out cases that played no part in development, the four configurations required by the framework reached 91.0% to 94.5% exact-match accuracy and a macro F1-score of up to 0.958 (Table 4.5). Every configuration diagnosed every normal, single-fault and instrumentation case correctly. Every diagnosis was explained. A fidelity test, which unlike the explainability rate can fail, showed that removing the evidence an explanation cites removed the diagnosis in 96 to 100% of cases. Between 85% and 94% of the maintenance plans addressed a fault that was really present. Reasoning took about a millisecond per case.

Adding abduction raised exact match from 91.0% to 94.5% on the held-out split ($p = 0.022$, or 0.135 after Holm's correction for six comparisons). On that one split no configuration could be shown to differ from another. Over five held-out seeds and 1275 cases they could: deductive + abductive reasoning was significantly the most accurate after the correction, at 93.6%, followed by the complete system at 92.9%, deductive + consistency-based reasoning at 91.9% and deduction alone at 90.8%, with a standard deviation of 1.2 to 1.6 points across seeds. The complete system was the only configuration

to catch the smallest supply-sensor bias in the calibration data, but under heavy noise it sometimes added a sensor fault that was not present. That is the only way in which it lost to deductive + abductive reasoning. The sensitivity analysis showed graceful degradation where deduction alone collapsed. At four times the nominal sensor noise, abduction kept 73% exact match against 42% for deduction. With 20% or more of the readings missing, the model-based configurations led deduction by about 22 percentage points. Precision stayed close to 1 with up to four simultaneous faults. Sensor biases of 3 K and more were reliably detected. When the simulator's physics was changed by 10 to 25%, the abductive configurations kept 90 to 91% exact match while deduction fell to 81%. Changes of 25 to 50% made every configuration fail, because three fixed thresholds encode the normal envelope of the nominal model. Re-baselining them from normal operating data alone restored the model-based configurations to about 83%. Standard classifiers trained on the raw readings reached 52 to 58% with the calibration data and at most 89.4% with ten times as much labelled data, fell to about 50% under mild model mismatch, and explained nothing. They did better than the expert system only on borderline and high-noise cases.

7.3 Answers to the Research Questions

RQ1: representation. Engineering knowledge was represented in seven layers of Prolog facts and Horn clauses, from component structure to maintenance plans (Section 2.5). Storing rules as data, with identifiers and sources, gave every conclusion a traceable provenance and let the report's tables be generated from the knowledge base. Graded symptoms, which treat missing evidence as unknown rather than absent, extended the representation to noisy and incomplete data without giving up its logical reading.

RQ2: comparison of reasoning modes. Deduction is the fastest and most transparent mode and is accurate on clean data, but crisp thresholds turn noise into false findings and a missing reading disables a rule. Abduction gave the largest gain in accuracy, and the largest gain in robustness to noise and to model error, because parsimony keeps weakly supported faults out of an explanation. Consistency checking adds the ability to reject explanations that leave a violated physical relation unexplained, and to separate sensor faults from equipment faults, at the cost of fragility when a relation's threshold is wrong or the noise is heavy. Over five seeds, that fragility costs it more exact matches than it gains, so the complete system ranks second to deductive + abductive reasoning. All modes remain tractable, at a few milliseconds per case.

RQ3: instrumentation faults. Yes, within the limits of the available redundancy. Treating sensors as components with fault models, and accepting a sensor fault only when assuming it restores an energy balance, let the system tell a biased sensor from a real fault. It also found real faults that a biased sensor was hiding. Every sensor fault in the held-out data was diagnosed correctly. The price is some false sensor alarms under heavy noise.

RQ4: explanation and maintenance. Yes. Every diagnosis in every configuration was accompanied by an explanation containing each element the framework requires, and by a sourced corrective plan. Plan interactions order the work when one repair depends on another, such as fixing a biased sensor before using it to verify a recirculation repair.

The central question. The project's evidence supports a qualified yes. Engineering knowledge of data centre cooling and power can be represented formally in Prolog and reasoned over to produce explainable, defensible diagnoses and appropriate maintenance decisions. The answer is qualified for two reasons. It has been demonstrated against a physics simulator, not yet against a real facility. And the model-mismatch experiment shows that the reasoning transfers to a different model while fixed thresholds do not, so a deployment would need its envelope thresholds calibrated against the site's own normal operation.

7.4 Contribution to Knowledge Representation and Reasoning

The project shows three classical theories of diagnosis working together on one engineering problem, and measures what each contributes. It shows that graded, noise-adaptive evidence lets abduction stay robust where crisp rules fail, without abandoning a symbolic, explainable core. It shows how fault models, combined with a hypothesis test on an energy balance, give consistency-based diagnosis the ability to exonerate components and separate sensors from equipment. It also documents the limits of that combination, in particular the fragility of hard consistency constraints to a mis-set relation threshold.

7.5 Engineering Contribution

For data centre engineering, the project provides a formal fault taxonomy for the cooling and power path of a row, a set of diagnostic signatures grounded in the physics, and analytical redundancy relations that can be checked from standard monitoring data. Examples are the outlet stratification that separates recirculation from a biased sensor, the valve-characteristic flow ratio that identifies chilled-water loss, and the rack power balance that separates an overload from a faulty current sensor. The physics simulator and its calibration and held-out protocol offer a way to test diagnostic knowledge before it is applied to a real facility.

7.6 Limitations and Future Work

The main limitation is that all test data are synthetic and share the knowledge base's engineering model. The most important next step is therefore validation on measured data with recorded faults. After that, the most valuable improvements are reasoning over time series rather than snapshots, sources for the remaining engineering assumptions, graded rather than hard consistency checking, a wider scope covering several rows and the chiller plant, and evaluation of the explanations by maintenance engineers. Section 6.6 lists the limitations in full and Section 6.7 the improvements.

7.7 Final Project Specification

Table 7.1 summarises the system as delivered.

Table 7.1: Final project specification.

Item	Specification
System	Cooling and power infrastructure of one data centre row: six racks of 20 servers, one chilled-water CRAH, one double-conversion UPS with VRLA battery, row and dual-feed rack PDUs
Fault classes	Normal, eight physical faults (F_1 to F_8) and two instrumentation faults (S_1, S_2)
Observation vector	26 variables, about 460 readings per case
Engineering model	60 tagged equations; 44 used by the code, 30 of them by the reasoning
Knowledge base	47 rules, 24 thresholds, 24 sources, 31 causal links, 20 redundancy relations, 14 fault models, 12 maintenance plans
Reasoning	Deductive, abductive and consistency-based, in four configurations
Explanation	Proof traces, supporting evidence, competing hypotheses with reasons for rejection, conflict sets; fidelity 96 to 100%
Maintenance	Sourced plan for every diagnosis, with plan interactions
Implementation	SWI-Prolog 10.0.2; about 1,900 lines of Prolog for the knowledge base and engines
Data	Synthetic: 4 smoke cases, 255 calibration cases, 255 held-out cases and 24 sensitivity sets of 60 cases, all regenerable from seeds
Performance, held-out	91.0% to 94.5% exact match; macro F1 up to 0.958; ExplainabilityRate 100%; about 1 ms per case
Performance, five seeds	90.8% to 93.6% exact match pooled over 1275 cases; standard deviation 1.2 to 1.6 points
Model mismatch	90 to 91% with abduction under mild mismatch; about 83% under strong mismatch after re-baselining three thresholds
Reproducibility	Generated tables; a 35-check reproduction script

Bibliography

Acton, M., Bertoldi, P., Booth, J., Newcombe, L., Rouyer, A., & Tozer, R. (2018). *2018 best practice guidelines for the EU code of conduct on data centre energy efficiency* (Version 9.1.0, EUR 29103 EN). Publications Office of the European Union. https://doi.org/10.2760/703236

Adadi, A., & Berrada, M. (2018). Peeking inside the black-box: A survey on explainable artificial intelligence (XAI). *IEEE Access*, *6*, 52138–52160. https://doi.org/10.1109/ACCESS.2018.2870052

Apt, K. R., & van Emden, M. H. (1982). Contributions to the theory of logic programming. *Journal of the ACM*, *29*(3), 841–862. https://doi.org/10.1145/322326.322339

Asgari, S., Gupta, R., Puri, I. K., & Zheng, R. (2021). A data-driven approach to simultaneous fault detection and diagnosis in data centers. *Applied Soft Computing*, *110*, 107638. https://doi.org/10.1016/j.asoc.2021.107638

ASHRAE. (2020). *2020 ASHRAE handbook: HVAC systems and equipment*.

ASHRAE Technical Committee 9.9. (2021). *Thermal guidelines for data processing environments* (5th ed.). ASHRAE.

Barroso, L. A., & Hölzle, U. (2007). The case for energy-proportional computing. *Computer*, *40*(12), 33–37. https://doi.org/10.1109/MC.2007.443

Barroso, L. A., Hölzle, U., & Ranganathan, P. (2018). *The datacenter as a computer: Designing warehouse-scale machines* (3rd ed.). Morgan & Claypool.

Bergman, T. L., Lavine, A. S., Incropera, F. P., & DeWitt, D. P. (2011). *Fundamentals of heat and mass transfer* (7th ed.). Wiley.

Brachman, R. J., & Levesque, H. J. (2004). *Knowledge representation and reasoning*. Morgan Kaufmann.

Bratko, I. (2012). *Prolog programming for artificial intelligence* (4th ed.). Pearson.

Buchanan, B. G., & Shortliffe, E. H. (Eds.). (1984). *Rule-based expert systems: The MYCIN experiments of the Stanford heuristic programming project*. Addison-Wesley.

Capozzoli, A., & Primiceri, G. (2015). Cooling systems in data centers: State of art and emerging technologies. *Energy Procedia*, *83*, 484–493. https://doi.org/10.1016/j.egypro.2015.12.168

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, *41*(3), 15:1–15:58. https://doi.org/10.1145/1541880.1541882

Clark, K. L. (1978). Negation as failure. In H. Gallaire & J. Minker (Eds.), *Logic and data bases* (pp. 293–322). Plenum Press. https://doi.org/10.1007/978-1-4684-3384-5_11

Colmerauer, A., & Roussel, P. (1993). The birth of Prolog. *The Second ACM SIGPLAN Conference on History of Programming Languages*, 37–52. https://doi.org/10.1145/154766.155362

Console, L., & Torasso, P. (1991). A spectrum of logical definitions of model-based diagnosis. *Computational Intelligence*, *7*(3), 133–141. https://doi.org/10.1111/j.1467-8640.1991.tb00388.x

Cordier, M.-O., Dague, P., Lévy, F., Montmain, J., Staroswiecki, M., & Travé-Massuyès, L. (2004). Conflicts versus analytical redundancy relations: A comparative analysis of the model based diagnosis approach from the artificial intelligence and automatic control perspectives. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, *34*(5), 2163–2177. https://doi.org/10.1109/TSMCB.2004.835010

CSB Energy Technology. (2026). *HRL12150WG datasheet: HRL series 12 V VRLA-AGM battery*. CSB Energy Technology. https://csb-battery.com/wp-content/uploads/2026/01/CSB-Datasheet-HRL12150WG-012026.pdf

Davis, R., Shrobe, H., & Szolovits, P. (1993). What is a knowledge representation? *AI Magazine*, *14*(1), 17–33. https://doi.org/10.1609/aimag.v14i1.1029

de Kleer, J., Mackworth, A. K., & Reiter, R. (1992). Characterizing diagnoses and systems. *Artificial Intelligence*, *56*(2–3), 197–222. https://doi.org/10.1016/0004-3702(92)90027-U

de Kleer, J., & Williams, B. C. (1987). Diagnosing multiple faults. *Artificial Intelligence*, *32*(1), 97–130. https://doi.org/10.1016/0004-3702(87)90063-4

Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning.

Ebrahimi, K., Jones, G. F., & Fleischer, A. S. (2014). A review of data center cooling technology, operating conditions and the corresponding low-grade waste heat recovery opportunities. *Renewable and Sustainable Energy Reviews*, *31*, 622–638. https://doi.org/10.1016/j.rser.2013.12.007

Fan, X., Weber, W.-D., & Barroso, L. A. (2007). Power provisioning for a warehouse-sized computer. *Proceedings of the 34th Annual International Symposium on Computer Architecture*, 13–23. https://doi.org/10.1145/1250662.1250665

Gao, Z., Cecati, C., & Ding, S. X. (2015). A survey of fault diagnosis and fault-tolerant techniques, Part I: Fault diagnosis with model-based and signal-based approaches. *IEEE Transactions on Industrial Electronics*, *62*(6), 3757–3767. https://doi.org/10.1109/TIE.2015.2417501

Giarratano, J. C., & Riley, G. D. (2005). *Expert systems: Principles and programming* (4th ed.). Thomson Course Technology.

Greiner, R., Smith, B. A., & Wilkerson, R. W. (1989). A correction to the algorithm in Reiter's theory of diagnosis. *Artificial Intelligence*, *41*(1), 79–88. https://doi.org/10.1016/0004-3702(89)90079-9

Hamscher, W., Console, L., & de Kleer, J. (Eds.). (1992). *Readings in model-based diagnosis*. Morgan Kaufmann.

Herrlin, M. K. (2005). Rack cooling effectiveness in data centers and telecom central offices: The rack cooling index (RCI). *ASHRAE Transactions*, *111*(2), 725–731.

House, J. M., Vaezi-Nejad, H., & Whitcomb, J. M. (2001). An expert rule set for fault detection in air-handling units. *ASHRAE Transactions*, *107*(1).

IEEE. (2005). *IEEE recommended practice for maintenance, testing, and replacement of valve-regulated lead-acid (VRLA) batteries for stationary applications*. IEEE.

Intel Corporation. (2024). *What is the maximum supported temperature for Intel Xeon Gold 6244 processor? (Article ID 000057788)*. Retrieved October 25, 2026, from https://www.intel.com/content/www/us/en/support/articles/000057788.html

International Electrotechnical Commission. (2020). *Electricity metering equipment: Particular requirements. Part 21: Static meters for AC active energy (classes 0,5, 1 and 2)*. IEC.

International Electrotechnical Commission. (2022). *Industrial platinum resistance thermometers and platinum temperature sensors*. IEC.

Isermann, R. (2005). Model-based fault-detection and diagnosis: Status and applications. *Annual Reviews in Control*, *29*(1), 71–85. https://doi.org/10.1016/j.arcontrol.2004.12.002

Isermann, R. (2006). *Fault-diagnosis systems: An introduction from fault detection to fault tolerance*. Springer. https://doi.org/10.1007/3-540-30368-5

Jackson, P. (1999). *Introduction to expert systems* (3rd ed.). Addison-Wesley.

Jardine, A. K. S., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, *20*(7), 1483–1510. https://doi.org/10.1016/j.ymssp.2005.09.012

Joint Committee for Guides in Metrology. (2008). *Evaluation of measurement data: Guide to the expression of uncertainty in measurement*. BIPM. https://doi.org/10.59161/JCGM100-2008E

Josephson, J. R., & Josephson, S. G. (1994). *Abductive inference: Computation, philosophy, technology*. Cambridge University Press.

Kakas, A. C., Kowalski, R. A., & Toni, F. (1992). Abductive logic programming. *Journal of Logic and Computation*, *2*(6), 719–770. https://doi.org/10.1093/logcom/2.6.719

Katipamula, S., & Brambley, M. R. (2005a). Methods for fault detection, diagnostics, and prognostics for building systems: A review, Part I. *HVAC&R Research*, *11*(1), 3–25. https://doi.org/10.1080/10789669.2005.10391123

Katipamula, S., & Brambley, M. R. (2005b). Methods for fault detection, diagnostics, and prognostics for building systems: A review, Part II. *HVAC&R Research*, *11*(2), 169–187. https://doi.org/10.1080/10789669.2005.10391133

Kim, W., & Katipamula, S. (2018). A review of fault detection and diagnostics methods for building systems. *Science and Technology for the Built Environment*, *24*(1), 3–21. https://doi.org/10.1080/23744731.2017.1318008

Kowalski, R. (1979). *Logic for problem solving*. North-Holland.

Liao, S.-H. (2005). Expert system methodologies and applications: A decade review from 1995 to 2004. *Expert Systems with Applications*, *28*(1), 93–103. https://doi.org/10.1016/j.eswa.2004.08.003

Lloyd, J. W. (1987). *Foundations of logic programming* (2nd ed.). Springer.

Masanet, E., Shehabi, A., Lei, N., Smith, S., & Koomey, J. (2020). Recalibrating global data center energy-use estimates. *Science*, *367*(6481), 984–986. https://doi.org/10.1126/science.aba3758

Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, *267*, 1–38. https://doi.org/10.1016/j.artint.2018.07.007

Moubray, J. (1997). *Reliability-centered maintenance* (2nd ed.). Industrial Press.

National Fire Protection Association. (2023). *NFPA 70: National electrical code*. NFPA.

Patankar, S. V. (2010). Airflow and cooling in a data center. *Journal of Heat Transfer*, *132*(7), 073001. https://doi.org/10.1115/1.4000703

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Peng, Y., & Reggia, J. A. (1990). *Abductive inference models for diagnostic problem-solving*. Springer.

Poole, D. (1989). Explanation and prediction: An architecture for default and abductive reasoning. *Computational Intelligence*, *5*(2), 97–110. https://doi.org/10.1111/j.1467-8640.1989.tb00319.x

Reiter, R. (1987). A theory of diagnosis from first principles. *Artificial Intelligence*, *32*(1), 57–95. https://doi.org/10.1016/0004-3702(87)90062-2

Robinson, J. A. (1965). A machine-oriented logic based on the resolution principle. *Journal of the ACM*, *12*(1), 23–41.

Rousseeuw, P. J., & Croux, C. (1993). Alternatives to the median absolute deviation. *Journal of the American Statistical Association*, *88*(424), 1273–1283. https://doi.org/10.1080/01621459.1993.10476408

Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, *1*(5), 206–215. https://doi.org/10.1038/s42256-019-0048-x

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Schein, J., Bushby, S. T., Castro, N. S., & House, J. M. (2006). A rule-based fault detection method for air handling units. *Energy and Buildings*, *38*(12), 1485–1492. https://doi.org/10.1016/j.enbuild.2006.04.014

Schmidt, R. R., Cruz, E. E., & Iyengar, M. K. (2005). Challenges of data center thermal management. *IBM Journal of Research and Development*, *49*(4/5), 709–723. https://doi.org/10.1147/rd.494.0709

Schroeder, B., & Gibson, G. A. (2007). Disk failures in the real world: What does an MTTF of 1,000,000 hours mean to you? *Proceedings of the 5th USENIX Conference on File and Storage Technologies*, 1–16.

Sharma, R. K., Bash, C. E., & Patel, C. D. (2002). Dimensionless parameters for evaluation of thermal design and performance of large-scale data centers. *8th AIAA/ASME Joint Thermophysics and Heat Transfer Conference*, (AIAA 2002-3091). https://doi.org/10.2514/6.2002-3091

Shehabi, A., Smith, S., Sartor, D., Brown, R., Herrlin, M., Koomey, J., Masanet, E., Horner, N., Azevedo, I. L., & Lintner, W. (2016). *United states data center energy usage report* (tech. rep. No. LBNL-1005775). Lawrence Berkeley National Laboratory.

Sterling, L., & Shapiro, E. (1994). *The art of Prolog* (2nd ed.). MIT Press.

Struss, P., & Dressler, O. (1989). "physical negation": Integrating fault models into the general diagnostic engine. *Proceedings of the 11th International Joint Conference on Artificial Intelligence*, 1318–1323.

Swartout, W. R. (1983). XPLAIN: A system for creating and explaining expert consulting programs. *Artificial Intelligence*, *21*(3), 285–325. https://doi.org/10.1016/S0004-3702(83)80014-9

U.S. Environmental Protection Agency. (n.d.). *Key product criteria: Uninterruptible power supplies (ENERGY STAR version 2.0)*. Retrieved October 25, 2026, from https://www.energystar.gov/products/uninterruptible_power_supplies/key_product_criteria

van Harmelen, F., Lifschitz, V., & Porter, B. (Eds.). (2008). *Handbook of knowledge representation*. Elsevier.

Venkatasubramanian, V., Rengaswamy, R., & Kavuri, S. N. (2003). A review of process fault detection and diagnosis. Part II: Qualitative models and search strategies. *Computers & Chemical Engineering*, *27*(3), 313–326.

Venkatasubramanian, V., Rengaswamy, R., Kavuri, S. N., & Yin, K. (2003). A review of process fault detection and diagnosis. Part III: Process history based methods. *Computers & Chemical Engineering*, *27*(3), 327–346.

Venkatasubramanian, V., Rengaswamy, R., Yin, K., & Kavuri, S. N. (2003). A review of process fault detection and diagnosis. Part I: Quantitative model-based methods. *Computers & Chemical Engineering*, *27*(3), 293–311.

Vishwanath, K. V., & Nagappan, N. (2010). Characterizing cloud computing hardware reliability. *Proceedings of the 1st ACM Symposium on Cloud Computing*, 193–204. https://doi.org/10.1145/1807128.1807161

Wang, J., Tian, Y., Qi, Z., Zeng, L., Wang, P., & Yoon, S. (2024). Sensor fault diagnosis and correction for data center cooling system using hybrid multi-label random forest and Bayesian inference. *Building and Environment*, *249*, 111124. https://doi.org/10.1016/j.buildenv.2023.111124

Wielemaker, J., Schrijvers, T., Triska, M., & Lager, T. (2012). SWI-Prolog. *Theory and Practice of Logic Programming*, *12*(1–2), 67–96. https://doi.org/10.1017/S1471068411000494